

LAPORAN TUGAS BESAR 2
IF3270 - PEMBELAJARAN MESIN
CONVOLUTIONAL NEURAL NETWORK DAN RECURRENT
NEURAL NETWORK



Disusun oleh:
Kelompok 8 - K03

Jonathan Kenan Budianto	13523139
Mahesa Fadhilah Andre	13523140
Muhammad Dzaky Atha Fadhilah	18223124

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026

DAFTAR ISI

DAFTAR ISI.....	2
1. Deskripsi Persoalan.....	4
1.1. Convolutional Neural Network (CNN).....	4
1.2. Simple Recurrent Neural Network (RNN) dan Long Short-Term Memory Network (LSTM).....	6
2. Implementasi.....	11
2.1. Deskripsi Kelas, Atribut, dan Method.....	11
2.1.1. Shared.....	11
2.1.2. CNN.....	30
2.1.3. RNN.....	40
2.1.4. LSTM.....	54
2.2. Forward Propagation.....	78
2.2.1. CNN.....	78
2.2.2. RNN.....	83
2.2.3. LSTM.....	84
2.3. Backward Propagation.....	85
2.3.1. CNN.....	85
2.3.2. RNN.....	91
2.3.3. LSTM.....	92
3. Pengujian.....	94
3.1. CNN.....	94
3.1.1. Perbandingan shared vs non-shared parameter.....	94
3.1.2. Pengaruh jumlah layer konvolusi.....	96
3.1.3. Pengaruh banyak filter per layer konvolusi.....	97
3.1.4. Pengaruh ukuran filter per layer konvolusi.....	98
3.1.5. Pengaruh jenis pooling layer.....	99
3.1.6. Perbandingan Keras vs from scratch.....	100
3.2. RNN dan LSTM.....	101
3.2.1. Perbandingan jumlah layer: RNN.....	101
3.2.2. Perbandingan jumlah layer: LSTM.....	102
3.2.3. Perbandingan ukuran hidden state: RNN.....	104
3.2.4. Perbandingan ukuran hidden state: LSTM.....	105
3.2.5. Perbandingan RNN vs LSTM.....	107
3.2.6. Perbandingan Keras vs from scratch: RNN.....	110
3.2.7. Perbandingan Keras vs from scratch: LSTM.....	111
3.2.8. Pengaruh panjang maksimum caption terhadap BLEU-4: RNN.....	111
3.2.9. Pengaruh panjang maksimum caption terhadap BLEU-4: LSTM.....	112

4. Kesimpulan dan Saran.....	114
4.1. Kesimpulan.....	114
4.2. Saran.....	115
5. Pembagian Tugas.....	116
REFERENSI.....	117
SURAT PERNYATAAN PENGGUNAAN AI.....	119

1. Deskripsi Persoalan

Tugas Besar II pada kuliah IF3270 Pembelajaran Mesin bertujuan agar peserta kuliah mendapatkan wawasan tentang bagaimana cara mengimplementasikan [Convolutional Neural Network \(CNN\)](#) dan [Recurrent Neural Network \(RNN\)](#). Pada tugas ini, peserta kuliah akan ditugaskan untuk mengimplementasikan modul *forward propagation* CNN, Simple RNN, dan LSTM *from scratch*. Selain itu, peserta juga akan mengeksplorasi pipeline *image captioning* melalui arsitektur encoder-decoder yang menggabungkan CNN dan RNN/LSTM, yang merupakan fondasi dari banyak sistem deep learning modern seperti LLM multimodal dan speech recognition.

1.1. Convolutional Neural Network (CNN)

Bagian CNN dikerjakan dalam konteks task *image classification* menggunakan dataset [Intel Image Classification](#) (~25.000 gambar, 6 kategori: buildings, forest, glacier, mountain, sea, street), dengan split *train*, *validation*, dan *test* yang sudah tersedia.

Arsitektur Model dan Implementasi

Model CNN minimal harus memiliki jenis layer berikut (urutan dan jumlah layer dapat disesuaikan):

- [Conv2D layer](#) Implementasi *shared parameter* dan non-shared parameter
- [Pooling layers](#)
- [Flatten/Global Pooling](#) layer
- [Dense layer](#)

Loss function: [Sparse Categorical Crossentropy](#). Optimizer: [Adam](#).

Tahapan Pengerjaan

• Bagian 1: Implementasi Utility Functions

Implementasikan utility functions berikut untuk pemrosesan data image menggunakan [PIL/Pillow](#) dan [NumPy](#) (tanpa Keras preprocessing):

- **Image loader**: load gambar dari file path menggunakan [PIL.Image.open](#), resize ke dimensi target, konversi ke [numpy array](#), dan normalisasi pixel values ke rentang [0, 1]
- **Batch loader**: load dan memproses sekumpulan gambar dari list file path menjadi numpy array dengan shape (N, H, W, C)
- **Feature extractor**: menerima list path gambar, menggunakan Keras CNN encoder (frozen) untuk mengekstraksi feature vectors, dan menyimpan hasilnya ke disk (format [.npy](#)) agar tidak perlu diekstraksi ulang

• Bagian 2: Implementasi Forward Propagation From Scratch

- Implementasikan modul *forward propagation from scratch* yang dapat membaca bobot hasil pelatihan Keras. Implementasikan secara modular: setiap layer memiliki method *forward* tersendiri yang menerima input numpy array dan mengembalikan output numpy array.
- Layer yang wajib diimplementasikan:
 - **Conv2D (shared parameters)**: operasi konvolusi 2D dengan sliding filter. Load bobot *kernel* (shape: [kH, kW, C_{in}, C_{out}]) dan *bias* dari

Keras. Implementasikan operasi: untuk setiap posisi (i,j) dan filter k, hitung dot product antara patch input dan kernel[k], tambahkan bias[k], lalu aplikasikan fungsi aktivasi.

- **LocallyConnected2D (non-shared parameters):** konvolusi tanpa parameter sharing. Load bobot *kernel* (shape: [output_rows*output_cols, kH*kW*C_in, C_out]) dan *bias* dari Keras.
- **MaxPooling2D / AveragePooling2D:** sliding window dengan operasi max atau average pada setiap channel.
Parameter: pool_size dan strides.
- **GlobalAveragePooling2D / GlobalMaxPooling2D:** reduksi spasial seluruh feature map menjadi satu nilai per channel (average atau max).
- **Flatten:** reshape tensor (H, W, C) menjadi vektor 1D dengan urutan [row-major \(C order\)](#), konsisten dengan Keras.
- **Fungsi aktivasi:** implementasikan [ReLU](#), [Softmax](#), dan aktivasi lain yang digunakan dalam model.
- Catatan: Khusus untuk **Dense layer**, Anda boleh menggunakan implementasi FFNN dari Tubes 1.
- Referensi implementasi:
 - [d2l.ai: Convolutions for Images \(implementasi dari nol\)](#)
 - [CS231n Notes: Convolutional Neural Networks](#)

● Bagian 3: Pelatihan Model

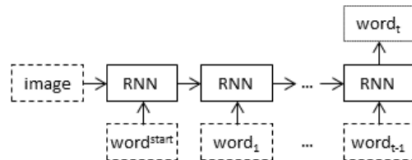
- Latih model CNN menggunakan Keras dengan **Conv2D** (shared parameters).
- Lakukan **variasi hyperparameter** berikut (hanya untuk model Conv2D / shared parameter):
 - Pengaruh **jumlah layer konvolusi**: pilih 3 2 variasi
 - Pengaruh **banyak filter per layer konvolusi**: pilih 3 2 variasi kombinasi
 - Pengaruh **ukuran filter per layer konvolusi**: pilih 3 2 variasi kombinasi
 - Pengaruh **jenis pooling layer**: pilih 2 variasi (max pooling atau average pooling)
- **Total ada 16 arsitektur yang perlu dieksperimenkan.**
- Catatan: Gunakan [macro F1-score](#) sebagai metrik perbandingan.
- Simpan bobot semua model hasil pelatihan.

● Bagian 4: Eksperimen dan Evaluasi

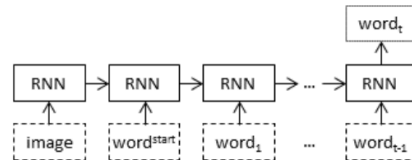
- Jalankan forward propagation from scratch menggunakan **arsitektur terbaik** hasil pelatihan **Bagian 3**, dengan dua variasi (**shared dan non-shared**):
 - Arsitektur awal.
 - Ganti semua **Conv2D** dengan **LocallyConnected2D**.
- Lakukan analisis:
 - Bandingkan hasil *forward propagation from scratch* (**cukup yang shared saja**) dengan Keras pada split data test menggunakan [macro F1-score](#).
 - Untuk setiap variasi hyperparameter dari Bagian 3: bandingkan hasil akhir prediksi dan grafik *training/validation loss*, serta berikan kesimpulan pengaruh hyperparameter tersebut.

- Untuk perbandingan **shared vs non-shared**: bandingkan macro F1-score, jumlah parameter, grafik *training/validation loss*, dan berikan kesimpulan bagaimana *parameter sharing* mempengaruhi kinerja dan efisiensi model.
- Berikan kesimpulan dan analisis perbedaan hasil jika ada.

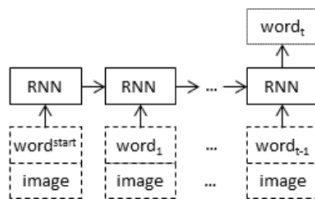
1.2. Simple Recurrent Neural Network (RNN) dan Long Short-Term Memory Network (LSTM)



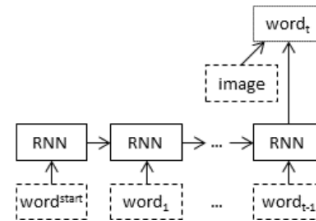
(a) Init-inject: The image vector is used as an initial hidden state vector for the RNN.



(b) Pre-inject: The image vector is used as a first word in the prefix.



(c) Par-inject: The RNN accepts two inputs at once in every time step: a word and an image.

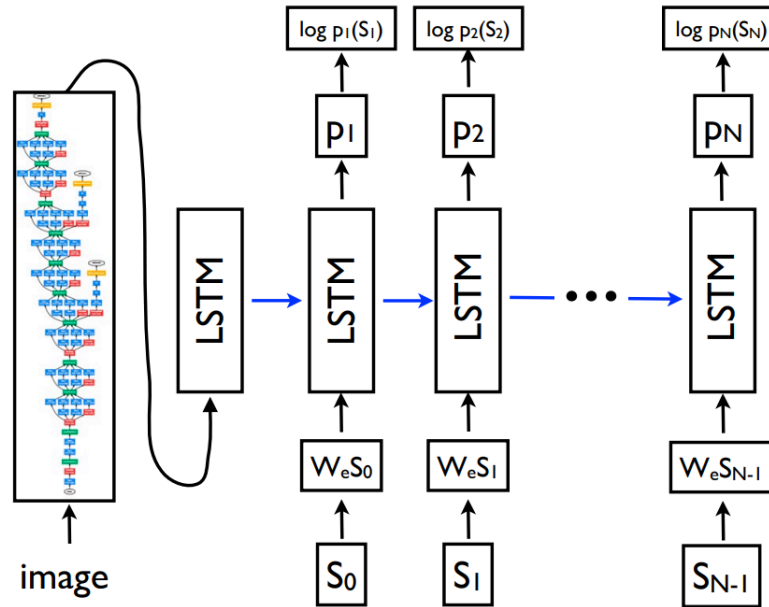


(d) Merge: The image vector is merged with the prefix outside of the RNN.

Gambar variasi arsitektur Image Captioning dari [paper \(Tanti et.al., 2017\)](#)

Bagian RNN dan LSTM dikerjakan dalam konteks task [image captioning](#) menggunakan dataset [Flickr8k](#) (8.092 gambar, 5 caption per gambar, split: 6.000 train / 1.000 validation / 1.000 test). Kedua arsitektur (RNN dan LSTM) dilatih secara terpisah sebagai *decoder* dan dibandingkan hasilnya.

Arsitektur Model



Gambar arsitektur dari [paper Show and Tell \(Vinyals et al., 2015\)](#)

Arsitektur mengacu pada [Show and Tell](#) (Vinyals et al., 2015), menggunakan *injection method pre-inject*: feature vector CNN di-project melalui [Dense layer](#) sebagai input x pada timestep $t=-1$, sebelum token **<start>**. Hidden state diinisialisasi dengan zeros (h_0). Untuk LSTM, c_0 diinisialisasi zeros. Snippet dari paper (hal. 4):

$$x_{-1} = \text{CNN}(I) \quad (10)$$

$$x_t = W_e S_t, \quad t \in \{0 \dots N-1\} \quad (11)$$

$$p_{t+1} = \text{LSTM}(x_t), \quad t \in \{0 \dots N-1\} \quad (12)$$

Arsitektur model Keras untuk masing-masing decoder:

- [Input layer](#) untuk caption sequence yang sudah di-prepend dengan feature vector CNN (shape: (seq_len+1, embed_dim))
- Untuk caption words: [Embedding layer](#): token $\rightarrow$ dense vector (embed_dim)
- Feature vector CNN di-project via [Dense layer](#) ke embed_dim, lalu di-concatenate di depan sequence embedding sebagai x_{-1}
- [SimpleRNN](#) atau [LSTM](#) dengan $h_0 = \text{zeros}$ (default Keras)
- [Dense output layer](#): $\rightarrow$ vocab_size, aktivasi softmax

Training menggunakan [teacher forcing](#): untuk tiap gambar, sequence input adalah

[CNN_feature, emb(<start>), emb(S_0), ..., emb(S_{N-1})],

dengan target output adalah [$S_0, S_1, \dots, S_N$] (digeser satu posisi).

Tahapan Pengerjaan

● Bagian 0: Implementasi Forward Propagation from Scratch

- Implementasikan secara modular: setiap layer memiliki method *forward* tersendiri yang menerima dan mengembalikan numpy array.
- Layer yang wajib diimplementasikan:

- **Embedding layer**
 - **SimpleRNN cell**
 - **LSTM cell**
 - **Dense projection layer:** load bobot dan bias dari Keras. Forward pass: $output = W \cdot x + b$, lalu project ke `embed_dim` untuk dijadikan x_{-1} .
 - **Dense output layer:** sama dengan Dense projection, dengan aktivasi softmax untuk menghasilkan distribusi probabilitas atas *vocabulary*.
- Catatan: Khusus untuk **Dense layer**, Anda boleh menggunakan implementasi FFNN dari Tubes 1.
- Referensi implementasi:
 - [d2l.ai: RNN from Scratch](#)
 - [d2l.ai: LSTM from Scratch](#)
 - [Keras LSTM weights format \(kernel, recurrent_kernel, bias\)](#)
- **Bagian 1: Feature Extraction (CNN Encoder, frozen):**
 - Gunakan **pretrained CNN** dari Keras (direkomendasikan [InceptionV3](#) atau [VGG16](#)) tanpa classification head (*top layer*) dan dengan bobot ImageNet yang di-freeze.
 - Lakukan *forward pass* seluruh gambar Flickr8k untuk **mengekstraksi feature vectors** dan **simpan** hasilnya ke disk (format .npy). Proses ini hanya dilakukan **sekali**, hasilnya digunakan bersama oleh decoder RNN dan LSTM.
 - Gunakan **utility functions image** yang sudah dibuat pada bagian CNN untuk image loading.
- **Bagian 2: Preprocessing Caption:**
 - Lakukan tokenisasi caption dan bangun vocabulary dari seluruh caption training dengan special tokens `<start>`, `<end>`, `<pad>`. Gunakan fungsi-fungsi berikut (**tidak perlu** diimplementasikan from **scratch**):
 - [tf.keras.layers.TextVectorization](#) atau Python [str.split\(\)](#): tokenisasi caption (mapping kata → integer)
 - Python built-in [str.lower\(\)](#), [re.sub\(\)](#): lowercase dan hapus tanda baca
 - [numpy.pad](#) atau [tf.keras.utils.pad_sequences](#): padding sequence ke panjang seragam
 - [numpy.save](#) / [json.dump](#): menyimpan vocabulary dict ke disk
 - Referensi preprocessing caption:
 - [TensorFlow Text: Tokenizers guide](#)
 - [NLTK tokenize](#): alternatif tokenizer berbasis Python
 - [GeeksforGeeks: Caption preprocessing untuk image captioning \(Flickr8k\)](#) (lihat bagian data preprocessing)
- **Bagian 3: Pelatihan Decoder menggunakan Keras (dilakukan dua kali: sekali untuk RNN, sekali untuk LSTM):**
 - Implementasikan arsitektur decoder sesuai deskripsi di atas. Loss function: [Sparse Categorical Crossentropy](#). Optimizer: [Adam](#).
 - Lakukan variasi pelatihan berikut untuk **masing-masing** decoder (**gunakan variasi yang sama untuk LSTM dan RNN**):

- Pengaruh **jumlah layer recurrent**: pilih 3 variasi (misal 1, 2, 3 layer)
 - Pengaruh **ukuran hidden state**: pilih 2 variasi (misal 128, 512)
 - Minimal ada **6 variasi** per LSTM dan RNN (**total 12 variasi**).
 - Nilai variasi bisa diubah, yang penting bisa menunjukkan pengaruhnya ke performa arsitektur.
- Simpan bobot semua model hasil pelatihan.
- **Bagian 4: Implementasi Arsitektur (dilakukan untuk kedua decoder, RNN dan LSTM):**
 - Load bobot CNN encoder dari Keras pretrained (**Bagian 1**) dan bobot decoder dari hasil pelatihan **Bagian 3** (terpisah untuk RNN dan LSTM).
 - Implementasikan 2 arsitektur, 1 dengan RNN, 1 dengan LSTM, menggunakan komponen yang sudah dibuat from scratch, sesuai arsitektur di atas dari raw image hingga menghasilkan caption.
- **Bagian 5: Eksperimen**
 - Jalankan **pipeline** untuk setiap variasi pada **Bagian 3**. Kalkulasi dan catat [BLEU-4 score](#) dan waktu eksekusi untuk setiap variasi.
 - Pilih 1 **variasi arsitektur terbaik** untuk masing-masing LSTM dan RNN. Lalu, implementasi dan jalankan **arsitektur keras** yang setara, catat score dan waktu eksekusi.
 - Pilih 1 **arsitektur terbaik** (berdasarkan score dan waktu eksekusi) antara ke 4 variasi (LSTM vs RNN, Keras vs Scratch). Lalu, lakukan variasi **panjang maksimum caption** yang dihasilkan, **minimal 3 variasi**, catat score.
- **Evaluasi dan Analisis**
 - a. **Variasi Jumlah Layer dan Hidden State**
 - Untuk setiap variasi jumlah layer dan ukuran hidden state dari **Bagian 3**:
 - Bandingkan [BLEU-4 score](#) dan [METEOR score](#) antara RNN dan LSTM pada split test Flickr8k
 - Bandingkan grafik training loss dan validation loss tiap epoch
 - Berikan kesimpulan bagaimana **jumlah layer** dan ukuran **hidden state** mempengaruhi kualitas caption dan waktu eksekusi.
 - b. **Perbandingan Keras vs Scratch**
 - Membandingkan score dan waktu eksekusi Keras vs from scratch
 - Bandingkan grafik training loss dan validation loss tiap epoch
 - Berikan kesimpulan dan asumsi alasan perbedaan jika ada
 - c. **Perbandingan RNN vs LSTM**
 - Membandingkan score dan waktu eksekusi antara decoder **RNN vs LSTM** (analisis utama)
 - Bandingkan grafik training loss dan validation loss tiap epoch
 - Lakukan **qualitative analysis**: tampilkan minimal 10 contoh gambar dengan **score yang bervariasi (tinggi, sedang, rendah)** beserta caption yang dihasilkan oleh decoder RNN dan LSTM, serta caption ground truth-nya. Identifikasi pola perbedaan antara keduanya.

- Berikan analisis: **mengapa satu arsitektur (LSTM/RNN) menghasilkan caption yang lebih baik dibandingkan arsitektur lain**, kaitkan dengan konsep [vanishing gradient](#) dan kemampuan memori jangka panjang.
- d. Pengaruh panjang maksimum caption**
- Bandingkan pengaruhnya terhadap BLEU-4 score.
- Beri kesimpulan.

Spesifikasi Bonus

Berikut merupakan beberapa spesifikasi bonus yang dapat Anda kerjakan:

- **[CNN] Visualisasi fitur:** implementasikan visualisasi [intermediate feature maps](#) dari layer konvolusi dan [Grad-CAM](#) untuk memvisualisasikan region gambar yang paling berpengaruh terhadap prediksi.
- **[RNN/LSTM] Image Captioning (init-inject):** implementasikan arsitektur alternatif di mana image feature dan caption context digabungkan (misalnya melalui Add atau Concatenate layer) setelah RNN/LSTM selesai memproses caption prefix. Bandingkan hasil BLEU-4 dengan versi pre-inject. Paper referensi: [\[Paper\]](#).
- **[RNN/LSTM] Beam Search Decoder:** implementasikan [beam search](#) sebagai alternatif greedy decoding ($k = 3$ atau $k = 5$). Bandingkan kualitas caption yang dihasilkan.
- **[Semua] Batch inference:** seluruh implementasi *forward propagation from scratch* harus bisa menangani kasus batch inference, di mana model dapat menerima lebih dari satu input untuk satu kali forward propagation. Jumlah instance dalam satu batch dapat diatur dengan hyperparameter *batch_size*.
- **[Semua] Backward propagation:** implementasikan fungsi *backward propagation from scratch* untuk seluruh layer yang digunakan.

2. Implementasi

2.1. Deskripsi Kelas, Atribut, dan Method

Pada bagian ini, akan dijelaskan mengenai deskripsi kelas apa saja yang diimplementasikan pada tugas ini beserta deskripsi atribut dan metodenya.

2.1.1. Shared

Pada bagian ini, akan dijelaskan mengenai berbagai implementasi kelas yang digunakan untuk implementasi CNN, RNN, dan LSTM.

- image_utils.py

image_utils.py berisikan metode-metode untuk memproses dataset gambar flickr8k. Berikut adalah beberapa metode yang diimplementasikan:

```
# fungsi load gambar dan resize ke dimensi 299x299
(input InceptionV3)
def load_image(path: str, target_dim: tuple = (299,
299)) -> np.ndarray:
    try:
        image = Image.open(path).convert("RGB")
        image_array =
np.array(image.resize(target_dim), dtype=np.float32)
        return preprocess_input(image_array)

    except Exception as e:
        print(f"Gagal memuat gambar {path}: {e}")
        return None
```

Fungsi load_image adalah fungsi utilitas yang dibuat untuk load dataset gambar Flickr8k dan melakukan beberapa *preprocessing* data agar gambar dapat dimasukkan sebagai input ke model CNN. Fungsi ini mengubah file path gambar menjadi array NumPy yang sudah dinormalisasi dan memiliki dimensi seragam agar dapat digunakan untuk training RNN dan LSTM. Fungsi ini menggunakan parameter default untuk dimensi gambar yaitu (299, 299) yang merupakan dimensi input standar untuk model pretrained *InceptionV3* yang kami gunakan untuk feature extraction dataset gambar.

```
# fungsi load batch gambar, mengembalikan array 4D
(batch_size, height, width, channels)
def load_batch(paths: list[str], target_dim: tuple =
(299, 299)) -> np.ndarray:
    batch = []
    valid_paths = [] # path yang berhasil dimuat

    for path in paths:
        img = load_image(path, target_dim)
        if img is not None:
            batch.append(img)
            valid_paths.append(path)

    return np.array(batch), valid_paths
```

Fungsi `load_batch` digunakan untuk memproses sekumpulan gambar sekaligus dalam sebuah batch dan mengembalikan menjadi 1 array berdimensi 4 yang terdiri dari (batch_size, height, width, channels) sebagai input ke model CNN. Fungsi melakukan iterasi pada setiap path dan memanggil fungsi `load_image` untuk memuat, mengubah ukuran, dan menormalisasi setiap gambar. Semua gambar yang berhasil dikumpulkan dikonversi menjadi satu NumPy array 4D untuk proses Batch inference selama feature extraction ataupun training.

```
# fungsi ekstrak fitur CNN dan simpan ke dictionary
.npy
def extract_features(paths: list[str], output_path:
str, batch_size: int = 64, encoder=None):
    # buat folder output
    output_dir =
os.path.dirname(os.path.abspath(output_path))
    if output_dir:
        os.makedirs(output_dir, exist_ok=True)

    if encoder is None:
        encoder = InceptionV3(weights='imagenet',
include_top=False, pooling='avg')
```

```

        target_dim = encoder.input_shape[1:3]
        if target_dim == (None, None): # fallback
            input_shape = None
            target_dim = (299, 299) # default InceptionV3

        # dictionary untuk menyimpan fitur, key: nama file,
        # value: vektor fitur
        features_dict = {}

        # ekstraksi fitur per batch untuk menghindari Out
        # of Memory
        print(f"Memulai ekstraksi fitur untuk {len(paths)}
        gambar (Batch size: {batch_size})...")
        for i in tqdm(range(0, len(paths), batch_size),
            desc="Extracting"):
            batch_paths = paths[i : i + batch_size]
            batch_images, valid_paths =
            load_batch(batch_paths, target_dim)

            # prediksi hanya jika ada gambar yang valid di
            # batch ini
            if len(batch_images) > 0:
                batch_features =
                encoder.predict(batch_images, verbose=0)

                # mapping filename ke vektor fitur
                for path, feature in zip(valid_paths,
                batch_features):
                    filename = os.path.basename(path) #
                    ambil nama file saja
                    features_dict[filename] = feature

        # simpan dictionary fitur ke file .npz
        np.save(output_path, features_dict)
        print(f"\nEkstraksi fitur selesai. Berhasil
        menyimpan {len(features_dict)} fitur ke {output_path}")

        return features_dict

```

Fungsi `extract_features` adalah implementasi utama dari **Bagian 1: Feature Extraction** dalam pipeline *Image Captioning*. Tujuannya adalah untuk membuat *feature vectors* (vektor fitur) dari sekumpulan gambar menggunakan CNN Encoder pretrained dan menyimpannya ke dalam format `.npy` agar dapat digunakan kembali oleh decoder RNN/LSTM. Fungsi ini secara default akan memuat model Keras **InceptionV3** dengan parameter `include_top=False` memastikan classification head/lapisan output dihilangkan dan `pooling='avg'` agar Global Average Pooling untuk mereduksi feature map spasial menjadi vektor fitur berdimensi 2048. Fungsi ini kemudian memproses *list of image paths* secara berulang dalam batch dan untuk setiap batch, fungsi memanggil `load_batch` untuk memuat, mengubah ukuran, dan menormalisasi gambar. Untuk batch yang berhasil dimuat, `encoder.predict()` dijalankan pada batch gambar dan menghasilkan sekumpulan vektor fitur. Vektor fitur yang dihasilkan dipetakan ke nama file aslinya dan disimpan dalam dictionary. Setelah semua gambar diproses, dictionary ini disimpan ke jalur `output_path` sebagai file `.npy`.

- `preprocessing.py`
`preprocessing.py` berisikan metode-metode untuk memproses dataset caption flickr8k menjadi format json. Berikut adalah beberapa metode yang diimplementasikan:

```
# fungsi untuk load dan preprocess caption.txt dari
dataset Flickr8k
def load_captions(filepath: str) -> dict[str,
list[str]]:
    # dictionary untuk menyimpan mapping image_id dan
    list of captions
    mapping = {}

    # membaca file caption.txt
    with open(filepath, "r", encoding="utf-8",
newline="") as file:
        # menggunakan CSV reader agar caption yang
        punya koma tetap terbaca utuh
        # contoh: "A dog shakes its head near the shore
        , a red ball next to it ."
        rows = csv.reader(file, delimiter='\t') #
        delimiter tab karena file menggunakan tab sebagai
```

```

pemisah

    # skip header: image,caption
    next(rows, None)

    # iterasi tiap baris
    for row in rows:
        # validasi jumlah kolom (minimal 2) agar
        # tidak error saat split
        if len(row) < 2:
            continue

        # ekstrak nama file gambar dan caption
        image_id = row[0].strip()
        caption = row[1].strip()

        # lewati baris kosong atau data hilang
        if not image_id or not caption:
            continue

        # simpan caption ke list per image (1 image
        # bisa punya > 1 caption)
        mapping.setdefault(image_id,
        []).append(caption)

    return mapping

```

Fungsi `load_captions` berfungsi sebagai langkah awal **Bagian 2: Preprocessing Caption** untuk mengambil data dari `captions.txt` dari dataset Flickr8k dan menyusunnya ke dalam format dictionary. Fungsi ini mengubah file teks yang berisi pasangan `image_id` dan `caption` menjadi sebuah dictionary dimana setiap `image_id` dipetakan ke sebuah list yang berisi semua `caption` yang dimilikinya. Hal ini dilakukan karena 1 gambar dalam dataset Flickr8k memiliki 5 `caption`. Fungsi ini membaca file menggunakan pembacaan csv dengan `delimiter='\t'` (tab) untuk memastikan `caption` yang mengandung koma tetap terbaca sebagai satu kolom. Setelah melewati header, fungsi mengiterasi setiap baris dan mengelompokkan semua `caption` ke dalam satu list untuk `image_id` yang

sesuai. Hasil akhirnya adalah dictionary yang siap untuk proses pembersihan dan tokenisasi berikutnya.

```
# fungsi untuk membersihkan caption serta menambahkan
token <start> dan <end>
def clean_and_wrap_captions(caption_mapping: dict[str,
list[str]]) -> dict[str, list[str]]:
    # dictionary baru untuk menyimpan hasil caption
    yang sudah dibersihkan dan ditambahkan token
    cleaned_mapping = {}

    # tabel translasi untuk menghapus tanda baca
    table = str.maketrans('', '', string.punctuation)

    # iterasi tiap image_id dan list of captions
    for key, captions in caption_mapping.items():
        # iterasi tiap caption untuk image tersebut
        for i in range(len(captions)):
            caption = captions[i]
            caption = caption.lower() # lowercase
            caption = caption.translate(table) # hapus
tanda baca menggunakan tabel translasi
            caption = f"<start> {caption} <end>" # wrap
dengan start & end token

            # jika image_id belum ada di
cleaned_mapping, buat list baru
            if key not in cleaned_mapping:
                cleaned_mapping[key] = []

            # tambahkan caption yang sudah dibersihkan
dan ditambahkan token ke list untuk image tersebut
            cleaned_mapping[key].append(caption)

    return cleaned_mapping
```

Fungsi `clean_and_wrap_captions` dibuat untuk menormalisasi dan mendefinisikan batas-batas urutan kata untuk pelatihan decoder RNN/LSTM. Fungsi ini menerima dictionary yang memetakan id gambar

ke daftar caption dan mengembalikan dictionary baru yang berisi caption yang sudah diproses. Fungsi ini akan membuat semua caption memiliki format yang konsisten dan menyertakan token batas yang diperlukan oleh model decoder. Pertama-tama, dibuat sebuah tabel translasi yang digunakan untuk menghapus semua tanda baca dari caption. Setiap caption juga dibuat menjadi huruf kecil untuk memastikan konsistensi dan mengurangi kompleksitas vocabulary. Kemudian, caption ditambahkan dengan token spesial yaitu <start> di awal dan <end> di akhir yang penting untuk model decoder sebagai penanda awal dan akhir urutan kata selama proses training dan generation. Fungsi ini akan mengembalikan dictionary berisikan caption yang sudah bersih dan id gambar sesuai yang siap untuk melalui proses tokenisasi berikutnya dimana setiap kata akan diubah menjadi indeks integer.

```
# fungsi untuk membangun vocabulary & tokenizer dari
seluruh caption yang sudah dibersihkan
def build_tokenizer(cleaned_mapping,
output_path="tokenizer.json"):
    # kumpulkan semua caption menjadi satu list panjang
    all_captions = []
    for key in cleaned_mapping:
        for caption in cleaned_mapping[key]:
            all_captions.append(caption)

    # menghapus tanda selain < dan > pada filter agar
    token <start> dan <end> tetap terjaga
    filters = '!"#$%&()*+,-./:;=?@[\\]^_`{|}~\t\n'

    # Tokenizer Keras otomatis menambahkan <pad>
    sebagai token 0 (Zero Padding)
    tokenizer = Tokenizer(filters=filters,
oov_token="<unk>")
    tokenizer.fit_on_texts(all_captions)

    # simpan tokenizer dalam format JSON (Keras
    to_json)
    tokenizer_json = tokenizer.to_json()
    with open(output_path, "w", encoding="utf-8") as f:
        f.write(json.dumps(tokenizer_json,
```

```

ensure_ascii=False))

    # hitung ukuran vocabulary (jumlah token unik + 1
    untuk padding)
    vocab_size = len(tokenizer.word_index) + 1
    print(f"Ukuran Vocabulary: {vocab_size}")
    print(f"Tokenizer berhasil disimpan di
    {output_path}")

    return tokenizer, all_captions

```

Fungsi `build_tokenizer` digunakan untuk membangun *vocabulary* dan membuat objek `Tokenizer Keras` yang memetakan setiap kata unik menjadi nilai integer. Fungsi ini akan menyiapkan data teks agar dapat diproses oleh RNN/LSTM dengan mengubah kata-kata menjadi urutan numerik/token. Pertama-tama, fungsi mengumpulkan semua caption dari semua gambar yang telah dimapping ke dalam satu list. Untuk pembuatan token, terdapat beberapa filter tanda baca yang spesifik untuk memastikan bahwa semua tanda baca dihilangkan kecuali karakter "<" dan ">" di dalam token <start> dan <end> yang penting untuk decoder. Fungsi kemudian membuat objek `tf.keras.preprocessing.text.Tokenizer` dengan filter tersebut dan `oov_token="<unk>"` untuk menangani kata-kata yang tidak dikenal. Metode `fit_on_texts` kemudian memindai data dan membuat kamus/*word_index* yang memberikan indeks integer unik pada setiap kata. Objek tokenizer kemudian dikonversi ke format JSON dan disimpan dengan nama `tokenizer.json`. Terakhir, ukuran akhir vocabulary akan dihitung dan hasilnya akan menentukan dimensi input untuk Embedding layer.

```

# fungsi untuk menghitung panjang maksimum dari seluruh
caption (untuk keperluan padding)
def calculate_max_length(all_captions):
    # iterasi seluruh caption dan hitung jumlah kata
    lalu cari yang paling panjang
    max_length = max(len(caption.split()) for caption
    in all_captions)
    print(f"Maximum Caption Length: {max_length}")

```

```
return max_length
```

Fungsi `calculate_max_length` digunakan untuk menghitung panjang caption terpanjang yang terdapat pada dataset. Fungsi akan mengiterasi semua caption dan mengembalikan nilai panjang terbesar untuk digunakan pada padding nantinya.

```
# fungsi untuk mengubah caption menjadi sekuens integer
dan padding
def captions_to_sequences_and_pad(tokenizer,
all_captions, max_length):
    # ubah caption menjadi urutan integer menggunakan
tokenizer
    sequences =
tokenizer.texts_to_sequences(all_captions)

    # padding semua caption agar memiliki panjang yang
sama
    padded =
tf.keras.preprocessing.sequence.pad_sequences(sequences
, maxlen=max_length, padding="post")

    return sequences, padded
```

Fungsi `captions_to_sequences_and_pad` digunakan untuk mengubah teks deskriptif menjadi format numerik yang menjadi input model RNN dan LSTM. Pertama-tama, fungsi akan melakukan konversi ke sekuens dengan mengganti setiap kata dalam seluruh caption (`all_captions`) dengan indeks integer yang sesuai dari vocabulary yang telah dibuat sebelumnya. Semua sekuens integer kemudian dilakukan padding menggunakan `tf.keras.preprocessing.sequence.pad_sequences` agar memiliki panjang yang seragam yaitu `max_length`. Parameter `padding="post"` memastikan bahwa nilai nol yang merepresentasikan token `<pad>` ditambahkan di akhir sekuens. Tujuan dari padding adalah untuk memastikan setiap input memiliki shape yang konsisten untuk batch training pada RNN dan LSTM.

```
# fungsi untuk menyimpan word_index ke JSON utk dipakai
```

```

di pipeline NumPy
def save_tokenizer_json(tokenizer,
output_path="tokenizer.json"):
    # ambil string JSON yang merepresentasikan
    keseluruhan objek Tokenizer
    tokenizer_json_str = tokenizer.to_json()

    with open(output_path, "w", encoding="utf-8") as f:
        # parse lalu dump lagi agar JSON mudah dibaca
        json.dump(json.loads(tokenizer_json_str), f,
ensure_ascii=False, indent=2)

    print(f"Tokenizer berhasil disimpan di
{output_path}")

```

Fungsi `save_tokenizer_json` digunakan untuk menyimpan objek Tokenizer Keras ke dalam format JSON. Fungsi ini memastikan konsistensi mapping kata ke indeks dan *reproducibility*. Setelah model dilatih, mapping yang sama harus digunakan saat memprediksi caption untuk gambar baru. Dengan menyimpan Tokenizer, decoder dapat secara akurat mengubah prediksi indeks token kembali menjadi kata-kata yang dapat dibaca. Proses penyimpanan melibatkan konversi objek Tokenizer menjadi string JSON yang di-parse dan di-dump ke file (`output_path`) dengan format yang mudah dibaca (pretty print).

- `dense.py`
 - `dense.py` berisikan kelas `DenseLayer` yang merupakan implementasi **Fully Connected (FC) Layer** atau **Dense Layer**. Dalam persoalan image captioning, layer ini digunakan untuk:

 - **Proyeksi Fitur CNN**
Layer ini mengubah dimensi vektor fitur CNN ke dimensi embedding layer sebelum dimasukkan ke sel RNN/LSTM (pre-inject)
 - **Output Layer**
Pada output layer, akan dilakukan perubahan hidden state RNN/LSTM menjadi vektor probabilitas berdimensi `vocab_size` dengan aktivasi Softmax untuk memprediksi token berikutnya. Berikut adalah implementasinya dari kelas `DenseLayer`:

```

class DenseLayer:
    def __init__(self, keras_layer=None,
input_size=None, output_size=None):
        if keras_layer is not None:
            weights = keras_layer.get_weights()
            self.weights = weights[0]
            self.bias = weights[1] if len(weights) > 1
else np.zeros(weights[0].shape[1])
        else:
            if input_size is not None and output_size
is not None:
                # inisialisasi bobot dengan metode
He/Xavier initialization
                self.weights =
np.random.randn(input_size, output_size) * np.sqrt(2. /
input_size)
                self.bias = np.zeros(output_size) #
bias diinisialisasi ke nol
            else:
                raise ValueError("NO WEIGHTS TO
INITIALIZE")

        # gradien bobot dan bias
        self.dW = None
        self.db = None

    def forward(self, x):
        self.x = x
        return x @ self.weights + self.bias

    def backward(self, grad_out):
        # gradien input x
        # nilai ini akan diteruskan ke layer sebelumnya
(RNN/LSTM)
        dL_dx = grad_out @ self.weights.T

        # gradien bobot: outer product untuk input 1D
(single sample)

```

```

        self.dW = np.outer(self.x, grad_out) # (in_d,
out_d)

        # gradien bias
        self.db = grad_out

    return dL_dx

```

Atribut dan metode utama:

- **__init__ (Konstruktor)**

Pada konstruktor, jika parameter keras_layer tersedia, fungsi akan memuat bobot (self.weights) dan bias (self.bias) yang sudah dilatih dari objek Keras. Jika keras_layer tidak tersedia, bobot diinisialisasi secara acak menggunakan metode inisialisasi He/Xavier dan bias diinisialisasi ke nol. Kemudian, dilakukan juga inisialisasi self.dW dan self.db untuk menyimpan gradien bobot dan bias selama backpropagation.

- **forward(self, x)**

Metode ini digunakan untuk operasi dasar forward pass layer Dense yaitu **Output = $X \cdot W + B$** . Input X dikalikan dengan matriks bobot W (self.weights) menggunakan operasi perkalian matriks dan ditambahkan dengan vektor bias B (self.bias). Input X disimpan dalam self.x untuk perhitungan gradien pada tahap backpropagation.

- **backward(self, grad_out)**

Metode ini digunakan untuk backpropagation dan menerima gradien loss dari layer berikutnya (grad_out). Pertama-tama, nilai gradien input dihitung dari perkalian grad_out dengan matriks bobot W transpose. Nilai ini akan diteruskan ke layer sebelumnya seperti sel RNN/LSTM atau Convolutional Layer. Kemudian, dilakukan perhitungan gradien bobot dengan mengalikan grad_out dengan matriks input X transpose. Dalam implementasinya, np.outer(self.x, grad_out) digunakan untuk sampel tunggal yang ekuivalen dengan outer product antara input dan gradien yang masuk. Untuk gradien bias, nilainya sama dengan grad_out.

- embedding_layer.py

embedding_layer.py berisikan kelas EmbeddingLayer yang merupakan implementasi dari Layer **Embedding** yang berfungsi sebagai lookup table untuk mengubah token input berupa indeks integer menjadi vektor kontinu. Lapisan ini berperan penting dalam konteks Natural Language Processing (NLP) untuk memungkinkan model RNN/LSTM memahami hubungan semantik antar kata. Berikut adalah implementasinya:

```
class EmbeddingLayer:
    """
    A layer that has the function of embedding tokens
    into vectors.
    """
    def __init__(self, keras_layer=None,
vocab_size=None, embed_dim=None):
        """
        vocab_size: the total number of the words the
model can learn
        embed_dim: the dimension of the vector
embedding
        """
        if keras_layer is not None:
            self.weights = keras_layer.get_weights()[0]
        else:
            if vocab_size is not None and embed_dim is
not None:
                self.weights =
np.random.randn(vocab_size, embed_dim)
            else:
                raise ValueError("NO WEIGHTS TO
INITIALIZE")

        self.embed_dim = embed_dim if embed_dim is not
None else self.weights.shape[1]

    def forward(self, token: int):
        self.token = token
        return self.weights[token]

    def backward(self, grad_out):
```

```

        # matriks graiden kosong berukuran sama dengan
matriks embedding
        self.dW = np.zeros_like(self.weights)

        # hanya update baris yang digunakan (token)
dengan gradien yang diterima
        self.dW[self.token] = grad_out

        # karena Embedding layer adalah layer paling
bawah/input
        # tidak ada gradien untuk diteruskan ke layer
sebelumnya
        return None

```

Atribut dan metode utama:

- **__init__ (Konstruktor)**

Pertama-tama, konstruktor menginisialisasi/memuat matriks bobot utama (self.weights) dengan dimensi (vocab_size, embed_dim). Matriks ini adalah tabel lookup dimana setiap baris mewakili satu kata unik dalam kosakata. Jika keras_layer tersedia, bobot dari model Keras yang sudah dilatih akan dimuat dan jika tidak, bobot diinisialisasi secara acak.

- **forward(self, token: int)**

Fungsi ini merupakan implementasi dari forward pass embedding layer. Fungsi menerima input berupa indeks integer dari token kata dan melakukan operasi lookup dengan mengambil/slice baris vektor dari matriks self.weights yang sesuai dengan indeks token yang diberikan (self.weights[token]). Vektor yang dikembalikan adalah representasi dense dari kata tersebut yang akan menjadi input untuk sel RNN/LSTM berikutnya.

- **backward(self, grad_out)**

Fungsi ini adalah implementasi dari backpropagation pada embedding layer. Fungsi akan menghitung gradien bobot (self.dW) yang akan digunakan oleh optimizer. Mekanisme backpropagation pada layer ini berbeda yaitu hanya menghitung dan memperbarui gradien pada baris/vektor yang digunakan dalam forward pass (self.dW[self.token] = grad_out) dan baris lainnya tetap nol. Fungsi ini akan mengembalikan

None karena merupakan layer input terendah sehingga tidak ada gradien yang perlu diteruskan kembali ke lapisan sebelumnya.

- activation_functions.py

activation_functions.py berisikan kumpulan fungsi aktivasi yang digunakan untuk membantu implementasi CNN, RNN dan LSTM. Berikut adalah beberapa fungsi aktivasi yang diimplementasikan:

```
def sigmoid(x):  
    return 1 / (1 + np.exp(-x))
```

Fungsi sigmoid digunakan dalam komponen **gates** LSTM yaitu forget gate, input gate, dan output gate. Fungsi ini mengatur dan menentukan seberapa banyak informasi dari hidden state atau cell state sebelumnya yang harus dipertahankan atau dilupakan pada setiap time step dengan mengubah input nilai real ke dalam rentang antara 0 dan 1.

```
def tanh(x):  
    return (np.exp(x) - np.exp(-x)) / (np.exp(x) +  
np.exp(-x))
```

Fungsi Tanh digunakan sebagai fungsi aktivasi utama pada hidden state (h_t) di dalam sel SimpleRNN dengan mengubah nilai input ke rentang -1 hingga 1. Fungsi ini juga digunakan pada input gate LSTM untuk menentukan kandidat nilai baru ($\tilde{C}_t$) yang akan ditambahkan ke cell state dengan memberikan bobot antara -1 hingga 1 pada informasi baru.

```
def relu(x):  
    return np.maximum(0, x)
```

Fungsi ReLU diterapkan pada CNN di layer konvolusi dan juga pada lapisan Dense yang memproyeksikan feature vector dari CNN InceptionV3 sebelum pre-inject ke decoder RNN/LSTM. Fungsi ini memberikan output nilai input itu sendiri jika positif dan nol jika negatif.

```
def softmax(x):  
    x = x - np.max(x, axis=-1, keepdims=True)  
    e = np.exp(x)
```

```
return e / np.sum(e, axis=-1, keepdims=True)
```

Fungsi Softmax digunakan pada Dense output layer dari model untuk klasifikasi multi-kelas. Fungsi ini akan menormalisasi sekelompok angka (*logits*) menjadi distribusi probabilitas yang jika dijumlahkan bernilai tepat 1. Pada implementasi CNN encoder, fungsi ini memprediksi probabilitas setiap kelas gambar. Sedangkan pada implementasi RNN & LSTM decoder, fungsi memprediksi probabilitas setiap kata dalam vocabulary (*vocab_size*) sebagai token berikutnya dalam caption.

- `loss_function.py`

`loss_function.py` berisikan kelas `SparseCategoricalCrossEntropy` yang berfungsi sebagai loss function utama yang menghitung error antara distribusi probabilitas kata yang diprediksi oleh decoder RNN/LSTM dengan target kata yang sebenarnya. Fungsi ini ditujukan untuk klasifikasi multi-kelas dengan label integer (*sparse labels*) dan dalam persoalan image captioning, label tersebut adalah indeks integer dari kata berikutnya dalam vocabulary. Berikut adalah implementasinya:

```
# Loss function Sparse Categorical Cross-Entropy untuk
prediksi kata berikutnya dalam caption
class SparseCategoricalCrossEntropy:
    def __init__(self):
        pass

    def forward(self, y_pred, y_true_index):
        """
        y_pred: numpy array shape (1, vocab_size) hasil
dari Softmax
        y_true_index: indeks dari kata yang benar
        """
        # clipping untuk mencegah log(0) yang
menghasilkan NaN
        y_pred_clipped = np.clip(y_pred, 1e-15, 1 -
1e-15).ravel()

        # probabilitas tebakan pada kelas yang benar
        correct_class_probabilities =
y_pred_clipped[y_true_index]
```

```

        # Loss: -log likelihood
        loss = -np.log(correct_class_probabilities)

        return loss

    def backward(self, y_pred, y_true_index):
        """
        Turunan Softmax + CCE:  $dZ = Y_{pred} - Y_{true}$ 
        """
        d_logits = y_pred.copy().ravel()

        # karena y_true adalah 1 hanya pada
        y_true_index,
        # (y_pred - y_true) sama dengan mengurangi 1 di
        indeks yang benar
        d_logits[y_true_index] -= 1.0

        return d_logits

```

Atribut dan metode utama:

- **__init__(self)**
Konstruktor tidak menerima parameter karena SparseCategoricalCrossEntropy adalah fungsi loss statis.
- **forward(self, y_pred, y_true_index)**
Fungsi ini menghitung nilai loss berdasarkan target yang benar. Fungsi menerima input y_pred yaitu vektor probabilitas hasil aktivasi Softmax dengan ukuran (1, vocab_size) dan y_true_index yaitu indeks integer dari token kata target yang benar. Pertama-tama, nilai y_pred di-clipping ke rentang $1e-15$ hingga $1 - 1e-15$ untuk mencegah nilai logaritma tak terdefinisi ($-\infty$) jika probabilitas yang diprediksi adalah 0. Hanya probabilitas yang diprediksi untuk kelas/indeks benar yang akan diambil. Loss dihitung menggunakan rumus Negative Log-Likelihood (NLL) yaitu **$-\text{np.log}(P_{\text{correct}})$** . Semakin rendah probabilitas (P_{correct}) maka semakin tinggi pula nilai loss.
- **backward(self, y_pred, y_true_index)**
Fungsi ini menghitung gradien d_logits untuk diteruskan kembali pada saat backpropagation ke layer Dense output dan sel RNN/LSTM.

Turunan gabungan dari Softmax dan Sparse Categorical Cross-Entropy (SCCE) memiliki bentuk sederhana yaitu selisih antara probabilitas prediksi (y_{pred}) dan label target one-hot (y_{true}). Implementasi ini mengambil y_{pred} lalu mengurangi nilai 1 tepat pada indeks kelas yang benar. Implementasi ini secara efektif menghasilkan vektor gradien yang dibutuhkan oleh layer sebelumnya tanpa perlu membuat vektor one-hot secara eksplisit.

- optimizer.py

optimizer.py berisikan kelas AdamOptimizer yang merupakan implementasi *optimizer update* bobot yang digunakan pada persoalan *image captioning* ini yaitu **Adam (Adaptive Moment Estimation)**. Optimizer Adam dirancang untuk menggabungkan **Momentum** (rata-rata bergerak dari gradien) dan **RMSprop** (rata-rata bergerak dari gradien kuadrat) sehingga dapat menyesuaikan *learning rate* secara individual untuk setiap parameter serta menghasilkan konvergensi yang lebih cepat dan stabil. Berikut adalah implementasinya:

```
# implementasi Adam optimizer utk backpropagation
class AdamOptimizer:
    def __init__(self, learning_rate=0.001, beta1=0.9,
beta2=0.999, epsilon=1e-8):
        self.lr = learning_rate
        self.beta1 = beta1
        self.beta2 = beta2
        self.epsilon = epsilon

        # dictionary utk menyimpan moving averages
        # (moment 1 dan moment 2)
        self.m = {}
        self.v = {}
        self.t = 0 # timestep/jumlah iterasi update

    def step(self, params_dict, grads_dict):
        """
        params_dict: dictionary referensi ke bobot
        model (misal: {'W_rnn': rnn.W})
        grads_dict: dictionary gradien yang sesuai
        (misal: {'W_rnn': dW_rnn})
        """
```

```

        self.t += 1 # tambah iterasi

        for key in params_dict.keys():
            param = params_dict[key]
            grad = grads_dict[key]

            # inisialisasi momentum dengan 0 pada awal
training
            if key not in self.m:
                self.m[key] = np.zeros_like(param)
                self.v[key] = np.zeros_like(param)

            # 1st & 2nd moment estimate
            self.m[key] = self.beta1 * self.m[key] + (1
- self.beta1) * grad
            self.v[key] = self.beta2 * self.v[key] + (1
- self.beta2) * (grad ** 2)

            # bias-corrected 1st & 2nd moment estimate
            m_hat = self.m[key] / (1 - self.beta1 **
self.t)
            v_hat = self.v[key] / (1 - self.beta2 **
self.t)

            # update bobot
            param -= self.lr * m_hat / (np.sqrt(v_hat)
+ self.epsilon)

```

Atribut dan metode kunci:

- **__init__ (Konstruktor)**
 - **self.lr (Learning Rate):** Mengontrol ukuran langkah.
 - **self.beta1 & self.beta2:** Exponential decay rates untuk momen pertama dan kedua dengan nilai default 0.9 dan 0.999
 - **self.m (First Moment/Momentum):** Dictionary untuk menyimpan estimasi rata-rata bergerak (moving average) dari gradien.

- **self.v (Second Moment/RMSprop):** Dictionary untuk menyimpan rata-rata bergerak dari gradien kuadrat. Ini adalah estimasi **varians** gradien yang tidak terpusat.
- **self.t (Timestep):** timestep yang digunakan untuk koreksi bias
- **step(self, params_dict, grads_dict)**

Metode utama ini digunakan untuk pembaruan bobot. Pertama-tama, Gradien saat ini (grad) digunakan untuk memperbarui self.m dan self.v. self.m menggabungkan informasi gradien dari masa lalu (Momentum) dan self.v menggabungkan informasi magnitudo gradien kuadrat (RMSprop) yang berfungsi untuk mengadaptasi step size. Karena self.m dan self.v diinisialisasi sebagai nol, nilainya cenderung bias mendekati nol pada iterasi awal. Koreksi dilakukan dengan membagi estimasi moment dengan faktor $(1 - \beta^t)$ untuk memastikan estimasi lebih akurat. Kemudian, parameter (param) diperbarui dengan menggunakan rasio bias-corrected first moment (m_hat) yang dibagi dengan akar kuadrat dari bias-corrected second moment (v_hat). Bagian $np.sqrt(v_hat)$ berfungsi untuk penskalaan adaptif dimana gradien yang besar dan sering akan menghasilkan pembagi yang besar sehingga langkah pembaruan menjadi lebih kecil.

2.1.2. CNN

Pada bagian ini, akan dijelaskan mengenai implementasi kelas yang digunakan untuk CNN, yang terdiri dari beberapa file yaitu layers.py, cnn.py, train.py, dan [utils.py](#).

layers.py

layers.py berisikan seluruh implementasi layer CNN from scratch yang dapat membaca bobot hasil pelatihan Keras. Setiap layer diimplementasikan sebagai kelas terpisah dengan method forward dan backward tersendiri.

- **ActivationLayer**

Kelas ActivationLayer merupakan layer aktivasi yang dipisah dari layer konvolusi maupun dense agar implementasi lebih modular. Kelas ini mendukung fungsi aktivasi ReLU, Softmax, dan Linear.

```
class ActivationLayer:
    def __init__(self, activation):
```

```
self.activation_name = activation
self.activation      = get_activation(activation)
```

Atribut dan metode utama:

- **__init__(self, activation)**

Konstruktor menerima nama fungsi aktivasi berupa string ('relu', 'softmax', atau 'linear'). Atribut `self.activation_name` menyimpan nama string untuk digunakan pada backward pass, sedangkan `self.activation` menyimpan fungsi aktivasi yang didapat dari `get_activation()`.

- **forward(self, x)**

Metode ini menyimpan input `x` ke `self.x` untuk keperluan backward, lalu mengaplikasikan fungsi aktivasi dan menyimpan hasilnya ke `self.out`. Mendukung input dengan dimensi arbitrer sehingga dapat digunakan setelah layer Conv2D maupun Dense.

- **backward(self, grad_out)**

Metode ini menghitung gradient terhadap input berdasarkan jenis aktivasi:

- ReLU: gradient diteruskan hanya di posisi input yang bernilai positif, yaitu $\text{grad_out} * (\text{self.x} > 0)$
- Softmax: menggunakan rumus Jacobian yang disederhanakan, yaitu $s * (\text{grad_out} - \text{dot})$ dimana $\text{dot} = \text{sum}(\text{grad_out} * s)$
- Linear: gradient diteruskan langsung tanpa modifikasi

- **Conv2DLayer**

Kelas `Conv2DLayer` mengimplementasikan operasi konvolusi 2D dengan shared parameter, artinya kernel yang sama digunakan untuk seluruh posisi spasial pada feature map. Kelas ini mendukung padding 'same' dan 'valid', stride arbitrer, serta batch inference.

```
class Conv2DLayer:
    def __init__(self, keras_layer=None, kernel=None,
                 bias=None, activation='relu', strides=(1,1),
                 padding='valid'):
```

Atribut Utama :

Atribut	Keterangan
self.kernel	Bobot kernel shape (kH, kW, C_in, C_out)
self.bias	Vektor bias shape (C_out,)
self.strides	Tuple stride (sH, sW)
self.padding	String 'valid' atau 'same'
self.activation_name	Nama fungsi aktivasi
self.kH, self.kW, self.C_in, self.C_out	Dimensi kernel
self.dkernel, self.dbias	Gradient bobot dan bias untuk backward
self.x_input	Input yang disimpan saat forward untuk backward
self.pre_act	Output sebelum aktivasi untuk backward ReLU

Metode utama:

- **__init__(self, keras_layer, kernel, bias, activation, strides, padding)**

Konstruktor mendukung dua mode inisialisasi: jika keras_layer tersedia, bobot langsung dimuat dari layer Keras via get_weights() sehingga hasil forward propagation from scratch identik dengan Keras. Jika tidak, bobot dapat diberikan secara langsung melalui parameter kernel dan bias.

- **_pad(self, x)**

Metode privat yang menghitung dan mengaplikasikan zero-padding pada input. Untuk padding 'same', jumlah padding dihitung menggunakan rumus:

```
pH = max((out_H - 1) * sH + self.kH - H, 0)
pW = max((out_W - 1) * sW + self.kW - W, 0)
```

Metode ini mendukung input 3D (H, W, C) maupun 4D (N, H, W, C).

- **_forward_single(self, x)**

Metode privat yang menjalankan operasi konvolusi untuk satu gambar (H, W, C). Untuk setiap posisi (i, j) pada output, dilakukan operasi dot product antara patch input dan kernel menggunakan `np.einsum('hwc,hwck->k', patch, kernel)` lalu ditambahkan bias. Hasilnya adalah feature map sebelum aktivasi dengan shape (out_H, out_W, C_out).

- **forward(self, x)**

Metode ini menjalankan forward propagation dan mendukung dua mode input:

- Single (H, W, C): memanggil `_forward_single` langsung
- Batch (N, H, W, C): memanggil `_forward_single` untuk setiap sampel lalu di-stack

Input disimpan ke `self.x_input` dan output sebelum aktivasi disimpan ke `self.pre_act` untuk keperluan backward. Hasil akhir adalah output setelah diaplikasikan fungsi aktivasi.

- **backward(self, grad_out)**

Metode ini mengimplementasikan backward propagation Conv2D. Alur perhitungannya adalah:

1. Gradient aktivasi ReLU diaplikasikan menggunakan `self.pre_act` sebagai mask
2. Untuk setiap sampel dan setiap posisi output (i, j), gradient kernel dihitung via `np.einsum('hwc,k->hwck', patch, g)` dan diakumulasikan ke `self.dkernel`
3. Gradient bias diakumulasikan ke `self.dbias`
4. Gradient input dihitung via `np.einsum('k,hwck->hwc', g, self.kernel)` dan dikembalikan sebagai dx
5. Jika padding 'same', padding dihapus dari `dx_pad` sebelum dikembalikan

- **LocallyConnected2DLayer**

Kelas `LocallyConnected2DLayer` mengimplementasikan operasi konvolusi 2D dengan non-shared parameter, artinya setiap posisi spasial output memiliki kernel tersendiri yang tidak dibagikan dengan posisi lain. Hal ini menghasilkan jumlah parameter yang jauh lebih besar dibanding `Conv2D` namun berpotensi menangkap pola lokal yang lebih spesifik.

```
class LocallyConnected2DLayer:
```

```
def __init__(self, keras_layer=None, kernel=None,
bias=None, activation='relu', strides=(1,1),
padding='valid', kernel_size=(3,3)):
```

Atribut Utama

Atribut	Keterangan
self.kernel	Bobot kernel shape (out_H*out_W, kH*kW*C_in, C_out)
self.bias	Bias shape (out_H*out_W, C_out)
self.kH, self.kW	Ukuran kernel
self.C_out	Jumlah filter output
self.dkernel, self.dbias	Gradient bobot dan bias untuk backward
self.x_input	Input yang disimpan saat forward untuk backward
self.pre_act	Output sebelum aktivasi untuk backward ReLU

Metode utama:

- **__init__(self, keras_layer, kernel, bias, activation, strides, padding, kernel_size)**

Sama dengan Conv2DLayer, mendukung inisialisasi dari Keras layer maupun bobot eksplisit. Perbedaan utama adalah shape kernel yang memiliki dimensi pertama out_H * out_W karena setiap posisi output memiliki kernel tersendiri.

- **_forward_single(self, x)**

Untuk setiap posisi output (i, j), patch input di-flatten menjadi vektor 1D, kemudian dikalikan dengan kernel pada posisi yang bersesuaian ($pos = i * out_W + j$) menggunakan operasi flat @ self.kernel[pos] + self.bias[pos]. Ini berbeda dari Conv2D yang menggunakan kernel yang sama untuk semua posisi.

- **forward(self, x)**

Mendukung input single (H, W, C) dan batch (N, H, W, C). Menyimpan self.x_input dan self.pre_act untuk backward.

- **backward(self, grad_out)**

Gradient kernel dan bias dihitung per posisi output menggunakan `np.outer(flat, g)` untuk `self.dkernel[pos]` dan langsung diakumulasikan ke `self.dbias[pos]`. Gradient input dihitung via `self.kernel[pos] @ g` lalu di-reshape ke `(kH, kW, C_in)` dan diakumulasikan ke `dx_pad`.

- **MaxPooling2DLayer**

Kelas `MaxPooling2DLayer` mengimplementasikan operasi max pooling 2D dengan sliding window, mengambil nilai maksimum dari setiap window pada setiap channel.

```
class MaxPooling2DLayer:
    def __init__(self, keras_layer=None, pool_size=(2,2),
strides=None):
```

Atribut Utama

Atribut	Keterangan
<code>self.pool_size</code>	Tuple ukuran window (pH, pW)
<code>self.strides</code>	Tuple stride, default sama dengan <code>pool_size</code>
<code>self.x_input</code>	Input yang disimpan saat forward untuk backward

Metode utama:

- **__init__(self, keras_layer, pool_size, strides)**

Jika `keras_layer` tersedia, konfigurasi `pool_size` dan `strides` diambil dari `get_config()`. Jika tidak, digunakan nilai default atau yang diberikan secara eksplisit. Jika `strides` tidak diberikan, nilainya sama dengan `pool_size` (non-overlapping pooling).

- **_forward_single(self, x)**

Untuk setiap posisi window `(i, j)`, diambil nilai maksimum dari tiap channel menggunakan `np.max(window, axis=(0,1))`.

- **forward(self, x)**

Mendukung input single `(H, W, C)` dan batch `(N, H, W, C)`. Input disimpan ke `self.x_input` untuk backward.

- **backward(self, grad_out)**

Gradient hanya diteruskan ke posisi yang merupakan nilai maksimum dalam window. Mask dibuat dengan `(window == max_val)` dan dibagi rata jika ada nilai maksimum yang sama (tie-breaking). Gradient kemudian dikalikan dengan `grad_out` dan diakumulasikan ke `dx`.

- **AveragePooling2DLayer**

Kelas `AveragePooling2DLayer` mengimplementasikan operasi average pooling 2D, mengambil nilai rata-rata dari setiap window pada setiap channel.

```
class AveragePooling2DLayer:
    def __init__(self, keras_layer=None, pool_size=(2,2),
strides=None):
```

Atribut dan cara inisialisasi sama dengan `MaxPooling2DLayer`. Perbedaan terletak pada:

- **_forward_single(self, x)**
Menggunakan `np.mean(window, axis=(0,1))` alih-alih `np.max`.
- **backward(self, grad_out)**
Gradient dibagi rata ke seluruh posisi dalam window dengan membagi `grad_out[n, i, j, :]` dengan `pH * pW`, karena setiap posisi berkontribusi sama terhadap output rata-rata.

- **GlobalMaxPooling2DLayer**

Kelas `GlobalMaxPooling2DLayer` mereduksi seluruh dimensi spasial (H, W) menjadi satu nilai per channel dengan mengambil nilai maksimum global.

Metode utama:

- **forward(self, x)**
Untuk input single (H, W, C) menggunakan `np.max(x, axis=(0,1))` menghasilkan vektor (C,). Untuk batch (N, H, W, C) menggunakan `np.max(x, axis=(1,2))` menghasilkan (N, C).
- **backward(self, grad_out)**
Gradient hanya diteruskan ke posisi maksimum per channel. Untuk setiap sampel dan channel, dibuat mask posisi maksimum lalu gradient dibagi rata jika ada tie.

- **GlobalAveragePooling2DLayer**

Kelas GlobalAveragePooling2DLayer mereduksi seluruh dimensi spasial menjadi satu nilai per channel dengan mengambil rata-rata global. Layer ini digunakan pada arsitektur Conv2D terbaik sebagai pengganti Flatten untuk mengurangi jumlah parameter.

Metode utama:

- **forward(self, x)**

Untuk input single (H, W, C) menggunakan `np.mean(x, axis=(0,1))`. Untuk batch (N, H, W, C) menggunakan `np.mean(x, axis=(1,2))`.

- **backward(self, grad_out)**

Gradient dibagi rata ke seluruh posisi spasial (H, W) per channel dengan `grad_out[n] / (H * W)`, karena setiap posisi berkontribusi sama terhadap rata-rata global.

- **FlattenLayer**

Kelas FlattenLayer mereshape tensor multidimensi menjadi vektor 1D (single) atau matriks 2D (batch) dengan urutan row-major (C order), konsisten dengan perilaku Keras.

Metode utama:

- **forward(self, x)**

Menyimpan `self.x_shape` untuk keperluan backward. Untuk input single (H, W, C) menggunakan `x.flatten(order='C')`. Untuk batch (N, H, W, C) menggunakan `x.reshape(N, -1)`.

- **backward(self, grad_out)**

Mereshape gradient kembali ke shape input asli menggunakan `grad_out.reshape(self.x_shape)`.

cnn.py

`cnn.py` berisikan kelas `CNNScratch` yang merupakan wrapper untuk menggabungkan seluruh layer CNN menjadi satu model utuh yang dapat menjalankan forward propagation, inferensi, dan backward propagation from scratch.

```
class CNNScratch:
    def __init__(self, layers):
        self.layers = layers
```

Atribut Utama

Atribut	Keterangan
self.layers	List layer CNN secara berurutan

Metode utama:

- **__init__(self, layers)**
Konstruktor menerima list layer yang sudah diinisialisasi dengan bobot dari Keras. Urutan layer dalam list menentukan urutan eksekusi forward propagation.
- **forward(self, x)**
Menjalankan forward propagation secara sekuensial melewati seluruh layer dalam self.layers. Mendukung input single (H, W, C) dan batch (N, H, W, C) karena setiap layer sudah mendukung kedua mode tersebut.
- **predict(self, x)**
Menjalankan forward propagation untuk satu gambar dan mengembalikan indeks kelas dengan probabilitas tertinggi menggunakan np.argmax.
- **predict_batch(self, images, batch_size=32)**
Menjalankan batch inference dengan membagi dataset menjadi mini-batch sesuai batch_size. Setiap mini-batch diproses sekaligus melalui forward() tanpa loop per gambar. Hasilnya dikumpulkan dan di-concatenate menjadi array prediksi akhir. Pendekatan ini jauh lebih efisien dibanding loop per gambar karena memanfaatkan operasi matriks NumPy.
- **predict_proba(self, images, batch_size=32)**
Sama dengan predict_batch namun mengembalikan probabilitas seluruh kelas (N, num_classes) alih-alih hanya indeks kelas.
- **backward(self, grad_out)**
Menjalankan backward propagation secara terbalik melewati seluruh layer. Hanya layer yang memiliki method backward yang dieksekusi, sehingga layer tanpa parameter (seperti Dropout) dapat diabaikan secara otomatis.

train.py

train.py berisikan fungsi-fungsi untuk membangun dan melatih model CNN menggunakan Keras, termasuk definisi arsitektur Conv2D dan LocallyConnected2D serta pipeline eksperimen 16 variasi arsitektur.

Fungsi utama:

- **load_dataset(data_dir, img_size, batch_size)**
Fungsi ini memuat dataset Intel Image Classification dari direktori menggunakan `keras.utils.image_dataset_from_directory`. Dataset training di-shuffle dengan seed tetap untuk reproducibility, sedangkan dataset validasi tidak di-shuffle. Normalisasi pixel ke rentang $[0, 1]$ dilakukan menggunakan `layers.Rescaling(1./255)`. Dataset kemudian di-cache dan di-prefetch untuk mengoptimalkan throughput training.
- **build_cnn_model(num_conv_layers, filters, kernel_sizes, pooling_type, img_size, num_classes)**
Fungsi ini membangun arsitektur CNN dengan Conv2D (shared parameter) menggunakan Keras Functional API. Arsitektur terdiri dari blok Conv2D + Pooling yang diulang sebanyak `num_conv_layers`, diikuti GlobalAveragePooling2D, Dense(128, ReLU), Dropout(0.3), dan Dense output dengan Softmax. Model dikompilasi dengan optimizer Adam dan loss Sparse Categorical Crossentropy.
- **build_locally_connected_model(filters, kernel_sizes, pooling_type, img_size, num_classes)**
Fungsi ini membangun arsitektur CNN dengan LocallyConnected2D (non-shared parameter). Karena parameter LocallyConnected2D sangat besar untuk input 150×150 , ukuran input diturunkan menjadi 32×32 . Perbedaan utama dari Conv2D adalah penggunaan `layers.LocallyConnected2D` dan `layers.Flatten()` alih-alih GlobalAveragePooling2D karena output spasial dari LocallyConnected2D perlu di-flatten sepenuhnya.
- **run_experiments(data_dir)**
Fungsi ini menjalankan eksperimen 16 variasi arsitektur Conv2D secara otomatis. Variasi dihasilkan dari kombinasi 2 jumlah layer $\times$ 2 ukuran filter $\times$ 2 ukuran kernel $\times$ 2 jenis pooling. Untuk setiap konfigurasi, model dibangun, dilatih selama 20 epoch, dan bobotnya disimpan ke WEIGHTS_DIR. Hasil training history dari setiap eksperimen dikembalikan untuk analisis lebih lanjut.

utils.py

utils.py berisikan fungsi-fungsi utilitas untuk evaluasi model dan visualisasi hasil training.

Fungsi utama:

- **compute_macro_f1(y_true, y_pred)**

Menghitung macro F1-score menggunakan `sklearn.metrics.f1_score` dengan `average='macro'`, yaitu rata-rata F1-score dari seluruh kelas tanpa mempertimbangkan jumlah sampel per kelas.

- **`evaluate_keras_model(model, X, y_true, class_names)`**
Mengevaluasi model Keras pada data test. Prediksi dilakukan via `model.predict()` dan indeks kelas diambil dengan `np.argmax`. Mengembalikan dictionary berisi `macro_f1`, `report` (classification report per kelas), dan `y_pred`.
- **`evaluate_scratch_model(scratch_model, X, y_true, class_names)`**
Mengevaluasi model from scratch menggunakan `predict_batch()`. Interface-nya identik dengan `evaluate_keras_model` sehingga perbandingan hasil keduanya dapat dilakukan secara langsung.
- **`plot_history(history, title, save_path)`**
Memvisualisasikan grafik training dan validation loss serta accuracy per epoch dalam dua subplot berdampingan. Grafik dapat disimpan ke file jika `save_path` diberikan.
- **`plot_f1_comparison(labels, f1_scores, title, save_path)`**
Memvisualisasikan perbandingan macro F1-score antar model atau konfigurasi dalam bentuk bar chart dengan label nilai di atas setiap bar.

2.1.3. RNN

Pada bagian ini, akan dijelaskan mengenai implementasi kelas dari RNN.

- `rnn.py`

`rnn.py` berisikan kelas RNN yang mengimplementasikan 1 sel **SimpleRNN** yang menjadi unit dasar arsitektur decoder berbasis RNN pada persoalan image captioning ini. SimpleRNN ini dibangun untuk memproses token kata sekuensial pada satu time step sambil mempertahankan hidden state untuk menangkap konteks temporal. RNN menggunakan **Parameter Sharing** dimana bobot `w_xh`, `w_hh`, dan `b_h` digunakan berulang kali pada setiap timestep. Berikut adalah implementasi dari kelas RNN:

```
class RNN:
    def __init__(self, keras_layer=None,
                 input_size=None, hidden_size=None):
        if keras_layer is not None:
            weights = keras_layer.get_weights()
```



```

        self.w_xh = weights[0] # shape
(input_size, hidden_size)
        self.w_hh = weights[1] # shape
(hidden_size, hidden_size)
        self.b_h = weights[2] # shape
(hidden_size,)
    else:
        if input_size is not None and hidden_size
is not None:
            # Inisialisasi bobot awal dengan nilai
kecil
            self.w_xh = np.random.randn(input_size,
hidden_size) * 0.01
            self.w_hh =
np.random.randn(hidden_size, hidden_size) * 0.01
            self.b_h = np.random.randn(hidden_size)
        else:
            raise ValueError("Incorrect
Parameters")

```

Konstruktor (`__init__`) menginisialisasi atau memuat tiga matriks bobot sel RNN yaitu `w_xh` (bobot input ke hidden layer), `w_hh` (bobot hidden ke hidden layer), dan `b_h` (bobot bias). Jika parameter `keras_layer` tersedia, bobot hasil pelatihan Keras akan dimuat dan jika tidak, bobot diinisialisasi secara acak dengan nilai sangat kecil (dikalikan 0.01) untuk membantu stabilitas training awal dan mencegah masalah exploding gradient.

```

def forward(self, x, h_prev):
    # x shape: (1, input_size)
    # h_prev shape: (1, hidden_size)
    self.x = x
    self.h_prev = h_prev

    net_h = np.dot(x, self.w_xh) + np.dot(h_prev,
self.w_hh) + self.b_h
    h = tanh(net_h)

```

```
# cache untuk backpropagation
cache = (x, h_prev, h)

return h, cache
```

Metode forward mengimplementasikan forward propagation dari sel RNN untuk satu time step. Fungsi menggabungkan input saat ini (x) dengan **hidden state** sebelumnya (h_{prev}). Hidden state berperan sebagai variabel memori yang menangkap konteks temporal dari urutan data yang telah diproses. Secara matematis, forward propagation diformulasikan sebagai berikut: $h_t = \tanh(x_t \cdot W_{xh} + h_{t-1} \cdot W_{hh} + b_h)$.

Penggunaan fungsi aktivasi tanh memastikan nilai state tetap berada dalam rentang -1 hingga 1 sehingga membantu menstabilkan aliran gradien dan mencegah nilai numerik meledak selama iterasi berulang. Hasil dari fungsi ini yaitu h akan diteruskan ke timestep berikutnya sebagai input memori baru.

```
def backward(self, dh, cache):
    x, h_prev, h = cache

    # backprop aktivasi tanh
    dtanh = dh * (1 - h**2) # shape (1,
hidden_size)

    # gradien untuk bobot dan bias
    dw_xh = np.dot(x.T, dtanh) # shape
(input_size, hidden_size)
    dw_hh = np.dot(h_prev.T, dtanh) # shape
(hidden_size, hidden_size)
    db_h = np.sum(dtanh, axis=0, keepdims=True) #
shape (1, hidden_size)

    # gradien untuk input dan hidden state
    sebelumnya
    dh_prev = np.dot(dtanh, self.w_hh.T) # shape
(1, hidden_size)
    dx = np.dot(dtanh, self.w_xh.T) # shape (1,
input_size)
```

```
return dx, dh_prev, dw_xh, dw_hh, db_h
```

Metode backward mengimplementasikan algoritma **Backpropagation Through Time (BPTT)** untuk menghitung gradien dari parameter sel RNN. BPTT bekerja dengan membuka lipatan RNN (unrolling) sepanjang sekuens waktu dan mengalirkan gradien mundur dari timestep terakhir menuju awal. Gradien total untuk setiap bobot yaitu w_{xh} , w_{hh} , dan b_h adalah hasil akumulasi gradien dari setiap timestep karena adanya mekanisme parameter sharing. Proses dimulai dengan menghitung turunan fungsi tanh yaitu $dtanh = dh * (1 - h^2)$. Gradien ini kemudian digunakan untuk menghitung dh_{prev} yang merupakan gradien yang harus diteruskan ke memori timestep sebelumnya.

- layers.py

layers.py berisikan kelas RNNDecoder yang merupakan implementasi dari arsitektur decoder berbasis RNN pada persoalan *image captioning* ini dengan metode Pre-Inject feature CNN. Kelas ini digunakan untuk menghasilkan sekuens kata (caption) berdasarkan fitur visual gambar yang di-inject di awal sekuens. Kelas ini terdiri dari beberapa metode yaitu sebagai berikut:

```
class RNNDecoder:
    def __init__(self, cnn_features_dim, embed_dim,
hidden_dim, vocab_size):
        # Proyeksi dimensi fitur CNN ke dimensi
embedding
        self.projection_layer =
DenseLayer(input_dim=cnn_features_dim,
output_size=embed_dim)

        # Embedding layer untuk token input
        self.embedding_layer =
EmbeddingLayer(vocab_size=vocab_size,
embed_dims=embed_dim)

        # RNN layer
        self.rnn_cell = RNN(input_size=embed_dim,
hidden_size=hidden_dim)
```

```

        # Output layer
        self.output_layer =
DenseLayer(input_size=hidden_dim,
output_size=vocab_size)

        # dimensi tersembunyi untuk forward pass
        self.hidden_dim = hidden_dim

```

Konstruktor (__init__) menginisialisasi empat komponen utama:

- **self.projection_layer:** Layer Dense yang mereduksi dimensi vektor fitur visual dari CNN menjadi dimensi yang sama dengan embedding kata dan menjadikannya input pertama
- **self.embedding_layer:** Layer embedding yang mengubah token integer kata menjadi dense vektor padat
- **self.rnn_cell:** Unit SimpleRNN yang memproses input sekuensial
- **self.output_layer:** Layer Dense terakhir yang memproyeksikan hidden state RNN (h_t) ke dimensi vocab_size untuk memprediksi probabilitas token kata berikutnya dengan fungsi aktivasi Softmax

```

def forward(self, image_features, caption_tokens):
    # inisialisasi hidden state awal (h_0)
    h_t = np.zeros((1, self.hidden_dim))

    # list probabilitas prediksi di setiap timestep
    predicted_probabilities = []

    # === fase 1: Pre-Inject gambar ===
    # ubah fitur gambar (2048) ke dimensi embedding
    x_image =
self.projection_layer.forward(image_features)

    # input RNN untuk timestep pertama adalah
embedding dari fitur gambar
    h_t = self.rnn_cell.forward(x_image, h_t)

    # === fase 2: pemrosesan sekuens teks ===

```

```

        for token in caption_tokens:
            # vektor embedding dari token input
            x_word =
self.embedding_layer.forward(token)

            # Embedding layer mengembalikan vektor 1D
            # ubah menjadi 2D untuk perhitungan dot
product dengan bobot RNN
            if x_word.ndim == 1:
                x_word = np.expand_dims(x_word, axis=0)
# ubah ke shape (1, embed_dim)

            # forward pass melalui RNN cell
            h_t = self.rnn_cell.forward(x_word, h_t)

            # prediksi distribusi probabilitas untuk
token berikutnya
            logits = self.output_layer.forward(h_t)
            probs = softmax(logits)

            # simpan probabilitas prediksi untuk
timestep ini
            predicted_probabilities.append(probs)

        return predicted_probabilities

```

Metode forward adalah implementasi dari forward pass dari decoder RNN untuk memproses seluruh sekuens input yaitu fitur gambar dan token kata menggunakan mekanisme **Teacher Forcing** untuk pelatihan/training. Proses ini dibagi menjadi dua fase utama:

- **Pre-Inject Gambar**
Vektor fitur visual (image_features) pertama-tama diubah dimensinya melalui projection_layer dan hasilnya kemudian dimasukkan ke rnn_cell bersama dengan hidden state awal h_0. Hasil hidden state (h_t) dari fase ini akan menjadi h_prev untuk kata pertama.
- **Pemrosesan Sekuens Teks**
Kemudian, akan dilakukan iterasi pada setiap token kata dalam caption_tokens dan untuk setiap token, Embedding Layer

mengubahnya menjadi vektor kata `x_word`. Vektor ini bersama dengan `h_prev` dimasukkan ke `rnn_cell` dan hidden state (`h_t`) yang baru kemudian dimasukkan ke `output_layer` yang menghasilkan distribusi probabilitas untuk memprediksi kata berikutnya dalam vocabulary. Seluruh hasil probabilitas ini dikumpulkan untuk menghitung loss di tahap pelatihan.

```
def backward(self, d_logits_list, caches_list,
caption_tokens):
    # d_logits_list: list gradien dari output layer
    # di setiap timestep
    # caches_list: list cache yang disimpan dari
    # forward pass di setiap timestep
    # caption_tokens: list token input untuk setiap
    # timestep

    # inisialisasi gradien akumulatif awal
    dw_xh_total = np.zeros_like(self.rnn_cell.w_xh)
    dw_hh_total = np.zeros_like(self.rnn_cell.w_hh)
    db_h_total = np.zeros_like(self.rnn_cell.b_h)

    # inisialisasi gradien untuk hidden state
    # berikutnya
    dh_next = np.zeros((1, self.hidden_dim))

    # panjang sekuens
    sequence_length = len(caches_list)

    # iterasi mundur dari timestep T ke 0
    # (Backpropagation Through Time)
    for t in reversed(range(sequence_length)):
        # backprop dense output layer
        d_out = d_logits_list[t]
        dh_output =
self.output_layer.backward(d_out)

        # total dh di timestep ini: gabungan
        # gradien dari output layer dan timestep berikutnya
        dh = dh_output + dh_next
```

```

        # backward pass RNN
        cache_t = caches_list[t]
        dx, dh_prev, dw_xh, dw_hh, db_h =
self.rnn_cell.backward(dh, cache_t)

        # akumulasi gradien untuk bobot RNN
        dw_xh_total += dw_xh
        dw_hh_total += dw_hh
        db_h_total += db_h

        # update dh_next untuk timestep berikutnya
        dh_next = dh_prev

        # backward pass embedding layer
        token_t = caption_tokens[t] # token input
di timestep t
        self.embedding_layer.token_id = token_t #
set token_id untuk update gradien
        self.embedding_layer.backward(dx)

    return dw_xh_total, dw_hh_total, db_h_total

```

Metode backward adalah implementasi algoritma Backpropagation Through Time (BPTT) pada decoder RNN di seluruh sekuens. Karena RNN menggunakan parameter sharing, gradien yang dihitung di setiap timestep akan diakumulasi. Proses ini beroperasi secara terbalik dari timestep terakhir ke awal dan terdiri dari beberapa tahapan:

- Gradien loss dari output layer (`d_logits_list`) di setiap timestep pertama kali akan dilakukan backpropagate melalui output_layer (Dense) untuk mendapatkan `dh_output`
- Gradien total `dh` untuk hidden state (`h_t`) saat ini adalah gabungan dari `dh_output` dan gradien yang diteruskan dari timestep berikutnya (`dh_next`)
- `dh` kemudian dimasukkan ke `rnn_cell.backward()` untuk menghitung gradien bobot (`dw_xh`, `dw_hh`, `db_h`) pada tiap timestep dan gradien `dh_prev` yang diteruskan ke timestep sebelumnya

- Gradien per-timestep ini dijumlahkan ke `dw_xh_total`, `dw_hh_total`, dan `db_h_total`
- Gradien input (`dx`) akan diteruskan ke `embedding_layer.backward()` untuk memperbarui gradien matriks `embedding`
- `train.py`
`train.py` berisikan metode-metode untuk membangun pipeline training dari model Keras RNN. Berikut adalah beberapa metode yang diimplementasikan:

```
# fungsi utk load CNN features dan membuat tokenizer
dari caption
def load_data():
    # load CNN features
    features = np.load(FEATURES_PATH,
allow_pickle=True).item()

    # load captions
    caption_mapping = load_captions(CAPTIONS_PATH)
    cleaned_mapping =
clean_and_wrap_captions(caption_mapping)

    # build tokenizer
    tokenizer, all_captions =
build_tokenizer(cleaned_mapping, TOKENIZER_PATH)
    max_length = calculate_max_length(all_captions)

    return features, tokenizer, cleaned_mapping,
max_length
```

Fungsi `load_data` berfungsi untuk memuat, memproses, dan memfinalisasi fitur visual CNN dan data teks yang sudah di-tokenisasi sebelum memulai pelatihan decoder RNN. Fungsi ini memanggil beberapa fungsi utilitas dari `image_utils.py` dan `preprocessing.py` untuk menyiapkan data. Langkah-langkah yang dilakukan:

- Memuat vektor fitur visual gambar berdimensi 2048 yang telah diekstraksi dan disimpan dalam format file `.npy`

- Memanggil `load_captions` untuk memuat teks mentah dan `clean_and_wrap_captions` untuk membersihkan, lowercase, serta menambahkan token `<start>` dan `<end>`
- Memanggil `build_tokenizer` untuk membangun vocabulary pemetaan kata ke indeks integer lalu menyimpannya dan memanggil `calculate_max_length` untuk menentukan panjang maksimum sekuens yang akan digunakan untuk *padding*.

Fungsi akan mengembalikan `features`, `tokenizer`, `cleaned_mapping`, dan `max_length` yang menjadi input untuk fungsi `prepare_sequences` selanjutnya.

```
# fungsi utk membuat pasangan input-output utk teacher
forcing
def prepare_sequences(features, tokenizer,
cleaned_mapping, max_length, embed_dim):
    vocab_size = len(tokenizer.word_index) + 1

    X, y = [], []

    for image_id, captions in cleaned_mapping.items():
        # ambil feature vector untuk gambar ini
        if image_id not in features:
            continue
        feature = features[image_id] # shape: (2048,)

        for caption in captions:
            # tokenize caption
            seq =
tokenizer.texts_to_sequences([caption])[0]

            # buat pasangan input-output dengan teacher
forcing
            for i in range(1, len(seq)):
                in_seq = seq[:i] # input:
semua token sebelum posisi i
                out_seq = seq[i] # target:
token di posisi i

            # padding input sequence
```

```

        in_seq =
keras.preprocessing.sequence.pad_sequences(
            [in_seq], maxlen=max_length,
padding='post'
        ) [0]

        X.append((feature, in_seq))
        y.append(out_seq)

return X, np.array(y)

```

Fungsi `prepare_sequences` digunakan untuk menyiapkan data pelatihan decoder RNN menggunakan metode **Teacher Forcing**. Dalam metode ini, model dilatih untuk memprediksi token berikutnya berdasarkan urutan token yang benar dari data ground truth sebelumnya bukan berdasarkan prediksinya sendiri. Fungsi ini bekerja dengan mengiterasi setiap `image_id` dalam `cleaned_mapping` untuk mengambil fitur CNN yang sesuai. Untuk setiap caption yang terkait dengan gambar tersebut, fungsi melakukan tokenisasi menjadi sekuens integer menggunakan `tokenizer.texts_to_sequences`. Selanjutnya, fungsi memotong sekuens tersebut menjadi pasangan input-output pada setiap posisi token melalui loop internal. Untuk setiap posisi, bagian sekuens dari awal hingga sebelum posisi tersebut diambil sebagai `in_seq` (input prefix) sementara token tepat pada posisi `i` ditetapkan sebagai `out_seq` (target). Setiap `in_seq` kemudian diberikan post-padding menggunakan `pad_sequences` agar memiliki panjang seragam `max_length` dengan menambahkan nol di bagian akhir sekuens. Akhirnya, fungsi menyusun output `X` sebagai kumpulan pasangan berupa tuple yang berisi fitur CNN gambar dan sekuens input yang telah di-pad serta `y` sebagai array NumPy yang berisi semua token target.

```

# fungsi untuk membangun model Keras SimpleRNN
def build_rnn_keras_model(vocab_size, embed_dim,
hidden_dim, max_length, cnn_feature_dim=2048):
    # CNN feature input
    cnn_input = Input(shape=(cnn_feature_dim,),
name='cnn_input')
    cnn_proj = Dense(embed_dim, activation='relu',
name='cnn_proj')(cnn_input)

```

```

cnn_proj      = keras.layers.Reshape((1,
embed_dim))(cnn_proj) # (1, embed_dim)

# caption sequence input
caption_input  = Input(shape=(max_length,),
name='caption_input')
caption_embed  = Embedding(vocab_size, embed_dim,
mask_zero=True)(caption_input)

# pre-inject: concat CNN feature di depan sequence
merged        = Concatenate(axis=1)([cnn_proj,
caption_embed]) # (max_length+1, embed_dim)

# RNN decoder
rnn_out       = SimpleRNN(hidden_dim,
return_sequences=False)(merged)

# output
outputs       = Dense(vocab_size,
activation='softmax')(rnn_out)

model = Model(inputs=[cnn_input, caption_input],
outputs=outputs)

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
return model

```

Fungsi `build_rnn_keras_model` merupakan implementasi arsitektur decoder image captioning RNN menggunakan metode pre-inject. Dalam mekanisme ini, fitur visual dari gambar tidak dimasukkan pada setiap timestep melainkan hanya di awal sebagai instruksi konteks bagi decoder RNN. Alur kerja fungsi ini dimulai dengan menerima dua input utama yaitu `cnn_input` (vektor fitur visual berdimensi 2048) dan `caption_input` (sekuens token kata). Fitur gambar pertama-tama diproyeksikan melalui lapisan Dense dengan aktivasi ReLU agar dimensinya sama dengan

dimensi embedding kata. Sementara itu, token kata diproses melalui lapisan Embedding untuk mendapatkan representasi vektor kontinu agar memiliki makna semantik.

Inti dari mekanisme pre-inject terjadi pada lapisan Concatenate dimana vektor gambar yang telah diproyeksikan diletakkan tepat di depan (prepend) sekuens embedding kata sebagai token awal pada timestep $t = -1$. Gabungan data ini kemudian dimasukkan ke dalam sel SimpleRNN untuk mempelajari pola temporal dan hubungan antara visual gambar dengan urutan kata. Terakhir, hidden state dari RNN di setiap langkah dipetakan kembali ke seluruh kosakata melalui lapisan Dense dengan aktivasi Softmax untuk menghasilkan distribusi probabilitas kata berikutnya. Model menggunakan optimizer Adam untuk update bobot dan Sparse Categorical Cross Entropy sebagai fungsi loss agar pelatihan efisien untuk memprediksi indeks kata secara langsung dari kosakata yang tersedia.

```
# fungsi training pipeline utama
def train():
    print("Loading data...")
    features, tokenizer, cleaned_mapping, max_length =
load_data()

    vocab_size = len(tokenizer.word_index) + 1
    print(f"vocab_size={vocab_size},
max_length={max_length}")

    print("Preparing sequences...")
    X, y = prepare_sequences(features, tokenizer,
cleaned_mapping, max_length, EMBED_DIM)

    # split X jadi cnn_features dan cap_sequences
    cnn_features = np.array([x[0] for x in X]) # (N,
2048)
    cap_sequences = np.array([x[1] for x in X]) # (N,
max_length)

    print(f"Training samples: {len(y)}")

    print("Building model...")
```

```

model = build_rnn_keras_model(vocab_size,
EMBED_DIM, HIDDEN_DIM, max_length)
model.summary()

print("Training...")
model.fit(
    [cnn_features, cap_sequences],
    y,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    validation_split=0.1,
    verbose=1
)

Path('weights').mkdir(exist_ok=True)
model.save_weights(WEIGHTS_PATH)
print(f"Weights saved to {WEIGHTS_PATH}")

```

Fungsi train adalah fungsi utama yang menggabungkan seluruh pipeline pelatihan model *Image Captioning* berbasis RNN menggunakan Keras. Fungsi ini digunakan mulai dari mempersiapkan data mentah hingga menyimpan bobot model yang telah dilatih. Berikut adalah beberapa tahapannya:

1. Fungsi memanggil `load_data()` untuk memuat feature vector CNN yang sudah diekstrak dan caption teks yang sudah di-tokenisasi serta mendapatkan ukuran vocabulary (`vocab_size`) dan panjang sekuens maksimum (`max_length`)
2. Kemudian, fungsi memanggil `prepare_sequences()` untuk menyiapkan pasangan input-output data pelatihan (`X`, `y`) menggunakan metode Teacher Forcing
3. Data `X` dipecah menjadi dua array NumPy terpisah yaitu `cnn_features` (vektor fitur visual dengan ukuran (N, 2048)) dan `cap_sequences` (sekuens caption yang sudah dipadding dengan ukuran (N, `max_length`)). Array `y` berisi token target yang akan diprediksi.
4. Model Keras RNN decoder dibangun dan dikompilasi menggunakan `build_rnn_keras_model` dengan semua parameter yang telah disiapkan. Ringkasan model ditampilkan melalui `model.summary()` untuk memverifikasi arsitektur.

5. Proses pelatihan dimulai dengan memanggil `model.fit()`. Model dilatih selama sejumlah epoch tertentu, menggunakan ukuran batch, dan mengalokasikan `validation_split=0.1` dari data pelatihan untuk set validasi untuk mencegah *overfitting*
6. Setelah pelatihan selesai, bobot akhir model disimpan dan dapat dimuat kembali pada implementasi from scratch untuk evaluasi sesuai spesifikasi tugas yaitu **Bagian 4: Implementasi Arsitektur**

2.1.4. LSTM

- [lstm.py](#)

`lstm.py` berisikan kelas LSTM yang mengimplementasikan satu sel **Long Short-Term Memory lengkap dengan forward pass, backward pass (BPTT)**, dan gate-gate individual. Berbeda dengan SimpleRNN, LSTM memiliki dua state yang mengalir antar timestep yaitu hidden state (h) dan cell state (c), serta empat gate yang mengontrol aliran informasi.

Layout bobot, semua gate digabung dalam satu matriks untuk efisiensi:

Matriks	Shape	Keterangan
W	(embed_dim, 4 x hidden_dim)	Bobot input untuk seluruh <i>gate</i>
U	(hidden_dim, 4 x hidden_dim)	Bobot <i>hidden state</i> untuk seluruh <i>gate</i>
b	(4 x hidden_dim)	Bias untuk setiap <i>gate</i>

Urutan gate dalam matriks: [i | f | c | o] (input, forget, candidate, output).

Berikut adalah implementasi [lstm.py](#):

```
import numpy as np
from shared.activation_functions import tanh, sigmoid

class LSTM:
    """
    Single LSTM cell with forward and backward (BPTT)
    support.

    Weight layout (all gates concatenated along axis
    1):

        W: (embed_dim, 4 * hidden_dim)  -- input
```

```

weights
    U: (hidden_dim, 4 * hidden_dim) -- recurrent
weights
    b: (4 * hidden_dim,) -- biases
    Gate order: [i | f | c | o]
    """
    def __init__(self, keras_layer=None, W=None,
U=None, b=None, embed_dim=None, hidden_dim=None):
        """
        Initialize weights from a Keras layer, explicit
arrays, or random values.

        Args:
            keras_layer: Keras LSTM layer to copy
weights from.
            W, U, b: explicit weight arrays (all three
required together).
            embed_dim: input embedding dimension (used
for random init).
            hidden_dim: number of hidden units (used
for random init).
        """
        if keras_layer is not None:
            self.W, self.U, self.b =
keras_layer.get_weights()
            self.hidden_dim = self.W.shape[1] // 4
        else:
            if W is not None and U is not None and b is
not None:
                self.W, self.U, self.b = W, U, b
                self.hidden_dim = hidden_dim or
self.W.shape[1] // 4
            elif embed_dim is not None and hidden_dim
is not None:
                self.hidden_dim = hidden_dim
                self.W = np.random.randn(embed_dim, 4 *
hidden_dim)
                self.U = np.random.randn(hidden_dim, 4
* hidden_dim)

```

```

        self.b = np.random.randn(4 *
hidden_dim)
    else:
        raise ValueError("Incorrect
Parameters")

    h = self.hidden_dim
    self.W_i, self.W_f, self.W_c, self.W_o =
self.W[:, 0*h:1*h], self.W[:, 1*h:2*h], self.W[:,
2*h:3*h], self.W[:, 3*h:4*h]
    self.U_i, self.U_f, self.U_c, self.U_o =
self.U[:, 0*h:1*h], self.U[:, 1*h:2*h], self.U[:,
2*h:3*h], self.U[:, 3*h:4*h]
    self.b_i, self.b_f, self.b_c, self.b_o =
self.b[0*h:1*h], self.b[1*h:2*h], self.b[2*h:3*h],
self.b[3*h:4*h]

    self.embed_dim = self.W.shape[0]
    self.zero_grad()

def forward(self, x, h_prev, c_prev):
    """
    Run one LSTM timestep.

    Args:
        x: input vector, shape (embed_dim,)
        h_prev: previous hidden state, shape
(hidden_dim,)
        c_prev: previous cell state, shape
(hidden_dim,)

    Returns:
        h: new hidden state, shape (hidden_dim,)
        c: new cell state, shape (hidden_dim,)
        cache: dict of intermediate values needed
for backward.
    """
    f = self.forget_gate(x, h_prev)
    i = self.input_gate(x, h_prev)
    candidate = self.candidate(x, h_prev)
    o = self.output_gate(x, h_prev)

```



```

        c = c_prev * f + (candidate * i)
        tanh_c = tanh(c)
        h = tanh_c * o

        # cache the current timestep for later use in
        backprop
        cache = {
            'x':          x,                # for dW_f, dW_i,
            dW_c, dW_o
            'h_prev':     h_prev,           # for dU_f,
            dU_i, dU_c, dU_o
            'c_prev':     c_prev,           # for dc/df
            'f':          f,                # for sigmoid
            derivative
            'i':          i,
            'candidate':  candidate,
            'o':          o,
            'c':          c,
            'tanh_c':     tanh_c,           # for tanh
            derivative
        }
        return h, c, cache

    def backward(self, dh, dc, cache):
        """
        Backpropagate through one LSTM timestep and
        accumulate weight gradients.

        Gradients are accumulated into dW_*, dU_*, db_*
        (reset via zero_grad).

        Args:
            dh: gradient of loss w.r.t. hidden state h,
            shape (hidden_dim,)
            dc: gradient of loss w.r.t. cell state c
            (from next timestep), shape (hidden_dim,)
            cache: dict returned by the corresponding
            forward call.

```

```

Returns:
    dx: gradient w.r.t. input x, shape
(embed_dim,)
    dh_prev: gradient w.r.t. h_prev, shape
(hidden_dim,)
    dc_prev: gradient w.r.t. c_prev, shape
(hidden_dim,)
    """
    x          = cache['x']
    h_prev     = cache['h_prev']
    c_prev     = cache['c_prev']
    f          = cache['f']
    i          = cache['i']
    candidate  = cache['candidate']
    o          = cache['o']
    c          = cache['c']
    tanh_c     = cache['tanh_c']

    # step 1 - gradient from h = tanh_c * o
    d_tanh_c = dh * o
    d_o      = dh * tanh_c

    # step 2 - gradient through cell state
    dc_total = d_tanh_c * (1 - tanh_c**2) + dc

    # step 3 - gradient through each gate (chain
rule through sigmoid/tanh)
    d_pre_o = d_o * o * (1 - o)
    d_pre_f = (dc_total * c_prev) * f * (1 - f)
    d_pre_i = (dc_total * candidate) * i * (1 - i)
    d_pre_c = (dc_total * i) * (1 - candidate**2)

    # step 4 - weight gradients (accumulated across
timesteps; reset by zero_grad())
    self.dW_o += x.reshape(-1,1) @
d_pre_o.reshape(1,-1)
    self.dW_f += x.reshape(-1,1) @
d_pre_f.reshape(1,-1)
    self.dW_i += x.reshape(-1,1) @

```

```

d_pre_i.reshape(1,-1)
    self.dW_c += x.reshape(-1,1) @
d_pre_c.reshape(1,-1)

    self.dU_o += h_prev.reshape(-1,1) @
d_pre_o.reshape(1,-1)
    self.dU_f += h_prev.reshape(-1,1) @
d_pre_f.reshape(1,-1)
    self.dU_i += h_prev.reshape(-1,1) @
d_pre_i.reshape(1,-1)
    self.dU_c += h_prev.reshape(-1,1) @
d_pre_c.reshape(1,-1)

    self.db_o += d_pre_o
    self.db_f += d_pre_f
    self.db_i += d_pre_i
    self.db_c += d_pre_c

    # step 5 - pass gradients to previous timestep
    dx      = d_pre_o @ self.W_o.T + d_pre_f @
self.W_f.T + d_pre_i @ self.W_i.T + d_pre_c @
self.W_c.T
    dh_prev = d_pre_o @ self.U_o.T + d_pre_f @
self.U_f.T + d_pre_i @ self.U_i.T + d_pre_c @
self.U_c.T
    dc_prev = dc_total * f

    return dx, dh_prev, dc_prev

def forget_gate(self, x, h_prev):
    """Compute forget gate: sigmoid(x @ W_f +
h_prev @ U_f + b_f)."""
    output_x = x @ self.W_f
    output_h = h_prev @ self.U_f
    output = sigmoid(output_x + output_h +
self.b_f)
    return output

```

```

def input_gate(self, x, h_prev):
    """Compute input gate: sigmoid(x @ W_i + h_prev
    @ U_i + b_i)."""
    output_x = x @ self.W_i
    output_h = h_prev @ self.U_i
    output = sigmoid(output_x + output_h +
self.b_i)
    return output

def candidate(self, x, h_prev):
    """Compute candidate cell: tanh(x @ W_c +
h_prev @ U_c + b_c)."""
    output_x = x @ self.W_c
    output_h = h_prev @ self.U_c
    output = tanh(output_x + output_h + self.b_c)
    return output

def output_gate(self, x, h_prev):
    """Compute output gate: sigmoid(x @ W_o +
h_prev @ U_o + b_o)."""
    output_x = x @ self.W_o
    output_h = h_prev @ self.U_o
    output = sigmoid(output_x + output_h +
self.b_o)
    return output

def zero_grad(self):
    """Reset all weight gradient accumulators to
zero. Call before each new sample."""
    self.dW_o = np.zeros_like(self.W_o)
    self.dW_f = np.zeros_like(self.W_f)
    self.dW_i = np.zeros_like(self.W_i)
    self.dW_c = np.zeros_like(self.W_c)
    self.dU_o = np.zeros_like(self.U_o)
    self.dU_f = np.zeros_like(self.U_f)
    self.dU_i = np.zeros_like(self.U_i)
    self.dU_c = np.zeros_like(self.U_c)
    self.db_o = np.zeros_like(self.b_o)
    self.db_f = np.zeros_like(self.b_f)

```

```
self.db_i = np.zeros_like(self.b_i)
self.db_c = np.zeros_like(self.b_c)
```

Konstruktor (__init__) mendukung tiga mode inisialisasi:

- Jika keras_layer tersedia, bobot dimuat dari layer Keras yang sudah dilatih via get_weights(). hidden_dim di-derive dari shape W (W.shape[1] // 4).
- Jika W, U, b di-pass secara eksplisit, matriks-matriks tersebut langsung digunakan.
- Jika embed_dim dan hidden_dim di-pass, bobot diinisialisasi random dengan distribusi normal.

Setelah inisialisasi, matriks W, U, dan b di-slice menjadi komponen per-gate (W_i, W_f, W_c, W_o, dst.) agar forward pass lebih readable. zero_grad() juga dipanggil di akhir konstruktor untuk menginisialisasi semua accumulator gradien ke nol.

Fungsi **forward(self, x, h_prev, c_prev)** melakukan tiga hal:

- Menjalankan satu timestep LSTM.
- Menerima input x (embed_dim,), hidden state sebelumnya h_prev (hidden_dim,), dan cell state sebelumnya c_prev (hidden_dim,).
- Mengembalikan h, c, dan cache berisi semua nilai intermediate yang dibutuhkan backward pass.

Fungsi **backward(self, dh, dc, cache)** melakukan tiga hal:

- Menjalankan backprop satu timestep dan mengakumulasi gradient ke dW_*, dU_*, db_*.
- Menerima dh (gradient dari hidden state), dc (gradient dari cell state timestep berikutnya), dan cache dari forward.
- Mengembalikan dx, dh_prev, dc_prev untuk diteruskan ke timestep sebelumnya.

Selanjutnya adalah **Gate methods**, masing-masing gate diimplementasikan sebagai method tersendiri untuk modularitas:

Method	Formula	Fungsi
forget_gate(x, h_prev)	$\sigma(x @ W_f + h_{prev} @ U_f + b_f)$	Mutusin berapa % cell state lama dipertahankan
input_gate(x, h_prev)	$\sigma(x @ W_i + h_{prev} @ U_i + b_i)$	Mutusin info baru mana yang masuk

candidate(x, h_prev)	$\tanh(x @ W_c + h_{prev} @ U_c + b_c)$	Kandidat nilai baru untuk cell state
output_gate(x, h_prev)	$\sigma(x @ W_o + h_{prev} @ U_o + b_o)$	Mutusin bagian mana dari cell state yang jadi output

zero_grad(self) dibuat untuk mereset semua accumulator gradien (dW_* , dU_* , db_*) ke nol. Harus dipanggil sebelum memproses setiap sample baru agar gradien tidak terakumulasi lintas sample.

- [layers.py](#)

layers.py berisikan kelas LSTMDecoder yang merupakan implementasi arsitektur decoder image captioning berbasis LSTM dengan metode pre-inject. Kelas ini mengorkestrasikan seluruh pipeline dari CNN feature sampai distribusi probabilitas kata, serta menyediakan interface untuk training (Keras maupun from scratch) dan inferensi (greedy decoding).

class LSTMDecoder:

```
import numpy as np
from shared.dense import DenseLayer
from shared.embedding_layer import EmbeddingLayer
from lstm.lstm import LSTM
from shared.optimizer import AdamOptimizer
from shared.loss_function import
SparseCategoricalCrossEntropy

class LSTMDecoder:
    """
    Image captioning decoder using a single LSTM cell.

    Architecture:
        1. CNN features are projected to embed_dim via
        dense_proj (pre-injection).
        2. The projected feature is fed as the first
        LSTM input (timestep -1).
        3. Caption tokens are embedded and fed
        sequentially through the LSTM.
        4. At each timestep, dense_out maps the hidden
```

```

state to a vocab distribution.
"""
def __init__(
    self,
    lstm_keras_layer=None,
    embedding_keras_layer=None,
    dense_proj_keras_layer=None,
    dense_out_keras_layer=None,
    W=None, U=None, b=None,
    embed_dim=None,
    hidden_dim=None,
    vocab_size=None,
    dense_proj_input=None,
):
    """
    Initialize the decoder from Keras layers or
    dimension specs for random init.

    Args:
        lstm_keras_layer: Keras LSTM layer
        (optional, for weight transfer).
        embedding_keras_layer: Keras Embedding
        layer (optional).
        dense_proj_keras_layer: Keras Dense layer
        for CNN projection (optional).
        dense_out_keras_layer: Keras Dense output
        layer (optional).
        W, U, b: explicit LSTM weight arrays.
        embed_dim: embedding dimension.
        hidden_dim: LSTM hidden state dimension.
        vocab_size: number of tokens in vocabulary.
        dense_proj_input: input size for the CNN
        projection dense layer.
    """
    self.lstm = LSTM(lstm_keras_layer, W, U,
b, embed_dim, hidden_dim)
    self.embedding =
EmbeddingLayer(embedding_keras_layer, vocab_size,
embed_dim)

```

```

        self.dense_proj =
DenseLayer(dense_proj_keras_layer, dense_proj_input,
embed_dim)

        self.dense_out =
DenseLayer(dense_out_keras_layer, hidden_dim,
vocab_size)

        self.hidden_dim = hidden_dim if hidden_dim is
not None else self.lstm.hidden_dim
        self.embed_dim = embed_dim if embed_dim is not
None else self.lstm.embed_dim
        self.vocab_size = vocab_size

    def forward(self, cnn_features, caption_tokens):
        """
        Run a full forward pass over one (image,
caption) pair.

        Args:
            cnn_features: CNN feature vector, shape
(cnn_feature_dim,)
            caption_tokens: list of token indices
(input side, e.g. starttoken..last-1)

        Returns:
            output: dict mapping timestep index to
logit vector, shape (vocab_size,)
            cache: dict mapping timestep index to LSTM
cache (used in backward).
            Key -1 is the pre-injection
timestep.
        """
        # Run CNN pre-injection
        pre_inject =
self.dense_proj.forward(cnn_features)

        # initiate hidden state, cell state, and cache
h = np.zeros(self.hidden_dim)
c = np.zeros(self.hidden_dim)
cache = {}

```



```

        # Run timestep t = -1
        h, c, cache[-1] = self.lstm.forward(pre_inject,
h, c)

        # Run many to many
        i = 0
        output = {}
        for token in caption_tokens:
            embedded_token =
self.embedding.forward(token)
            h, c, cache[i] =
self.lstm.forward(embedded_token, h, c)
            output[i] = self.dense_out.forward(h)
            i += 1
        return output, cache

    def backward(self, cache, grad_outputs):
        """
        Run BPTT over all timesteps and accumulate
        gradients into each layer.

        Args:
            cache: dict returned by forward (keys: -1,
0, 1, ..., T-1).
            grad_outputs: dict mapping timestep index
to gradient of loss
                        w.r.t. the logit output at
that timestep, shape (vocab_size,).
        """
        dh = np.zeros(self.hidden_dim,)
        dc = np.zeros(self.hidden_dim,)

        for t in reversed(range(len(cache) - 1)): # -1
because cache[-1] is for pre-injection
            dh =
self.dense_out.backward(grad_outputs[t]) + dh #
accumulate BPTT gradient
            dx, dh, dc = self.lstm.backward(dh, dc,

```

```

cache[t])
        dx, dh, dc = self.lstm.backward(dh, dc,
cache[-1])
        self.dense_proj.backward(dx)

    def predict(self, cnn_features, start_token,
end_token, max_length=20):
        """
        Generate a caption via greedy decoding.

        Args:
            cnn_features: CNN feature vector, shape
(cnn_feature_dim,)
            start_token: integer index of the start
token.
            end_token: integer index of the end token
(generation stops here).
            max_length: maximum number of tokens to
generate.

        Returns:
            generated_tokens: list of predicted token
indices (excludes start token).
        """
        # Run CNN pre-injection
        pre_inject =
self.dense_proj.forward(cnn_features)

        # Initialize hidden state and cell state
        h = np.zeros(self.hidden_dim)
        c = np.zeros(self.hidden_dim)

        # Run pre-injection
        h, c, _ = self.lstm.forward(pre_inject, h, c)

        # Run caption generation
        i = 0
        last_generated_token = None
        generated_tokens = []

```

```

        while i < max_length and last_generated_token
!= end_token:
            if i == 0:
                embedded_token =
self.embedding.forward(start_token)
            else:
                embedded_token =
self.embedding.forward(last_generated_token)
            h, c, _ = self.lstm.forward(embedded_token,
h, c)

            output = self.dense_out.forward(h)
            token = np.argmax(output)
            generated_tokens.append(token)

            # Update loop conditions
            last_generated_token = token
            i += 1
        return generated_tokens

    def fit(self, features, captions, targets,
epochs=10, lr=0.001, use_keras=True, keras_model=None):
        """
        Train the decoder using either Keras or the
from-scratch implementation.

        Args:
            features: list of CNN feature vectors.
            captions: list of input token sequences
(one per sample).
            targets: list of target token sequences
(one per sample).
            epochs: number of training epochs.
            lr: learning rate for Adam (from-scratch
mode only).
            use_keras: if True, delegate training to
keras_model.fit.
            keras_model: compiled Keras Model (required
when use_keras=True).
        """

```

```

        if use_keras:
            if keras_model is None:
                raise ValueError("keras_model required
for Keras training")
            return keras_model.fit(features, targets,
epochs=epochs)

        optimizer = AdamOptimizer(learning_rate=lr)
        loss_fn = SparseCategoricalCrossEntropy()

        for epoch in range(epochs):
            total_loss = 0

            for cnn_feature, caption_tokens,
target_tokens in zip(features, captions, targets):
                self.lstm.zero_grad()

                # forward
                outputs, cache =
self.forward(cnn_feature, caption_tokens)

                # compute loss + grad_outputs per
timestep

                grad_outputs = {}
                for t in outputs:
                    total_loss +=
loss_fn.forward(outputs[t], target_tokens[t])
                    grad_outputs[t] =
loss_fn.backward(outputs[t], target_tokens[t])

                # backward
                self.backward(cache, grad_outputs)

                # update weights
                params = {
                    'W_i': self.lstm.W_i, 'W_f':
self.lstm.W_f,
                    'W_c': self.lstm.W_c, 'W_o':
self.lstm.W_o,

```

```

        'U_i': self.lstm.U_i, 'U_f':
self.lstm.U_f,
        'U_c': self.lstm.U_c, 'U_o':
self.lstm.U_o,
        'dense_out_W':
self.dense_out.weights,
        'dense_out_b': self.dense_out.bias,
        'dense_proj_W':
self.dense_proj.weights,
        'dense_proj_b':
self.dense_proj.bias,
    }
    grads = {
        'W_i': self.lstm.dW_i, 'W_f':
self.lstm.dW_f,
        'W_c': self.lstm.dW_c, 'W_o':
self.lstm.dW_o,
        'U_i': self.lstm.dU_i, 'U_f':
self.lstm.dU_f,
        'U_c': self.lstm.dU_c, 'U_o':
self.lstm.dU_o,
        'dense_out_W': self.dense_out.dW,
        'dense_out_b': self.dense_out.db,
        'dense_proj_W': self.dense_proj.dW,
        'dense_proj_b': self.dense_proj.db,
    }
    optimizer.step(params, grads)

    print(f"Epoch {epoch+1}/{epochs} - Loss:
{total_loss:.4f}")

```

Konstruktor (__init__) menginisialisasi empat komponen utama:

Komponen	Tipe	Fungsi
self.lstm	LSTM	Sel LSTM untuk memproses sequence

self.embedding	EmbeddingLayer	Mengubah token integer ke dense vector
self.dense_proj	DenseLayer	Memproyeksikan CNN feature (2048), ke (embed_dim,) sebagai x_{-1}
self.dense_out	DenseLayer	Memproyeksikan hidden state h_t ke (vocab_size,) untuk prediksi kata

Tiap komponen menerima Keras layer-nya masing-masing secara terpisah, atau dapat diinisialisasi random jika tidak ada.

Fungsi **forward(self, cnn_features, caption_tokens)** menjalankan full forward pass satu pasang (gambar, caption). Terdiri dari dua fase:

1. **Pre-injection (timestep $t = -1$):**

CNN feature vector diproyeksikan via dense_proj ke dimensi embedding, kemudian dimasukan ke LSTM sebagai input pertama dengan $h_0 = \text{zeros}$ dan $c_0 = \text{zeros}$.

2. **Pemrosesan caption (timestep $t = 0$ hingga $T-1$):**

Setiap token di caption_tokens di-embed via embedding, lalu diproses oleh LSTM. Output hidden state h_t diproyeksikan via dense_out menghasilkan logit vector shape (vocab_size,). Semua output dan cache disimpan per-timestep.

Forward mengembalikan output (dict logits per timestep) dan cache (dict cache LSTM per timestep, key -1 untuk pre-injection).

Fungsi **backward(self, cache, grad_outputs)** menjalankan BPTT mundur dari timestep terakhir ke pre-injection. Di setiap timestep, gradient dari dense_out dijumlahkan dengan dh dari timestep berikutnya (akumulasi BPTT), lalu lstm.backward dipanggil untuk mendapatkan dx , dh , dan dc . Setelah semua timestep caption selesai, lstm.backward dipanggil sekali lagi untuk timestep pre-injection (-1), dan dense_proj.backward dipanggil dengan dx yang dihasilkan.

Fungsi **predict(self, cnn_features, start_token, end_token, max_length=20)** menghasilkan caption via greedy decoding. Alurnya mirip forward, namun tidak ada ground truth, output token dari timestep

sebelumnya menjadi input timestep berikutnya. Generasi berhenti saat model memproduksi end_token atau mencapai max_length.

Fungsi `fit(self, features, captions, targets, epochs, lr, use_keras, keras_model)` menyediakan dua mode training:

1. **Keras mode (use_keras=True):** mendelegasikan training ke `keras_model.fit()`.
 2. **From-scratch mode:** menjalankan training loop manual, forward pass, hitung loss per timestep via `SparseCategoricalCrossEntropy`, backward pass, lalu update weights via `AdamOptimizer`.
- `train.py`
 `train.py` berisikan fungsi-fungsi yang membentuk pipeline training model Keras LSTM untuk image captioning. Berikut adalah code yang diimplementasikan:

```
import numpy as np
import json
from pathlib import Path
import tensorflow as tf

gpus = tf.config.list_physical_devices('GPU')
if gpus:
    for gpu in gpus:
        tf.config.experimental.set_memory_growth(gpu,
True)
    print(f"GPU enabled: {[gpu.name for gpu in gpus]}")
else:
    print("No GPU found, running on CPU")

from tensorflow import keras
from tensorflow.keras.applications import InceptionV3
from tensorflow.keras.layers import Input, LSTM, Dense,
Embedding, Concatenate
from tensorflow.keras.models import Model
from tensorflow.keras.preprocessing.text import
tokenizer_from_json

from shared.preprocessing import (
    load_captions, clean_and_wrap_captions,
```

```

build_tokenizer,
    calculate_max_length, captions_to_sequences_and_pad
)

# Config
CAPTIONS_PATH = 'data/captions.txt'
FEATURES_PATH = 'data/flickr8k_features.npy'
TOKENIZER_PATH = 'data/tokenizer.json'
WEIGHTS_PATH = 'weights/lstm_keras.weights.h5'

EMBED_DIM = 256
HIDDEN_DIM = 512
EPOCHS = 20
BATCH_SIZE = 64

# Load Data
def load_data():
    """
    Load CNN features, captions, and build the
    tokenizer.

    Returns:
        features: dict mapping image_id to feature
        vector, shape (2048,).
        tokenizer: fitted Keras tokenizer.
        cleaned_mapping: dict mapping image_id to list
        of cleaned caption strings.
        max_length: maximum caption length in tokens.
    """
    # load CNN features
    features = np.load(FEATURES_PATH,
allow_pickle=True).item()

    # load captions
    caption_mapping = load_captions(CAPTIONS_PATH)
    cleaned_mapping =
clean_and_wrap_captions(caption_mapping)

    # build tokenizer

```



```

        tokenizer, all_captions =
build_tokenizer(cleaned_mapping, TOKENIZER_PATH)
        max_length = calculate_max_length(all_captions)

        return features, tokenizer, cleaned_mapping,
max_length

# Prepare X, Y
def prepare_sequences(features, tokenizer,
cleaned_mapping, max_length, embed_dim):
    """
        Build teacher-forcing input/output pairs from
captions.

        For each caption of length T, produces T-1
(input_seq, cnn_feature) -> target_token
        pairs by shifting the sequence one step at a time.

        Args:
            features: dict mapping image_id to CNN feature
vector.
            tokenizer: fitted Keras tokenizer.
            cleaned_mapping: dict mapping image_id to list
of caption strings.
            max_length: sequence length to pad/truncate
inputs to.
            embed_dim: unused here, kept for interface
consistency.

        Returns:
            X: list of (cnn_feature, padded_input_seq)
tuples.
            y: numpy array of target token indices, shape
(N,).
    """
    vocab_size = len(tokenizer.word_index) + 1

    X, y = [], []

```

```

    for image_id, captions in cleaned_mapping.items():
        # ambil feature vector untuk gambar ini
        if image_id not in features:
            continue
        feature = features[image_id] # shape: (2048,)

        for caption in captions:
            # tokenize caption
            seq =
tokenizer.texts_to_sequences([caption])[0]

            # buat pasangan input-output dengan teacher
forcing
            for i in range(1, len(seq)):
                in_seq = seq[:i] # input:
semua token sebelum posisi i
                out_seq = seq[i] # target:
token di posisi i

                # padding input sequence
                in_seq =
keras.preprocessing.sequence.pad_sequences(
                    [in_seq], maxlen=max_length,
padding='post'
                )[0]

                X.append((feature, in_seq))
                y.append(out_seq)

    return X, np.array(y)

# Build Model
def build_keras_model(vocab_size, embed_dim,
hidden_dim, max_length, cnn_feature_dim=2048):
    """
    Build and compile the Keras LSTM captioning model.

    Uses a pre-injection architecture: CNN features are
    projected to embed_dim

```

and prepended to the embedded caption sequence before the LSTM.

Args:

 vocab_size: total number of tokens in the vocabulary.
 embed_dim: embedding and CNN projection dimension.
 hidden_dim: LSTM hidden state size.
 max_length: fixed caption input length (tokens).
 cnn_feature_dim: dimension of the input CNN feature vector (default 2048 for InceptionV3).

Returns:

 model: compiled Keras Model with Adam optimizer and sparse CCE loss.

"""

CNN feature input

cnn_input = Input(shape=(cnn_feature_dim,),
name='cnn_input')

cnn_proj = Dense(embed_dim, activation='relu',
name='cnn_proj')(cnn_input)

cnn_proj = keras.layers.Reshape((1,
embed_dim))(cnn_proj) *# (1, embed_dim)*

caption sequence input

cap_input = Input(shape=(max_length,),
name='cap_input')

cap_embed = Embedding(vocab_size, embed_dim,
mask_zero=True)(cap_input)

pre-inject: concat CNN feature di depan sequence

merged = Concatenate(axis=1)([cnn_proj,
cap_embed]) *# (max_length+1, embed_dim)*

LSTM decoder

lstm_out = LSTM(hidden_dim,
return_sequences=False)(merged)

```

        # output
        outputs = Dense(vocab_size,
activation='softmax')(lstm_out)

        model = Model(inputs=[cnn_input, cap_input],
outputs=outputs)
        model.compile(
            optimizer='adam',
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy']
        )
        return model

# Train
def train():
    """Full training pipeline: load data, prepare
sequences, build model, train, and save weights."""
    print("Loading data...")
    features, tokenizer, cleaned_mapping, max_length =
load_data()

    vocab_size = len(tokenizer.word_index) + 1
    print(f"vocab_size={vocab_size},
max_length={max_length}")

    print("Preparing sequences...")
    X, y = prepare_sequences(features, tokenizer,
cleaned_mapping, max_length, EMBED_DIM)

    # split X jadi cnn_features dan cap_sequences
    cnn_features = np.array([x[0] for x in X]) # (N,
2048)
    cap_sequences = np.array([x[1] for x in X]) # (N,
max_length)

    print(f"Training samples: {len(y)}")

    print("Building model...")

```

```

        model = build_keras_model(vocab_size, EMBED_DIM,
HIDDEN_DIM, max_length)
        model.summary()

        print("Training...")
        model.fit(
            [cnn_features, cap_sequences],
            y,
            epochs=EPOCHS,
            batch_size=BATCH_SIZE,
            validation_split=0.1,
            verbose=1
        )

        Path('weights').mkdir(exist_ok=True)
        model.save_weights(WEIGHTS_PATH)
        print(f"Weights saved to {WEIGHTS_PATH}")

if __name__ == '__main__':
    train()

```

Fungsi **def load_data()**:

Fungsi `load_data` memuat CNN features yang sudah diekstrak, memproses caption teks, dan membangun tokenizer. Fungsi ini memanggil `load_captions` dan `clean_and_wrap_captions` untuk memuat dan membersihkan caption dari file `captions.txt`, kemudian `build_tokenizer` untuk membangun vocabulary dan `calculate_max_length` untuk menentukan panjang sequence maksimum. Mengembalikan features (dict `image_id` → feature vector), tokenizer, `cleaned_mapping`, dan `max_length`.

Fungsi **`prepare_sequences()`** menyiapkan pasangan input-output untuk training dengan metode Teacher Forcing. Untuk setiap caption panjang `T`, fungsi menghasilkan `T-1` pasangan dengan cara sliding window, input adalah prefix sequence hingga posisi `i`, dan target adalah token di posisi `i`. Setiap input sequence di-pad ke panjang `max_length` dengan post-padding. Output-nya adalah list `X` berisi tuple (`cnn_feature`, `padded_input_seq`) dan array `y` berisi token target.

Fungsi **build_keras_model(vocab_size, embed_dim, hidden_dim, max_length, cnn_feature_dim=2048):**

Fungsi `build_keras_model` membangun dan mengkompilasi arsitektur decoder LSTM menggunakan Keras Functional API dengan metode pre-inject. Arsitektur menerima dua input

- **cnn_input shape (2048,)** → diproyeksikan via Dense + ReLU ke (embed_dim,) → di-reshape ke (1, embed_dim)
- **cap_input shape (max_length,)** → diproses via Embedding layer ke (max_length, embed_dim)

Keduanya digabung via Concatenate menghasilkan sequence (max_length+1, embed_dim) dimana CNN feature berada di posisi pertama sebagai x_{-1} (pre-inject). Sequence ini diproses oleh LSTM layer dengan `return_sequences=False` menghasilkan hidden state terakhir, lalu Dense + Softmax menghasilkan distribusi probabilitas vocab. Model dikompilasi dengan optimizer Adam dan loss Sparse Categorical Crossentropy.

Fungsi **train()** adalah pipeline utama yang menggabungkan seluruh tahapan training:

1. Memanggil `load_data()` untuk memuat data
2. Memanggil `prepare_sequences()` untuk membuat pasangan training
3. Memisahkan `X` menjadi `cnn_features` shape (N, 2048) dan `cap_sequences` shape (N, max_length)
4. Membangun model via `build_keras_model()`
5. Melatih model dengan `model.fit()` menggunakan `validation_split=0.1`
6. Menyimpan bobot ke file .h5 di folder weights/

2.2. Forward Propagation

2.2.1. CNN

Forward propagation pada CNN for scratch diimplementasikan secara modular dimana setiap layer memiliki method forward tersendiri yang menerima input numpy array dan mengembalikan output numpy array. Bobot yang digunakan adalah bobot hasil pelatihan Keras yang dimuat ke dalam setiap layer. Berikut adalah penjelasan forward propagation untuk setiap layer yang digunakan pada arsitektur CNN.

Conv2DLayer

Operasi konvolusi 2D pada Conv2DLayer merupakan inti dari forward propagation CNN. Untuk setiap posisi (i, j) pada output feature map,

dilakukan operasi dot product antara patch input berukuran (kH, kW, C_in) dengan kernel berukuran (kH, kW, C_in, C_out), lalu ditambahkan bias. Secara matematis, output pada posisi (i, j) untuk filter ke-k adalah:

$$\text{out}[i, j, k] = \sum(\text{patch}[h, w, c] * \text{kernel}[h, w, c, k]) + \text{bias}[k]$$

Berikut adalah tahapan forward propagation pada Conv2DLayer:

1. Padding Input

Jika padding='same', input di-pad dengan zero padding agar ukuran output spasial sama dengan input dibagi stride. Jumlah padding dihitung sebagai:

```
pH = max((out_H - 1) * sH + self.kH - H, 0)
pW = max((out_W - 1) * sW + self.kW - W, 0)
```

Jika padding='valid', input tidak dimodifikasi.

2. Sliding Window Convolution

Untuk setiap posisi (i, j) pada output, diambil patch dari input yang dipadding berukuran (kH, kW, C_in), lalu dilakukan operasi einsum:

```
out[i, j] = np.einsum('hwc,hwck->k', patch, self.kernel) +
self.bias
```

Operasi ini secara efisien menghitung dot product antara patch dan seluruh filter sekaligus, menghasilkan vektor output berukuran (C_out,).

3. Simpan Pre-Aktivasi

Output sebelum aktivasi disimpan ke self.pre_act untuk keperluan backward propagation gradient ReLU.

4. Aplikasi Fungsi Aktivasi

Output konvolusi dilewatkan melalui fungsi aktivasi (ReLU pada arsitektur CNN yang digunakan):

```
out_activated = ReLU(out) = max(0, out)
```

5. Dukungan Batch Inference

Jika input berupa batch (N, H, W, C), proses di atas dijalankan untuk setiap sampel secara iteratif menggunakan _forward_single, kemudian hasilnya di-stack menjadi (N, out_H, out_W, C_out).

LocallyConnected2DLayer

Forward propagation LocallyConnected2DLayer serupa dengan Conv2DLayer, namun setiap posisi output (i, j) menggunakan kernel yang berbeda. Kernel memiliki shape (out_H*out_W, kH*kW*C_in, C_out)

dimana dimensi pertama merepresentasikan setiap posisi output. Berikut adalah tahapan forward propagation pada LocallyConnected2DLayer:

1. Padding Input

Sama dengan Conv2DLayer, dilakukan zero padding jika padding='same'.

2. Per-Position Convolution

Untuk setiap posisi (i, j) pada output, patch input di-flatten menjadi vektor 1D, kemudian dikalikan dengan kernel pada indeks posisi yang bersesuaian:

```
flat      = patch.flatten()
pos       = i * out_W + j
out[i, j] = flat @ self.kernel[pos] + self.bias[pos]
```

Berbeda dengan Conv2DLayer yang menggunakan kernel yang sama untuk semua posisi, di sini self.kernel[pos] unik untuk setiap posisi pos.

3. Simpan Pre-Aktivasi dan Aplikasi Aktivasi

Sama dengan Conv2DLayer, output sebelum aktivasi disimpan ke self.pre_act, lalu fungsi aktivasi diaplikasikan.

MaxPooling2DLayer

Forward propagation MaxPooling2DLayer melakukan downsampling dengan mengambil nilai maksimum dari setiap window sliding pada setiap channel. Berikut adalah tahapan forward propagation:

1. Sliding Window

Untuk setiap posisi (i, j) pada output, diambil window dari input berukuran (pH, pW, C):

```
out[i, j] = np.max(x[i*sH:i*sH+pH, j*sW:j*sW+pW, :],
axis=(0,1))
```

2. Simpan Input

Input x disimpan ke self.x_input karena backward propagation membutuhkan informasi posisi nilai maksimum.

3. Output Shape

Ukuran output spasial dihitung sebagai:

```
out_H = (H - pH) // sH + 1
out_W = (W - pW) // sW + 1
```

AveragePooling2DLayer

Forward propagation AveragePooling2DLayer serupa dengan MaxPooling namun menggunakan rata-rata alih-alih maksimum. Untuk setiap posisi (i, j) pada output:

```
out[i, j] = np.mean(x[i*sH:i*sH+pH, j*sW:j*sW+pW, :],  
axis=(0,1))
```

Setiap posisi dalam window berkontribusi sama terhadap output sehingga tidak ada informasi posisi yang perlu disimpan untuk backward (gradient dibagi rata).

GlobalAveragePooling2DLayer

Forward propagation GlobalAveragePooling2DLayer mereduksi seluruh dimensi spasial menjadi satu nilai per channel dengan mengambil rata-rata global di seluruh posisi (H, W).

```
single input (H, W, C) -> (C,)
return np.mean(x, axis=(0,1))
```

```
batch input (N, H, W, C) -> (N, C)
return np.mean(x, axis=(1,2))
```

Layer ini digunakan pada arsitektur Conv2D terbaik sebagai pengganti Flatten sebelum Dense layer. Keuntungannya adalah menghasilkan vektor fitur yang lebih ringkas dan tidak tergantung ukuran spasial input, serta mengurangi risiko overfitting karena tidak ada parameter tambahan.

FlattenLayer

Forward propagation FlattenLayer mereshape tensor multidimensi menjadi vektor 1D atau matriks 2D dengan urutan row-major (C order), konsisten dengan perilaku Keras.

```
single input (H, W, C) → (H*W*C,)
return x.flatten(order='C')
```

```
batch input (N, H, W, C) → (N, H*W*C)
return x.reshape(x.shape[0], -1)
```

Shape asli input disimpan ke self.x_shape untuk keperluan backward propagation.

ActivationLayer

Forward propagation ActivationLayer mengaplikasikan fungsi aktivasi pada input dan menyimpan input asli untuk backward.

```
self.x = x
self.out = self.activation(x)
return self.out
```

Fungsi aktivasi yang digunakan pada arsitektur CNN:

- ReLU pada output Conv2DLayer dan Dense hidden layer: $\max(0, x)$
- Softmax pada Dense output layer: $\exp(x) / \sum(\exp(x))$

DenseLayer

Forward propagation DenseLayer melakukan operasi perkalian matriks antara input dan bobot, ditambah bias:

```
self.x = x
return x @ self.weights + self.bias
```

Input disimpan ke self.x untuk keperluan perhitungan gradient pada backward propagation. Aktivasi tidak diaplikasikan di dalam DenseLayer karena dipisah menggunakan ActivationLayer agar implementasi lebih modular.

Alur Forward Propagation Keseluruhan

Berikut adalah alur forward propagation lengkap pada arsitektur CNN terbaik (conv3_large_k5_average, 3 layer Conv2D dengan filter [64, 128, 128], kernel 5×5, dan average pooling):

1. Input (N, 150, 150, 3)
2. Conv2DLayer(64 filter, 5×5, same) + ReLU -> (N, 150, 150, 64)
3. AveragePooling2DLayer(2×2) -> (N, 75, 75, 64)
4. Conv2DLayer(128 filter, 5×5, same) + ReLU -> (N, 75, 75, 128)
5. AveragePooling2DLayer(2×2) -> (N, 37, 37, 128)
6. Conv2DLayer(128 filter, 5×5, same) + ReLU -> (N, 37, 37, 128)
7. AveragePooling2DLayer(2×2) -> (N, 18, 18, 128)
8. GlobalAveragePooling2DLayer -> (N, 128)
9. DenseLayer(128) + ReLU -> (N, 128)
10. DenseLayer(6) + Softmax -> (N, 6)
11. Output: distribusi probabilitas 6 kelas

Seluruh alur ini dijalankan oleh CNNScratch.forward() yang memanggil layer.forward(out) secara sekuensial untuk setiap layer dalam self.layers, dengan bobot yang identik dengan hasil pelatihan Keras sehingga hasil prediksi from scratch sama persis dengan Keras (selisih F1-score = 0.000000).

2.2.2. RNN

Forward propagation pada Simple Recurrent Neural Network (RNN) untuk persoalan *image captioning* adalah pemrosesan token kata sekuensial pada setiap time step dengan mempertahankan memori kontekstual melalui Hidden State (h_t). RNN menggunakan bobot yang sama (w_{xh} , w_{hh} , b_h) untuk setiap time step dan proses forward pass dilakukan secara iteratif dengan unrolling di sepanjang sekuens dimana output dari satu timestep diteruskan sebagai input memori ke time step berikutnya. Setiap sel RNN menerima dua input utama dan menghasilkan Hidden State baru (h_t) sebagai memori. Input saat ini (x_t) merupakan representasi vektor hasil embedding dari token kata pada timestep t sedangkan Hidden State sebelumnya ($h_{(t-1)}$) diteruskan dari time step sebelumnya. Berikut adalah langkah-langkah perhitungan dari forward pass RNN:

1. Pertama-tama dilakukan perhitungan linear (net_h) dengan rumus berikut:

$$net_h = (x_t \cdot w_{xh}) + (h_{(t-1)} \cdot w_{hh}) + b_h$$

Keterangan:

$x_t \cdot w_{xh}$: Transformasi linear input kata saat ini

$h_{(t-1)} \cdot w_{hh}$: Transformasi linear memori dari time step sebelumnya

b_h : Vektor bias

2. Hasil net_h kemudian dilewatkan melalui fungsi aktivasi tanh untuk menghasilkan Hidden State baru (h_t): **$h_t = \tanh(net_h)$** . Fungsi tanh memetakan nilai ke rentang -1 hingga 1 untuk membantu menstabilkan nilai dan gradien.
3. h_t yang telah dihitung diteruskan ke Dense Output Layer untuk memprediksi probabilitas token kata berikutnya dan juga diteruskan ke time step $t+1$ sebagai h_{prev}

Pada implementasinya dalam RNNDecoder dengan metode Pre-Inject, berikut adalah tahapannya:

1. Time Step $t = -1$ (Pre-Inject)
Vektor fitur CNN setelah diproyeksikan melalui `projection_layer` dimasukkan sebagai input $x_{(-1)}$ dan hidden state awal (h_0) diinisialisasi dengan nol. Output dari langkah ini menjadi hidden state awal untuk memproses sekuens kata.
2. Time Step $t=0$ hingga T
Input x_t berupa embedding dari token kata (teacher forcing) dan $h_{(t-1)}$ adalah hidden state dari langkah sebelumnya. Proses ini berlanjut hingga semua token dalam sekuens diproses.

Setiap hasil h_t dan input terkaitnya akan disimpan dalam cache untuk digunakan dalam perhitungan backward propagation.

2.2.3. LSTM

Forward propagation pada LSTM untuk persoalan image captioning adalah pemrosesan token kata sekuensial pada setiap time step dengan mempertahankan dua state memori secara bersamaan: Hidden State (h_t) sebagai working memory jangka pendek dan Cell State (c_t) sebagai long-term memory. Berbeda dengan RNN, LSTM menggunakan empat gate (forget, input, candidate, output) untuk mengontrol aliran informasi secara selektif, sehingga mampu mengatasi masalah vanishing gradient pada sekuens panjang. LSTM menggunakan bobot yang sama (W, U, b) untuk setiap time step dan proses forward pass dilakukan secara iteratif di sepanjang sekuens. Setiap sel LSTM menerima tiga input utama dan menghasilkan Hidden State baru (h_t) serta Cell State baru (c_t). Input saat ini (x_t) merupakan representasi vektor hasil embedding dari token kata pada timestep t , Hidden State sebelumnya ($h_{(t-1)}$) diteruskan dari time step sebelumnya, dan Cell State sebelumnya ($c_{(t-1)}$) yang membawa informasi memori jangka panjang.

Berikut adalah langkah-langkah perhitungan dari forward pass LSTM:

1. Pertama-tama dilakukan perhitungan keempat gate secara paralel menggunakan rumus berikut:

$$\begin{aligned}\text{Forget Gate: } f_t &= \sigma(x_t @ W_f + h_{(t-1)} @ U_f + b_f) \\ \text{Input Gate: } i_t &= \sigma(x_t @ W_i + h_{(t-1)} @ U_i + b_i) \\ \text{Candidate: } g_t &= \tanh(x_t @ W_c + h_{(t-1)} @ U_c + b_c) \\ \text{Output Gate: } o_t &= \sigma(x_t @ W_o + h_{(t-1)} @ U_o + b_o)\end{aligned}$$

Keterangan:

- a. $x_t @ W_*$: Transformasi linear input kata saat ini untuk masing-masing gate
 - b. $h_{(t-1)} @ U_*$: Transformasi linear hidden state dari time step sebelumnya
 - c. b_* : Vektor bias masing-masing gate
 - d. σ : Fungsi sigmoid yang memetakan nilai ke rentang $[0, 1]$
 - e. $\tanh$: Fungsi aktivasi yang memetakan nilai ke rentang $[-1, 1]$
2. Hasil keempat gate digunakan untuk memperbarui Cell State (c_t) dengan rumus:

$$c_t = f_t \odot c_{(t-1)} + i_t \odot g_t$$

Keterangan

- a. $f_t \odot c_{(t-1)}$: Forget gate memutuskan berapa persen informasi cell state lama yang dipertahankan. Nilai mendekati 0 berarti informasi lama dilupakan, nilai mendekati 1 berarti dipertahankan.
 - b. $i_t \odot g_t$: Input gate (i_t) memutuskan seberapa banyak kandidat nilai baru (g_t) yang ditambahkan ke cell state.
 - c. $\odot$: Operasi perkalian element-wise (Hadamard product)
3. Cell State yang telah diperbarui kemudian digunakan untuk menghasilkan Hidden State baru (h_t):

$$h_t = o_t \odot \tanh(c_t)$$

Output gate (o_t) mengontrol bagian mana dari cell state yang diekspos sebagai output. $\tanh(c_t)$ memastikan nilai h_t berada dalam rentang $[-1, 1]$.

4. h_t yang telah dihitung diteruskan ke Dense Output Layer untuk memprediksi probabilitas token kata berikutnya dan juga diteruskan ke time step $t+1$ sebagai h_{prev} bersama c_t sebagai c_{prev} .

Pada implementasinya dalam LSTMDecoder dengan metode Pre-Inject, berikut adalah tahapannya:

1. **Time Step $t = -1$ (Pre-Inject)** Vektor fitur CNN setelah diproyeksikan melalui `dense_proj` dimasukkan sebagai input $x_{(-1)}$ dan hidden state awal (h_0) serta cell state awal (c_0) diinisialisasi dengan nol. Output h_0' dan c_0' dari langkah ini menjadi state awal untuk memproses sekuens kata.
2. **Time step $t = 0$ hingga $T-1$** Input x_t berupa embedding dari token kata (teacher forcing) dan $h_{(t-1)}$, $c_{(t-1)}$ adalah state dari langkah sebelumnya. Setiap h_t kemudian diproses oleh `dense_out` menghasilkan logit vector yang dikonversi ke distribusi probabilitas via softmax untuk memprediksi token berikutnya. Proses ini berlanjut hingga semua token dalam sekuens diproses. Setiap hasil h_t , c_t , dan seluruh nilai intermediate (f_t , i_t , g_t , o_t , c_t , $\tanh(c_t)$) disimpan dalam cache per timestep untuk digunakan dalam perhitungan backward propagation.

2.3. Backward Propagation

2.3.1. CNN

Backward propagation pada CNN from scratch diimplementasikan menggunakan algoritma backpropagation standar dimana gradient loss diteruskan mundur dari layer output hingga layer pertama. Setiap layer memiliki method backward tersendiri yang menerima `grad_out` (gradient dari layer berikutnya) dan mengembalikan `dx` (gradient terhadap input layer tersebut). Bobot yang digunakan adalah bobot hasil pelatihan Keras

sehingga backward propagation here berfungsi untuk memverifikasi aliran gradient, bukan untuk memperbarui bobot.

ActivationLayer

Backward propagation ActivationLayer menghitung gradient terhadap input berdasarkan jenis fungsi aktivasi.

ReLU:

Turunan ReLU adalah 1 jika input positif dan 0 jika input negatif atau nol. Gradient diteruskan hanya pada posisi dimana input asli bernilai positif:

$$dL/dx = \text{grad_out} * (x > 0)$$

```
return grad_out * (self.x > 0).astype(float)
```

Softmax:

Turunan Softmax menggunakan Jacobian yang disederhanakan. Untuk kasus dimana Softmax digabung dengan Cross-Entropy loss, gradient yang masuk ke Softmax sudah merupakan selisih antara prediksi dan label. Namun pada implementasi ini, gradient Softmax dihitung secara umum menggunakan rumus:

$$dL/dx = s * (\text{grad_out} - \text{dot})$$

dimana $\text{dot} = \sum(\text{grad_out} * s)$

```
s = self.out
dot = np.sum(grad_out * s, axis=-1, keepdims=True)
return s * (grad_out - dot)
```

Linear:

Gradient diteruskan langsung tanpa modifikasi karena turunan fungsi linear adalah 1:

$$dL/dx = \text{grad_out}$$

Conv2DLayer

Backward propagation Conv2DLayer menghitung tiga gradient utama: gradient terhadap input (dx), gradient terhadap kernel (dkernel), dan gradient terhadap bias (dbias). Berikut adalah tahapan backward propagation pada Conv2DLayer:

1. Gradient Aktivasi ReLU

Sebelum menghitung gradient Conv2D, gradient dari layer berikutnya dimodifikasi terlebih dahulu menggunakan mask dari `self.pre_act` (output sebelum aktivasi yang disimpan saat forward):

```
grad_out = grad_out * (pre_act > 0)
```

Ini memastikan gradient hanya mengalir pada posisi yang melewati ReLU saat forward.

2. Gradient Kernel (dkernel)

Untuk setiap posisi (i, j) pada output dan setiap sampel n, gradient kernel dihitung sebagai outer product antara patch input dan gradient output menggunakan einsum:

$$dL/dkernel[h,w,c,k] = \Sigma(patch[h,w,c] * grad_out[n,i,j,k])$$

```
self.dkernel += np.einsum('hwc,k->hwck', patch, g)
```

Gradient diakumulasikan dari seluruh posisi output dan seluruh sampel dalam batch karena kernel yang sama digunakan untuk semua posisi (shared parameter).

3. Gradient Bias (dbias)

Gradient bias adalah jumlah dari seluruh gradient output pada setiap channel:

$$dL/dbias[k] = \Sigma(grad_out[n,i,j,k])$$

```
self.dbias += g
```

4. Gradient Input (dx)

Gradient terhadap input dihitung dengan mengalikan gradient output dengan kernel yang di-transposisi, diakumulasikan ke seluruh posisi yang berkontribusi terhadap output tersebut:

$$dL/dx[i*sH:i*sH+kH, j*sW:j*sW+kW, c] += \Sigma(grad_out[n,i,j,k] * kernel[h,w,c,k])$$

```
dx_pad[i*sH:i*sH+self.kH, j*sW:j*sW+self.kW, :] += \
    np.einsum('k,hwck->hwc', g, self.kernel)
```

5. Hapus Padding

Jika `padding='same'`, padding dihapus dari `dx_pad` untuk mengembalikan gradient dengan shape yang sama dengan input asli.

LocallyConnected2DLayer

Backward propagation LocallyConnected2DLayer serupa dengan Conv2DLayer namun gradient kernel dihitung per posisi output karena setiap posisi memiliki kernel tersendiri. Berikut adalah tahapan backward propagation:

1. Gradient Aktivasi ReLU

Sama dengan Conv2DLayer, gradient dimodifikasi menggunakan self.pre_act:

```
grad_out = grad_out * (pre_act > 0)
```

2. Gradient Kernel per Posisi (dkernel)

Untuk setiap posisi (i, j), patch input di-flatten dan gradient kernel pada posisi tersebut dihitung menggunakan outer product:

```
dL/dkernel[pos] += flat @ g
```

```
flat = patch.flatten()
pos = i * out_W + j
self.dkernel[pos] += np.outer(flat, g)
```

Berbeda dari Conv2DLayer, gradient tidak diakumulasikan lintas posisi karena setiap posisi memiliki kernel sendiri (non-shared parameter).

3. Gradient Bias per Posisi (dbias)

```
self.dbias[pos] += g
```

4. Gradient Input (dx)

Gradient input dihitung dengan mengalikan gradient output dengan kernel pada posisi tersebut:

```
dx_flat = kernel[pos] @ g # shape: (kH*kW*C_in,)

dx_flat = self.kernel[pos] @ g
dx_pad[i*sH:i*sH+kH, j*sW:j*sW+kW, :] +=
dx_flat.reshape(kH, kW, -1)
```

MaxPooling2DLayer

Backward propagation MaxPooling2DLayer meneruskan gradient hanya ke posisi yang merupakan nilai maksimum dalam window saat forward pass. Posisi lain mendapatkan gradient nol. Berikut adalah tahapan backward propagation:

1. Buat Mask Posisi Maksimum

Untuk setiap window, dibuat mask boolean yang bernilai True hanya pada posisi nilai maksimum:

```
max_val = np.max(window, axis=(0,1), keepdims=True)
mask     = (window == max_val).astype(float)
```

2. Tie-Breaking

Jika terdapat lebih dari satu nilai maksimum yang sama dalam satu window (tie), gradient dibagi rata di antara seluruh posisi tersebut:

```
mask /= (mask.sum(axis=(0,1), keepdims=True) + 1e-8)
```

3. Distribusi Gradient

Gradient dari layer berikutnya dikalikan dengan mask dan diakumulasikan ke posisi yang bersesuaian:

```
dL/dx[i*sH:i*sH+pH, j*sW:j*sW+pW, :] += mask *
grad_out[n,i,j,:]

dx[i*sH:i*sH+pH, j*sW:j*sW+pW, :] += mask * grad_out[n, i,
j, :]
```

AveragePooling2DLayer

Backward propagation AveragePooling2DLayer mendistribusikan gradient secara merata ke seluruh posisi dalam window karena setiap posisi berkontribusi sama terhadap output rata-rata. Untuk setiap window pada posisi (i, j), gradient dibagi rata ke seluruh pH * pW posisi:

```
dL/dx[i*sH:i*sH+pH, j*sW:j*sW+pW, :] += grad_out[n,i,j,:] / (pH *
pW)

dx[i*sH:i*sH+pH, j*sW:j*sW+pW, :] += grad_out[n, i, j, :] / (pH * pW)
```

Tidak diperlukan informasi dari forward pass karena distribusi gradient tidak bergantung pada nilai input.

GlobalAveragePooling2DLayer

Backward propagation GlobalAveragePooling2DLayer mendistribusikan gradient secara merata ke seluruh posisi spasial (H, W) untuk setiap channel. Gradient yang masuk berukuran (N, C) dan perlu didistribusikan kembali ke shape (N, H, W, C). Karena setiap posisi spasial berkontribusi sama terhadap rata-rata global, gradient untuk setiap posisi adalah:

```
dL/dx[n, h, w, c] = grad_out[n, c] / (H * W)
```

```
dx[n] = grad_out[n] / (H * W)
```

GlobalMaxPooling2DLayer

Backward propagation GlobalMaxPooling2DLayer meneruskan gradient hanya ke posisi nilai maksimum global per channel, serupa dengan MaxPooling2DLayer namun mencakup seluruh dimensi spasial. Untuk setiap channel c dan setiap sampel n :

```
max_val = np.max(x[n, :, :, c])
mask = (x[n, :, :, c] == max_val).astype(float)
mask /= (mask.sum() + 1e-8)
dx[n, :, :, c] = mask * grad_out[n, c]
```

FlattenLayer

Backward propagation FlattenLayer mereshape gradient kembali ke shape input asli yang disimpan saat forward pass:

```
dL/dx = grad_out.reshape(self.x_shape)

return grad_out.reshape(self.x_shape)
```

DenseLayer

Backward propagation DenseLayer menghitung tiga gradient:

1. Gradient Input (dL/dx)

Gradient terhadap input dihitung dengan mengalikan gradient output dengan transpose bobot:

$$dL/dx = \text{grad_out} @ W^T$$

```
dL_dx = grad_out @ self.weights.T
```

2. Gradient Bobot (dW)

Gradient bobot dihitung dengan mengalikan input yang disimpan saat forward dengan gradient output:

$$dL/dW = x^T @ \text{grad_out}$$

```
self.dW = self.x.T @ grad_out
```

3. Gradient Bias (db)

Gradient bias adalah jumlah gradient output sepanjang dimensi batch:

$$dL/db = \Sigma(\text{grad_out}, \text{axis}=0)$$

```
self.db = grad_out.sum(axis=0)
```

Alur Backward Propagation Keseluruhan

Backward propagation dijalankan oleh `CNNScratch.backward()` yang memanggil `layer.backward(grad)` secara terbalik dari layer output hingga layer pertama:

1. grad dari loss
2. `ActivationLayer(softmax)` -> dx
3. `DenseLayer(6)` -> dx, dW, db
4. `ActivationLayer(relu)` -> dx
5. `DenseLayer(128)` -> dx, dW, db
6. `GlobalAveragePooling2DLayer` -> dx (N, 18, 18, 128)
7. `AveragePooling2DLayer(2×2)` -> dx (N, 37, 37, 128)
8. `ActivationLayer(relu)` -> dx
9. `Conv2DLayer(128, 5×5)` -> dx, dkernel, dbias
10. `AveragePooling2DLayer(2×2)` -> dx (N, 75, 75, 128)
11. `ActivationLayer(relu)` -> dx
12. `Conv2DLayer(128, 5×5)` -> dx, dkernel, dbias
13. `AveragePooling2DLayer(2×2)` -> dx (N, 150, 150, 64)
14. `ActivationLayer(relu)` -> dx
15. `Conv2DLayer(64, 5×5)` -> dx, dkernel, dbias
16. dx terhadap input gambar (N, 150, 150, 3)

Setiap layer yang tidak memiliki method backward (seperti Dropout) dilewati secara otomatis oleh `CNNScratch.backward()` berkat pengecekan `if hasattr(layer, 'backward')`.

2.3.2. RNN

Backward propagation pada Simple Recurrent Neural Network (RNN) menerapkan algoritma **Backpropagation Through Time (BPTT)**. Algoritma ini diperlukan karena sifat RNN yang menggunakan **Parameter Sharing** dimana set bobot yang sama (w_{xh} , w_{hh} , dan b_h) digunakan berulang kali di setiap timestep untuk memproses sekuens input. Karena bobot-bobot ini berkontribusi terhadap error di setiap timestep, gradien yang dihitung pada setiap timestep harus diakumulasikan untuk mendapatkan pembaruan bobot yang akurat.

Berdasarkan implementasi pada kelas RNN dan RNNDecoder, proses BPTT dilakukan dengan cara unrolling RNN sepanjang sekuens waktu dan mengalirkan gradien mundur dari timestep terakhir (T) menuju awal (0). Berikut adalah tahapan lengkap dari proses perhitungannya:

1. **Gradien Output dan Hidden State**

Pada setiap timestep t , gradien mula-mula berasal dari output_layer (d_logits) yang menghasilkan gradien terhadap hidden state saat ini (dh_output). Gradien total untuk hidden state (dh) adalah jumlah dari gradien output tersebut dan gradien yang diteruskan dari timestep berikutnya (dh_next).

2. **Turunan Aktivasi Tanh**

Gradien total (dh) kemudian melewati fungsi aktivasi tanh. Sesuai metode `rnn_cell.backward`, turunan fungsi ini adalah $dtanh = dh * (1 - h^2)$, dimana h adalah output hidden state pada timestep tersebut.

3. **Kalkulasi Gradien Bobot**

Semua gradien pada masing-masing bobot (dw_xh , dw_hh , dan db_h) dihitung melalui operasi perkalian matriks antara input terkait (x atau h_prev) dengan $dtanh$

4. **Akumulasi Gradien**

Karena menerapkan parameter sharing, gradien dari setiap timestep ditambahkan ke variabel akumulasi dw_xh_total , dw_hh_total , db_h_total .

5. **Propagasi ke Timestep Sebelumnya**

Gradien terhadap hidden state sebelumnya (dh_prev) dihitung untuk diteruskan sebagai dh_next bagi timestep $t-1$. Proses ini terus berulang hingga mencapai tahap pre-inject fitur gambar.

2.3.3. LSTM

Backward propagation pada LSTM menerapkan algoritma **Backpropagation Through Time (BPTT)** yang serupa dengan RNN, namun lebih kompleks karena LSTM memiliki dua state yang mengalir antar timestep (h_t dan c_t) serta empat gate yang masing-masing memiliki jalur gradiennya sendiri. Algoritma ini diperlukan karena sifat LSTM yang menggunakan Parameter Sharing dimana set bobot yang sama (W , U , dan b untuk setiap gate) digunakan berulang kali di setiap timestep. Karena bobot-bobot ini berkontribusi terhadap error di setiap timestep, gradien yang dihitung pada setiap timestep harus diakumulasikan untuk mendapatkan pembaruan bobot yang akurat.

Berdasarkan implementasi pada kelas LSTM dan LSTMDecoder, proses BPTT dilakukan dengan cara unrolling LSTM sepanjang sekuens waktu dan mengalirkan gradien mundur dari timestep terakhir ($T-1$) menuju pre-injection ($t=-1$). Berikut adalah tahapan lengkap dari proses perhitungan gradiennya:

1. **Gradien Output dan Hidden State:** Pada setiap timestep t , gradien mula-mula berasal dari dense_out (d_{logits}) yang menghasilkan gradien terhadap hidden state saat ini (dh_{output}). Gradien total untuk hidden state (dh) adalah jumlah dari gradien output tersebut dan gradien yang diteruskan dari timestep berikutnya (dh dari iterasi sebelumnya dalam loop mundur). Untuk gradien cell state (dc), nilainya juga diakumulasikan dari timestep berikutnya.
2. **Gradien melalui Hidden State $h_t = \tanh(c_t) \odot o_t$:** Gradien total dh kemudian dialirkan mundur melalui operasi yang membentuk h_t . Karena h_t merupakan hasil perkalian element-wise antara $\tanh(c_t)$ dan o_t , gradien terbagi ke dua jalur:
 - a. $d_{\tanh_c} = dh \odot o_t$ (gradien menuju cell state)
 - b. $d_o = dh \odot \tanh_c$ (gradien menuju output gate)
3. **Gradien melalui Cell State $c_t = f_t \odot c_{(t-1)} + i_t \odot g_t$:** Gradien dari jalur $\tanh$ digabung dengan gradien dc dari timestep berikutnya menggunakan turunan $\tanh$:

$$dc_{\text{total}} = d_{\tanh_c} \odot (1 - \tanh^2 c_t) + dc$$

$$dc_{\text{total}}$$
 kemudian dialirkan ke seluruh komponen yang membentuk c_t .
4. **Turunan Aktivasi Tiap Gate (Chain Rule melalui Sigmoid/Tanh):**
 Gradien dc_{total} dan d_o dialirkan mundur melalui fungsi aktivasi masing-masing gate. Turunan sigmoid $\sigma(z)$ adalah $\sigma(z) \odot (1 - \sigma(z))$, sedangkan turunan $\tanh$ adalah $(1 - \tanh^2(z))$:
 - a. $d_{\text{pre_o}} = d_o \odot o_t \odot (1 - o_t)$
 - b. $d_{\text{pre_f}} = dc_{\text{total}} \odot c_{(t-1)} \odot f_t \odot (1 - f_t)$
 - c. $d_{\text{pre_i}} = dc_{\text{total}} \odot g_t \odot i_t \odot (1 - i_t)$
 - d. $d_{\text{pre_g}} = dc_{\text{total}} \odot i_t \odot (1 - g_t^2)$
5. **Kalkulasi dan Akumulasi Gradien Bobot:** Gradien untuk setiap matriks bobot dihitung melalui outer product antara input terkait (x_t atau $h_{(t-1)}$) dengan gradien pre-aktivasi gate yang sesuai. Karena menerapkan parameter sharing, gradien dari setiap timestep diakumulasikan menggunakan operasi $+=$:
 - a. $dW_o += x_t.T \otimes d_{\text{pre_o}}$, $dU_o += h_{(t-1)}.T \otimes d_{\text{pre_o}}$,
 $db_o += d_{\text{pre_o}}$
 - b. $dW_f += x_t.T \otimes d_{\text{pre_f}}$, $dU_f += h_{(t-1)}.T \otimes d_{\text{pre_f}}$,
 $db_f += d_{\text{pre_f}}$
 - c. $dW_i += x_t.T \otimes d_{\text{pre_i}}$, $dU_i += h_{(t-1)}.T \otimes d_{\text{pre_i}}$,
 $db_i += d_{\text{pre_i}}$

$$d. \quad dW_g += x_t.T \otimes d_pre_g, \quad dU_g += h_{(t-1)}.T \otimes d_pre_g, \\ db_g += d_pre_g$$

($\otimes$ adalah outer product yang diimplementasikan via reshape dan matrix multiplication)

6. **Propagasi ke Timestep Sebelumnya** Gradien terhadap input x_t dan state sebelumnya (dh_prev , dc_prev) dihitung untuk diteruskan ke timestep t-1:

$$a. \quad dx = d_pre_o @ W_o.T + d_pre_f @ W_f.T + d_pre_i @ W_i.T + d_pre_g @ W_c.T$$

$$b. \quad dh_prev = d_pre_o @ U_o.T + d_pre_f @ U_f.T + d_pre_i @ U_i.T + d_pre_g @ U_c.T$$

$$c. \quad dc_prev = dc_total \odot f_t$$

dh_prev dan dc_prev menjadi dh dan dc untuk timestep t-1. Proses ini terus berulang hingga mencapai timestep pre-injection ($t=-1$), dimana dx yang dihasilkan diteruskan ke `dense_proj.backward(dx)` untuk mengalirkan gradien ke layer proyeksi CNN.

3. Pengujian

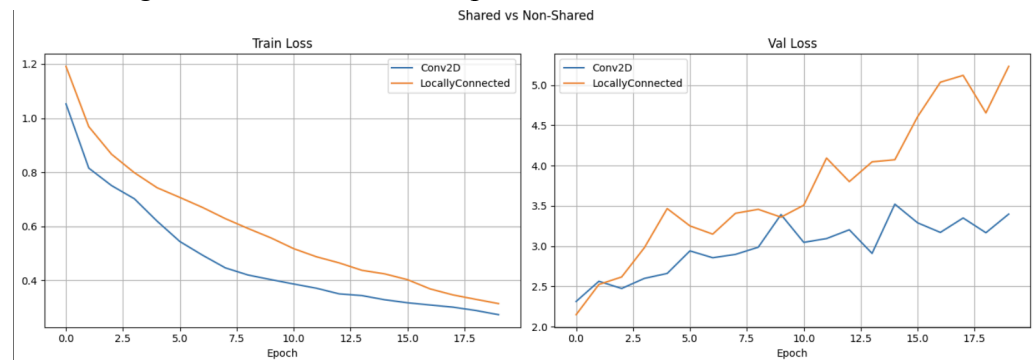
Features :  features

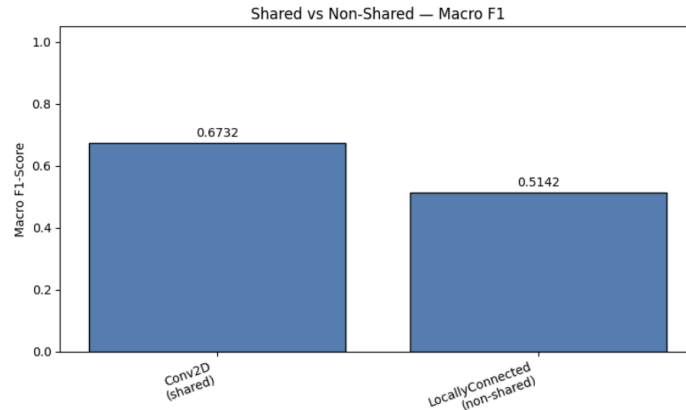
Weights :  weights

3.1. CNN

Pengujian CNN dilakukan menggunakan dataset Intel Image Classification dengan split train (10.257 gambar) dan test (3.000 gambar) untuk 6 kelas: buildings, forest, glacier, mountain, sea, dan street. Metrik yang digunakan adalah macro F1-score. Seluruh model dilatih selama 20 epoch dengan optimizer Adam dan loss Sparse Categorical Crossentropy.

3.1.1. Perbandingan shared vs non-shared parameter



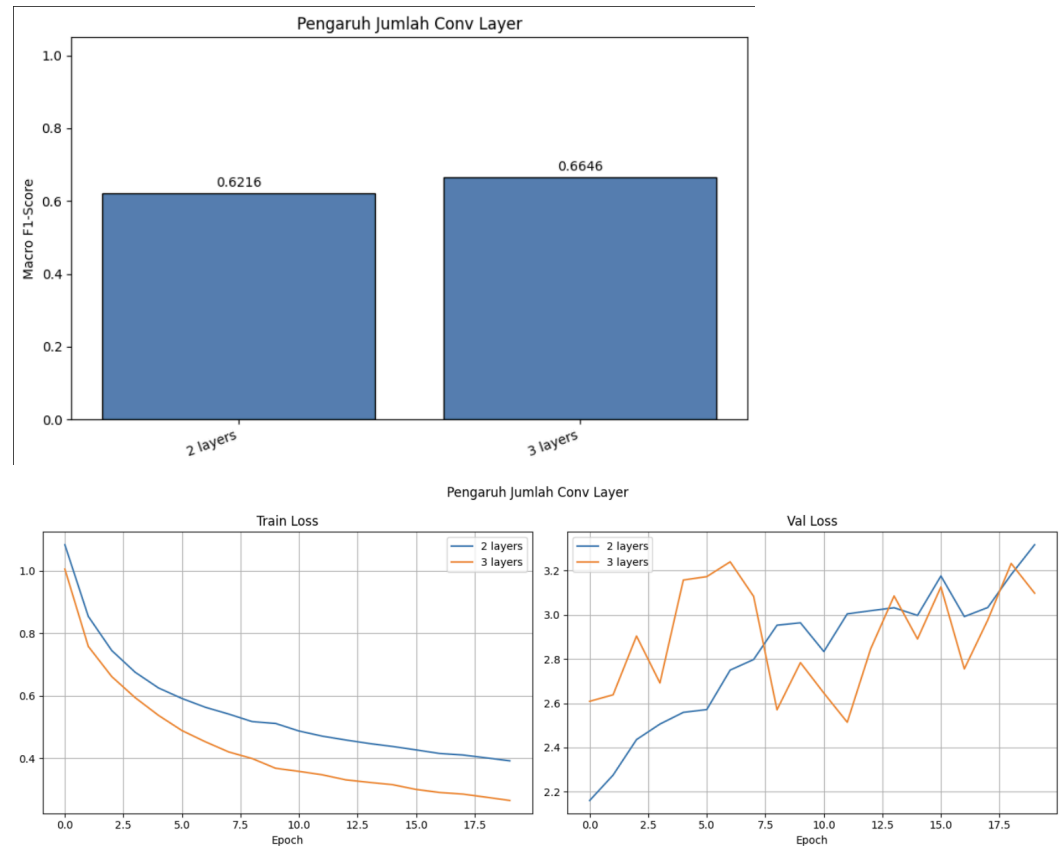


Perbandingan dilakukan antara arsitektur terbaik Conv2D (shared parameter) dengan LocallyConnected2D (non-shared parameter) menggunakan konfigurasi yang sama yaitu filter [64, 128, 128], kernel 5×5 , dan average pooling. Karena LocallyConnected2D memiliki jumlah parameter yang sangat besar untuk input 150×150 , ukuran input diturunkan menjadi 32×32 .

Model	Macro F1	Jumlah Parameter
Conv2D (Shared)	0.6732	636.806
LocallyConnected2D (Non-Shared)	0.5142	189.492.742

Dari hasil pengujian, Conv2D dengan shared parameter menghasilkan F1-score yang lebih tinggi (0.6732) dibandingkan LocallyConnected2D (0.5142) meskipun jumlah parameternya jauh lebih sedikit (636.806 vs 189.492.742). Hal ini menunjukkan bahwa parameter sharing pada Conv2D memberikan efek regularisasi implisit yang menguntungkan model dipaksa untuk mempelajari fitur yang berlaku secara umum di seluruh posisi spasial sehingga lebih mampu melakukan generalisasi. Sebaliknya, LocallyConnected2D dengan jumlah parameter yang sangat besar cenderung lebih mudah overfit terutama pada dataset dengan ukuran terbatas, dan juga membutuhkan memori serta waktu komputasi yang jauh lebih besar.

3.1.2. Pengaruh jumlah layer konvolusi

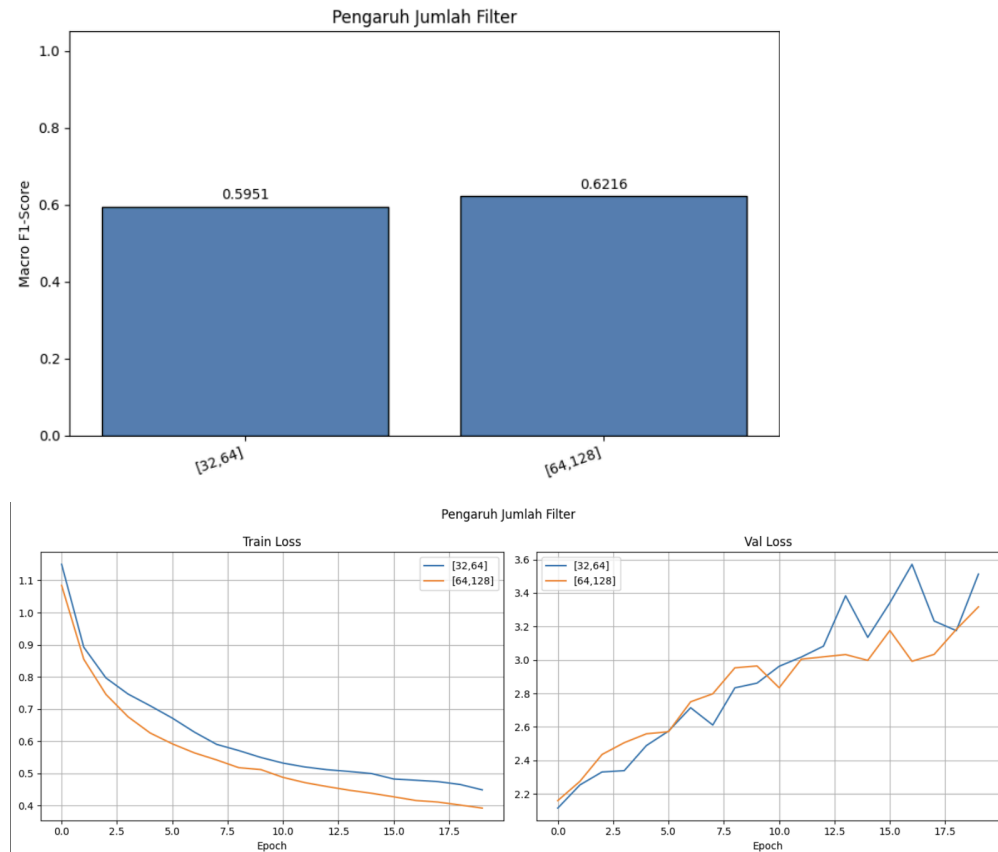


Perbandingan dilakukan antara arsitektur dengan 2 dan 3 layer konvolusi menggunakan konfigurasi filter [64, 128] dan [64, 128, 128], kernel 3×3, dan max pooling.

Jumlah Layer	Konfigurasi	Macro F1
2 layer	conv2_large_k3_max	0.6216
3 layer	conv3_large_k3_max	0.6646

Dari hasil pengujian, arsitektur dengan 3 layer konvolusi menghasilkan F1-score yang lebih tinggi (0.6646) dibandingkan 2 layer (0.6216). Penambahan layer konvolusi memungkinkan model untuk mempelajari representasi fitur yang lebih abstrak dan hierarkis, layer pertama mendeteksi fitur rendah seperti tepi dan tekstur, sedangkan layer ketiga dapat menangkap pola yang lebih kompleks dan semantis. Hal ini konsisten dengan prinsip deep learning bahwa kedalaman arsitektur berkorelasi dengan kemampuan representasi model.

3.1.3. Pengaruh banyak filter per layer konvolusi

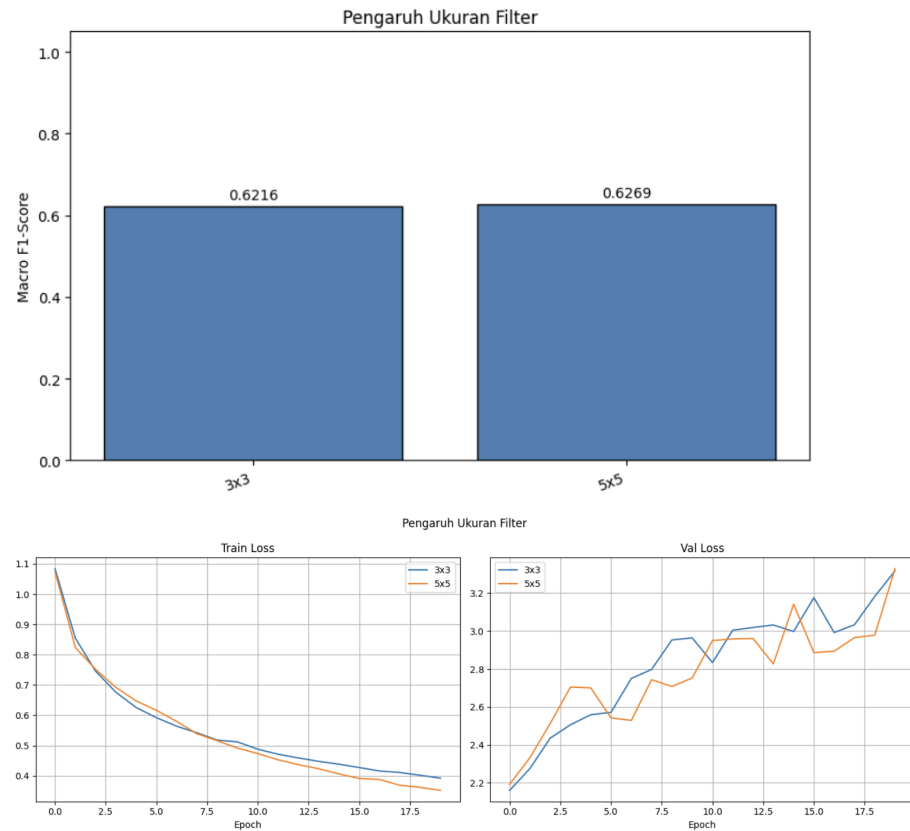


Perbandingan dilakukan antara dua variasi jumlah filter: small [32, 64] dan large [64, 128] untuk 2 layer konvolusi, serta small [32, 64, 64] dan large [64, 128, 128] untuk 3 layer konvolusi, menggunakan kernel 3×3 dan max pooling.

Variasi Filter	Konfigurasi	Macro F1
Small [32, 64]	conv2_small_k3_max	0.5951
Large [64, 128]	conv2_large_k3_max	0.6216

Dari hasil pengujian, konfigurasi filter yang lebih besar (large) menghasilkan F1-score yang lebih tinggi (0.6216) dibandingkan filter yang lebih kecil (small, 0.5951). Jumlah filter menentukan kapasitas representasi dari setiap layer, filter yang lebih banyak memungkinkan model menangkap lebih banyak variasi pola visual yang berbeda. Namun peningkatan ini tidak drastis karena dataset Intel memiliki karakteristik visual yang cukup berbeda antar kelas sehingga filter yang lebih sedikit pun sudah dapat menangkap fitur diskriminatif yang cukup.

3.1.4. Pengaruh ukuran filter per layer konvolusi

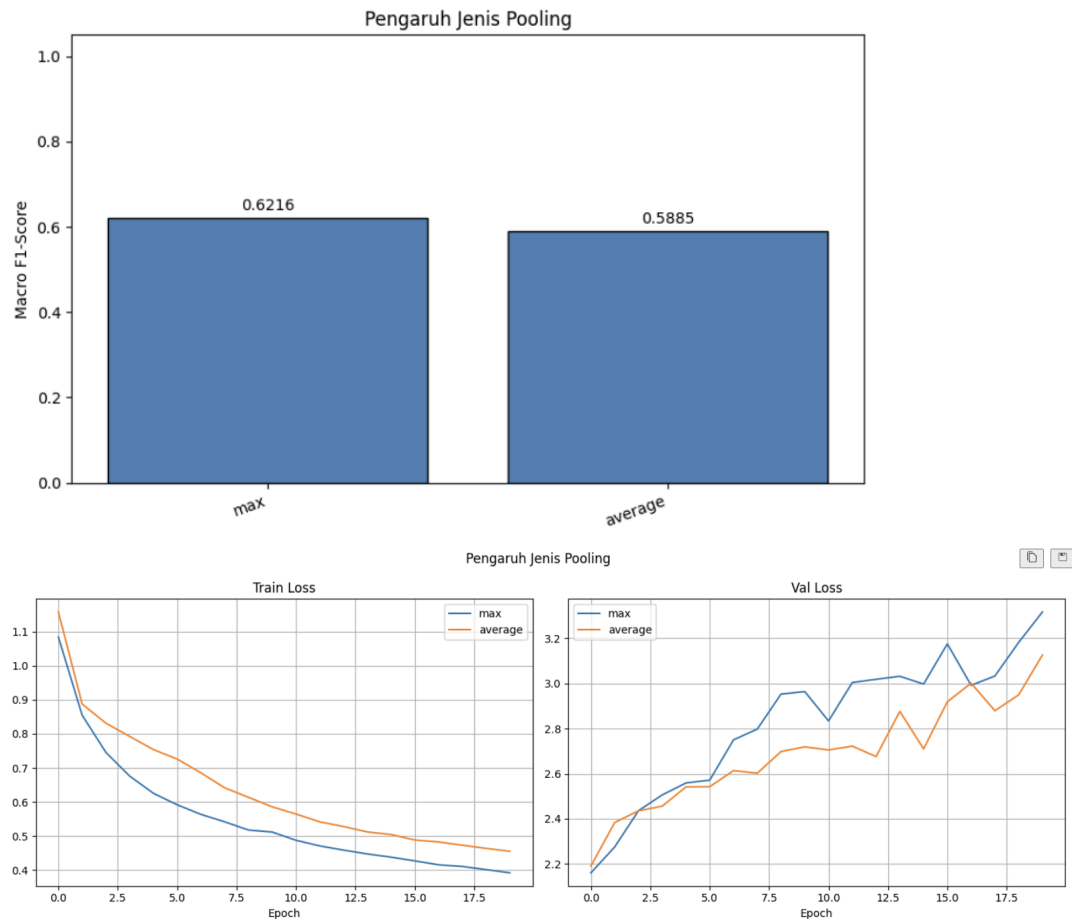


Perbandingan dilakukan antara dua ukuran kernel: 3×3 dan 5×5 , menggunakan konfigurasi filter large [64, 128] dan max pooling untuk 2 layer konvolusi.

Ukuran Filter	Konfigurasi	Macro F1
3×3	conv2_large_k3_max	0.6216
5×5	conv2_large_k5_max	0.6269

Dari hasil pengujian, kernel 5×5 menghasilkan F1-score yang lebih tinggi (0.6269) dibandingkan kernel 3×3 (0.6216) meskipun selisihnya kecil. Kernel yang lebih besar memiliki receptive field yang lebih luas sehingga setiap filter dapat menangkap konteks spasial yang lebih besar dalam satu operasi konvolusi. Untuk dataset Intel Image Classification yang memiliki objek dengan variasi tekstur dan struktur global yang signifikan (seperti glacier vs mountain), kemampuan menangkap konteks yang lebih luas ini sedikit menguntungkan. Namun selisih yang kecil menunjukkan bahwa untuk dataset ini perbedaan antara 3×3 dan 5×5 tidak terlalu signifikan.

3.1.5. Pengaruh jenis pooling layer



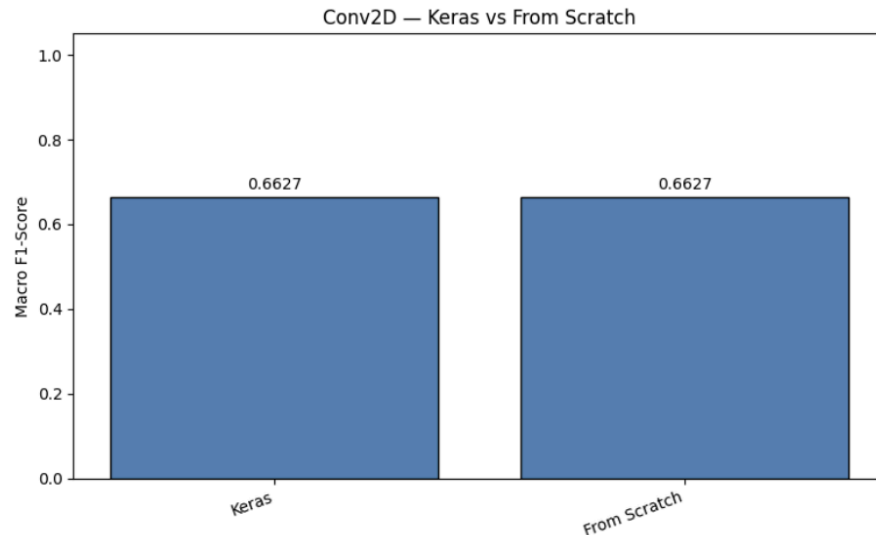
Perbandingan dilakukan antara max pooling dan average pooling menggunakan konfigurasi filter large [64, 128], kernel 3×3 , dan 2 layer konvolusi.

Jenis Pooling	Konfigurasi	Macro F1
Max Pooling	conv2_large_k3_max	0.6216
Average Pooling	conv2_large_k3_aver age	0.5885

Dari hasil pengujian, max pooling menghasilkan F1-score yang lebih tinggi (0.6216) dibandingkan average pooling (0.5885). Max pooling lebih efektif karena hanya meneruskan nilai aktivasi tertinggi dari setiap window, sehingga informasi tentang keberadaan fitur yang paling menonjol tetap dipertahankan meskipun lokasinya bergeser. Hal ini sesuai dengan task klasifikasi gambar dimana keberadaan fitur lebih penting daripada distribusi rata-ratanya. Average pooling di sisi lain menghaluskan informasi spasial yang dapat mengaburkan fitur diskriminatif. Namun menariknya, pada konfigurasi 3 layer dengan filter large dan kernel 5×5 ,

average pooling justru menghasilkan performa terbaik (conv3_large_k5_average, F1=0.6732), menunjukkan bahwa interaksi antara kedalaman arsitektur dan jenis pooling bersifat kompleks.

3.1.6. Perbandingan Keras vs from scratch



Perbandingan dilakukan antara forward propagation Keras dan from scratch menggunakan arsitektur terbaik (conv3_large_k5_average) pada 200 sampel test yang diambil secara acak merata dari seluruh kelas.

Model	Macro F1	Beda Prediksi
Keras	0.6627	-
From Scratch	0.6627	0 dari 200
Selisih	0.000000	-

Kelas	Precision	Recall	F1-Score	Support
buildings	0.0000	0.0000	0.0000	39
forest	0.9655	0.9655	0.9655	29
glacier	0.8696	0.6061	0.7143	33
mountain	0.7234	0.9444	0.8193	36
sea	0.6923	0.9000	0.7826	30
street	0.5323	1.0000	0.6947	33

macro avg	0.6305	0.7360	0.6627	200
-----------	--------	--------	--------	-----

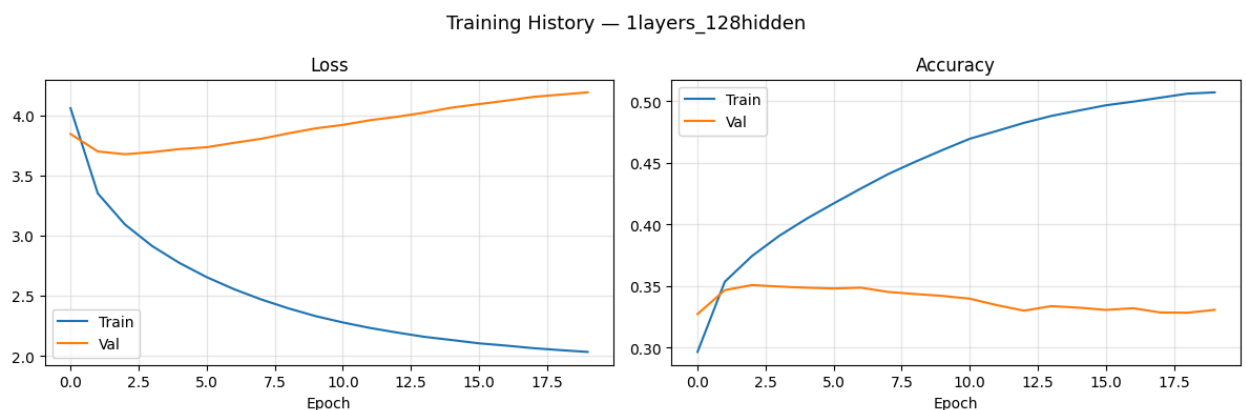
Model	Macro F1	Beda Prediksi
LC Keras	0.4566	-
LC From Scratch	0.4566	0 dari 200
Selisih	0.000000	-

Dari hasil pengujian, implementasi forward propagation from scratch menghasilkan prediksi yang identik dengan Keras untuk kedua arsitektur (Conv2D dan LocallyConnected2D) dengan selisih F1-score 0.000000 dan 0 prediksi yang berbeda dari 200 sampel. Hal ini membuktikan bahwa implementasi operasi konvolusi, pooling, dan dense layer from scratch menggunakan NumPy telah benar secara matematis dan konsisten dengan implementasi Keras. Kelas buildings memiliki F1-score 0 karena model cenderung memprediksi sampel buildings sebagai kelas lain, kemungkinan karena fitur visual buildings yang lebih ambigu dan tumpang tindih dengan kelas street.

3.2. RNN dan LSTM

3.2.1. Perbandingan jumlah layer: RNN

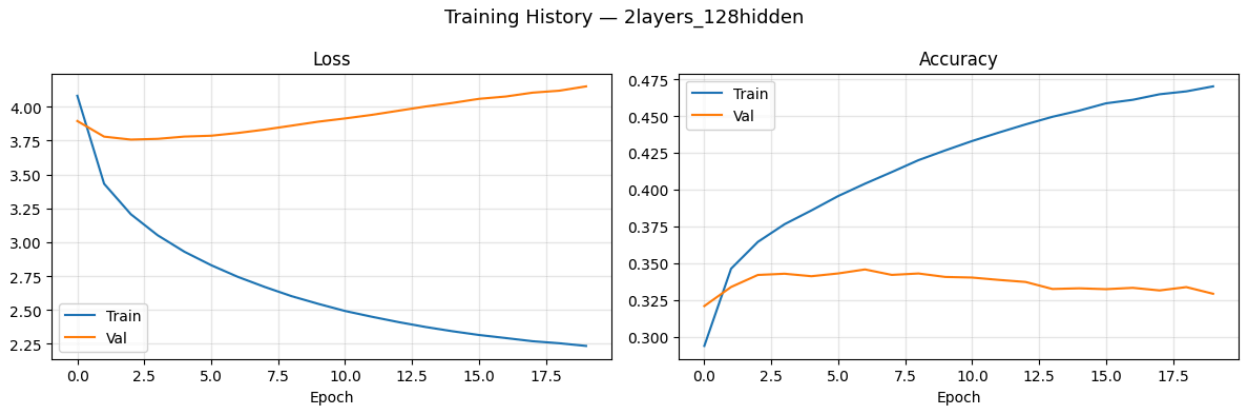
- 1 Layer



RNN dengan 1 layer menjadi model terbaik hasil training kami. Validation Accuracy mencapai puncaknya di sekitar 0.35 pada epoch 2-5 sebelum akhirnya menjadi datar. Kurva Train/Val Loss juga memiliki jarak yang paling ideal dibandingkan variasi lainnya. Dengan demikian, untuk dataset sekecil Flickr8k,

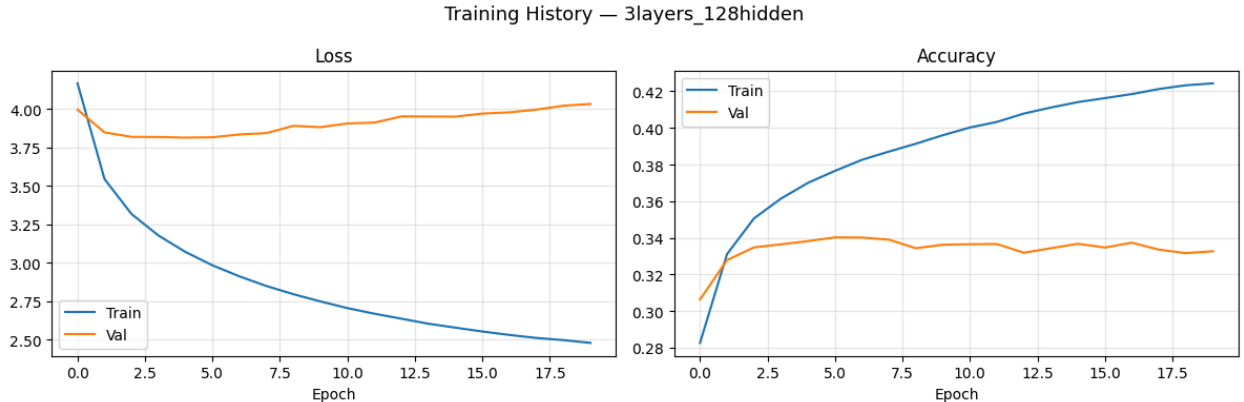
arsitektur sederhana dengan 1 layer sudah cukup untuk menangkap relasi antar kata.

- 2 Layer



Dapat dilihat pada kurva bahwa penambahan layer ke-2 tidak memberikan peningkatan performa yang signifikan. Validation Accuracy maksimal hanya tertahan di sekitar 0.34 dan model mengalami Diminishing Returns dimana penambahan kompleksitas tidak sebanding dengan hasil.

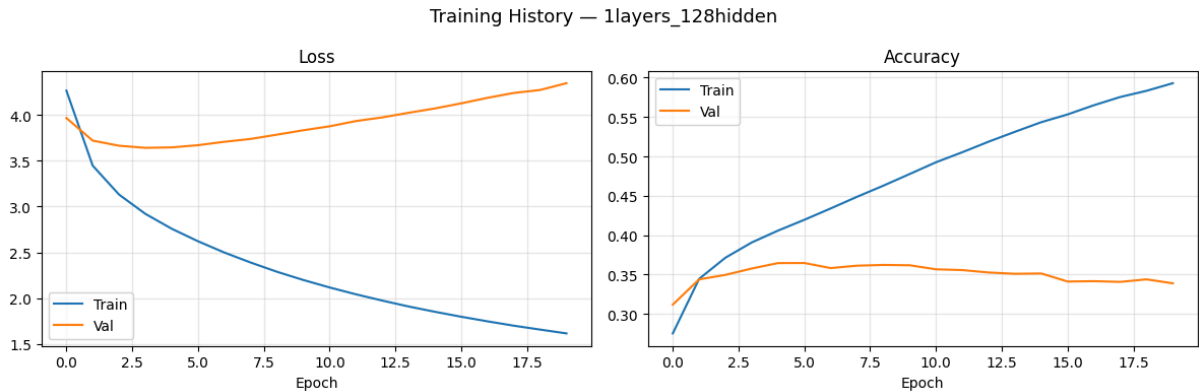
- 3 Layer



Sama seperti 2 layer, performa validasi mentok di 0.34. Namun, Training Accuracy di akhir epoch yaitu 0.42 lebih rendah daripada model 1 layer yang bernilai 0.50. Hal ini dikarenakan semakin dalam layer RNN, semakin sulit pula model mengalirkan untuk gradien sehingga model lebih lambat belajar dan cenderung mengalami Vanishing Gradient.

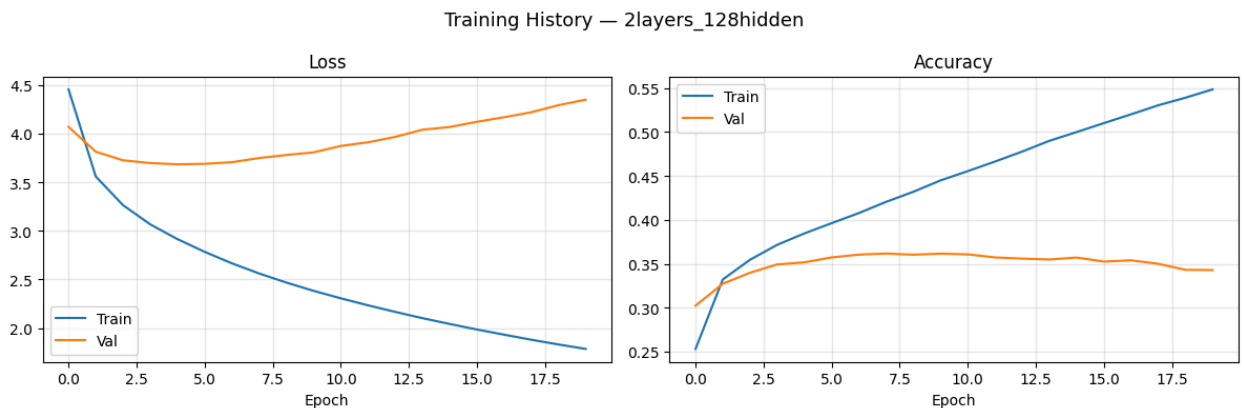
3.2.2. Perbandingan jumlah layer: LSTM

- 1 Layer



Kurva loss menunjukkan Training Loss turun dari 4.2 menuju 1.6 di epoch akhir, sementara Validation Loss sempat turun ke titik terendah sekitar 3.6 di epoch ke-2 sebelum perlahan naik kembali menuju 4.2 di epoch ke-20. Validation Accuracy mencapai puncak di sekitar 0.36 pada epoch ke-3 hingga ke-5, kemudian menjadi datar dan sedikit menurun di akhir epoch. Jarak antara Training Loss dan Validation Loss yang semakin melebar menunjukkan terjadinya overfitting, namun Validation Accuracy yang stabil di 0.35 mengindikasikan bahwa model masih mampu menangkap pola bahasa yang cukup baik dengan arsitektur sederhana.

- **2 Layer**



Pola kurva 2 layer sangat mirip dengan 1 layer, namun dengan sedikit perbedaan. Training Loss berakhir di sekitar 1.8 dan Validation Loss naik menuju 4.3 di akhir epoch, sedikit lebih tinggi dibandingkan variasi 1 layer. Validation Accuracy juga mencapai puncak di 0.36 namun sedikit lebih cepat pada epoch ke-2 hingga ke-3 sebelum menjadi stabil di 0.35. Penambahan layer kedua tidak memberikan peningkatan signifikan terhadap kemampuan generalisasi model, mengindikasikan bahwa kompleksitas tambahan tidak sebanding dengan peningkatan performa pada dataset Flickr8k yang relatif kecil.

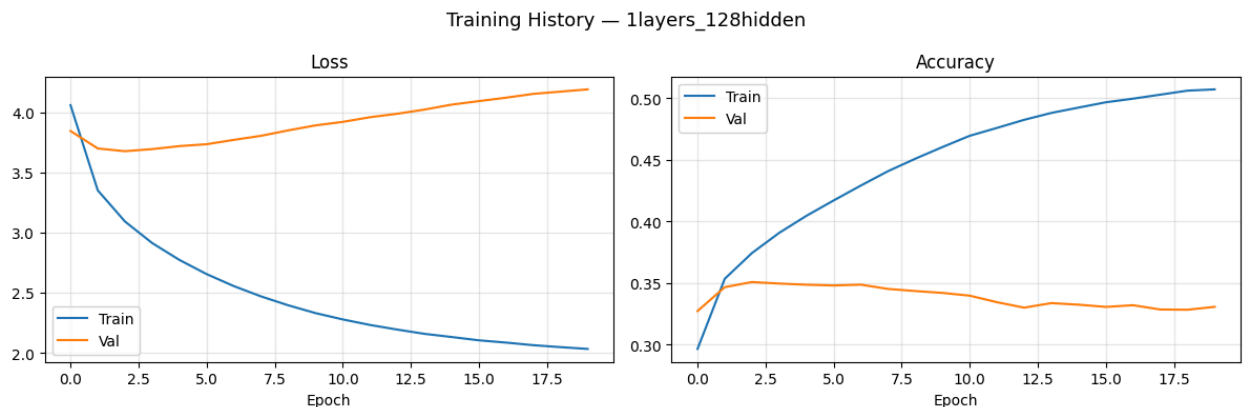
- **3 Layer**



Training Loss berakhir di sekitar 2.0, lebih tinggi dibandingkan variasi 1 dan 2 layer, sementara Validation Loss naik menuju 4.2. Validation Accuracy maksimal di 0.36 namun Training Accuracy di akhir epoch hanya mencapai 0.49, lebih rendah dari 2 variasi sebelumnya. Hal ini mengindikasikan bahwa penambahan layer ketiga justru mempersulit proses training karena semakin dalamnya jaringan membuat gradien semakin sulit mengalir ke layer awal. Meskipun LSTM lebih tahan terhadap vanishing gradient dibandingkan RNN berkat mekanisme cell state, penambahan layer yang berlebihan tetap memperlambat konvergensi dan tidak menghasilkan peningkatan performa yang berarti.

3.2.3. Perbandingan ukuran hidden state: RNN

- **Hidden state 128**



Grafik menunjukkan bahwa Training Loss turun perlahan mendekati 2.0. Meskipun Validation Loss mulai stagnan dan sedikit naik setelah epoch ke-3 (menuju angka 4.0-4.2), jarak antara loss pelatihan dan validasi masih dalam batas yang relatif wajar. Validation Accuracy berhasil naik secara konsisten di epoch-epoch awal dan mencapai puncak stabil di kisaran 0.35. Dari hasil ini, dapat disimpulkan bahwa dengan hidden state 128, kapasitas representasi memori model sudah cukup ideal untuk ukuran dataset Flickr8k. Keterbatasan parameter

memaksa model untuk benar-benar mempelajari pola bahasa secara umum (generalization) bukan sekadar menghafal.

- **Hidden state 512**



Model dengan 512 hidden state mengalami Overfitting yang cukup signifikan. Training Loss turun dengan sangat cepat dan mulus mendekati angka 2.5 namun Validation Loss langsung meroket tajam sejak epoch pertama dan berakhir di angka sekitar 4.6. Jarak yang sangat lebar ini menandakan bahwa model menghafal data training tetapi gagal menebak data baru. Validation Accuracy terjebak dan tidak menembus angka 0.32 bahkan grafiknya cenderung menurun di akhir epoch, berbanding terbalik dengan Training Accuracy yang terus naik menembus 0.42. Dari hasil ini, dapat disimpulkan bahwa hidden state berjumlah 512 terlalu besar untuk dataset flicker8k yang relatif kecil dan model mengalami overfitting karena mencoba mengingat setiap detail noise spesifik pada gambar dan teks training sehingga kehilangan kemampuannya untuk mempelajari pola bahasa pada dataset.

- **Perbandingan antar hidden state**

Model dengan *hidden state* 128 menunjukkan performa generalisasi yang jauh lebih baik dibandingkan 512. Kapasitas memori yang lebih kecil memaksa model mempelajari pola kata daripada menghafal kalimat pada dataset.

3.2.4. Perbandingan ukuran hidden state: LSTM

- **Hidden State 128**



Dari tiga variasi layer dengan hidden state 128, seluruhnya menunjukkan pola yang konsisten: Training Loss turun perlahan dan Validation Loss stabil di sekitar 3.6 hingga 3.8 sebelum naik tipis di akhir epoch. Validation Accuracy konsisten mencapai puncak di kisaran 0.35 hingga 0.36 dan relatif stabil sepanjang training. Jarak antara training dan validation loss terjaga dalam batas yang wajar, sehingga model berhasil mempelajari pola bahasa secara umum tanpa menghafal data training secara berlebihan. Ini mengkonfirmasi bahwa hidden state 128 memberikan kapasitas representasi yang cukup untuk dataset Flickr8k.

• Hidden State 512



Dari tiga variasi layer dengan hidden state 512, seluruhnya menunjukkan pola overfitting yang jauh lebih ekstrem dibandingkan hidden state 128. Training Loss turun sangat cepat dan agresif. Pada variasi 1 layer 512 hidden, Training Loss berakhir di 0.8 sementara Validation Loss meroket ke 5.3. Training Accuracy bahkan menembus 0.75, jauh di atas Validation Accuracy yang terjebak di 0.32 hingga 0.34. Jarak yang sangat lebar ini menandakan model dengan 512 hidden state terlalu besar kapasitasnya untuk ukuran dataset Flickr8k, sehingga model cenderung menghafal setiap detail dari data training dan kehilangan kemampuan generalisasi. Satu-satunya pengecualian adalah variasi 2 layer 512 hidden yang berhasil mendapatkan BLEU-4 = 0.2298 dan METEOR = 0.4254 tertinggi dari semua variasi, menunjukkan bahwa kombinasi 2 layer dengan hidden state besar

masih dapat memberikan representasi yang lebih kaya meski risiko overfitting tetap ada.

- **Perbandingan antar hidden state**

Model dengan hidden state 128 secara konsisten menunjukkan generalisasi yang lebih stabil dibandingkan 512 pada hampir semua variasi layer. Namun berbeda dengan RNN, pada LSTM terdapat satu variasi 512 hidden yaitu 2 layer yang justru menghasilkan performa terbaik secara keseluruhan. Hal ini menunjukkan bahwa LSTM berkat mekanisme gate dan cell state lebih mampu memanfaatkan kapasitas hidden state yang besar dibandingkan RNN, meskipun tetap rentan overfitting pada dataset yang kecil.

3.2.5. Perbandingan RNN vs LSTM

Tabel 3.1 Hasil evaluasi RNN

Jumlah Layer	Jumlah Hidden	Hasil
1	128	BLEU-4=0.0630 METEOR=0.2784 Time=2355.5s
2	128	BLEU-4=0.1015 METEOR=0.2877 Time=1483.3s
3	128	BLEU-4=0.1021 METEOR=0.2707 Time=1732.0s
1	512	BLEU-4=0.0723 METEOR=0.2643 Time=1592.9s
2	512	BLEU-4=0.0775 METEOR=0.2677 Time=1542.5s
3	512	BLEU-4=0.0701 METEOR=0.2406 Time=1866.7s

Berdasarkan evaluasi yang telah dilakukan menggunakan metrik-metrik yang ditentukan, model RNN terbaik adalah model dengan 2 layer dan 128 hidden state. Model ini mendapatkan skor METEOR tertinggi yaitu 0.2877, skor BLEU-4 sebesar 0.1015 yang hampir menyamai nilai 3 layer, dan waktu eksekusi

tercepat yaitu 1483,3 detik. Data tabel tersebut mengkonfirmasi mengenai Overfitting dimana model dengan hidden state/memori lebih kecil yaitu 128 memiliki performa lebih baik daripada model dengan hidden state/memori lebih besar yaitu 512. Rata-rata skor BLEU-4 untuk arsitektur 128 hidden jauh lebih baik daripada 512 dan nilai METEOR pada arsitektur 512 terus menurun seiring bertambahnya layer hingga jatuh ke angka 0.2406. Hal ini terjadi karena ukuran 512 hidden state membuat jumlah matriks parameter RNN terlalu banyak dan model cenderung lebih menghafal dataset training dan tidak mempelajari struktur/pola kalimat pada data testing. Oleh karena itu, hidden state yang lebih kecil yaitu 128 menjadi ukuran yang lebih ideal untuk dataset sekecil Flickr8k.

Adapun disini dapat dikenal kelemahan dari Simple RNN yaitu Vanishing Gradient. Pada variasi 128 hidden, penambahan dari 1 layer ke 2 layer memberikan nilai lonjakan drastis pada BLEU-4 yaitu dari 0.0630 menjadi 0.1015 dan hal ini dapat diartikan bahwa 1 layer RNN terlalu dangkal untuk memahami pola bahasa dan layer kedua membantu mengekstrak konteks kata dengan jauh lebih baik. Saat ditambah menjadi 3 layer, nilai BLEU-4 naik tidak terlalu signifikan yaitu dari 0.1015 ke 0.1021 sementara METEOR malah turun dari 0.2877 menjadi 0.2707, serta waktu eksekusinya juga lebih lambat. Inilah bukti dari Vanishing Gradient dimana semakin dalam layer-nya maka semakin sulit pula algoritma Backpropagation untuk memperbarui bobot di layer pertama dan berakibat pada layer ke-3 yang justru menjadi beban komputasi/noise untuk model. Dari data waktu eksekusi, model dengan 1 Layer 128 Hidden memiliki waktu eksekusi paling lama yaitu 2355.5 detik dan hal ini dikarenakan delay I/O dari Google Colab yang kami gunakan untuk eksekusi skrip pengujian.

Tabel 3.2 Hasil evaluasi LSTM

Jumlah Layer	Jumlah Hidden	Hasil
1	128	BLEU-4=0.1393 METEOR=0.3470 Time=1051.0s
2	128	BLEU-4=0.1543 METEOR=0.3682 Time=1158.5s
3	128	BLEU-4=0.1483 METEOR=0.3427 Time=1069.1s
1	512	BLEU-4=0.1463

		METEOR=0.3500 Time=1172.0s
2	512	BLEU-4=0.2298 METEOR=0.4254 Time=1091.6s
3	512	BLEU-4=0.2115 METEOR=0.4089 Time=1090.6s

Berdasarkan evaluasi yang telah dilakukan menggunakan metrik-metrik yang ditentukan, model LSTM terbaik adalah model dengan 2 layer dan 512 hidden state. Model ini mendapatkan skor BLEU-4 tertinggi yaitu 0.2298, skor METEOR tertinggi yaitu 0.4254, dan waktu eksekusi yang kompetitif yaitu 1091.6 detik.

Berbeda dengan RNN, pada LSTM model dengan hidden state lebih besar yaitu 512 justru menghasilkan performa terbaik pada konfigurasi 2 layer. Hal ini menunjukkan bahwa LSTM lebih mampu memanfaatkan kapasitas parameter yang besar berkat mekanisme gate yang mengontrol aliran informasi secara selektif. Meskipun demikian, pada variasi 1 layer dan 3 layer, model dengan hidden state 128 tetap menunjukkan generalisasi yang lebih stabil. Rata-rata BLEU-4 untuk arsitektur 128 hidden adalah 0.1473 sementara untuk 512 hidden adalah 0.1959, namun perbedaan ini sebagian besar didorong oleh performa unggul variasi 2 layer 512 hidden.

Pada variasi 128 hidden, penambahan dari 1 layer ke 2 layer memberikan peningkatan BLEU-4 dari 0.1393 menjadi 0.1543 dan METEOR dari 0.3470 menjadi 0.3682. Saat ditambah menjadi 3 layer, BLEU-4 justru turun menjadi 0.1483 dan METEOR turun menjadi 0.3427. Pada variasi 512 hidden, lonjakan paling signifikan terjadi dari 1 layer ke 2 layer dimana BLEU-4 melonjak drastis dari 0.1463 menjadi 0.2298 dan METEOR dari 0.3500 menjadi 0.4254. Penambahan ke 3 layer kembali menurunkan performa dengan BLEU-4 = 0.2115 dan METEOR = 0.4089.

Pola ini menunjukkan bahwa 2 layer merupakan kedalaman optimal untuk LSTM pada dataset Flickr8k. Berbeda dengan RNN yang mengalami vanishing gradient semakin parah seiring bertambahnya layer, LSTM berhasil memanfaatkan layer kedua untuk mengekstrak representasi yang lebih kaya karena cell state menjaga aliran gradien tetap stabil. Namun penambahan layer ketiga tetap mengakibatkan diminishing returns karena kompleksitas yang ditambahkan tidak sebanding

dengan ukuran dataset yang relatif kecil. Dari data waktu eksekusi, seluruh variasi LSTM berada di kisaran 1051 hingga 1172 detik, jauh lebih konsisten dibandingkan RNN yang memiliki variasi waktu lebih besar.

3.2.6. Perbandingan Keras vs from scratch: RNN

Tabel 3.3 Perbandingan Keras vs from scratch RNN

Model	BLEAU-4	METEOR	Time(s)
Keras	0.2105	0.3343	5.5
Scratch	0.0515	0.1078	1.01

Berdasarkan hasil pengujian pada tabel 3.3, terdapat perbedaan performa yang signifikan antara implementasi SimpleRNN menggunakan framework Keras dan implementasi from scratch. Model Keras menghasilkan nilai BLEU-4 sebesar 0.2105 dan METEOR sebesar 0.3343, sementara implementasi from scratch hanya mampu mencapai BLEU-4 sebesar 0.0515 dan METEOR sebesar 0.1078. Perbedaan ini dapat dijelaskan melalui beberapa faktor:

- Implementasi from scratch menerapkan mekanisme greedy decoding yang dieksekusi secara sekuensial (timestep per timestep) menggunakan CPU. Sebaliknya, modul Keras teroptimasi dengan memanfaatkan akselerasi GPU secara paralel sehingga proses pembaruan bobot dan inferensi berjalan jauh lebih stabil dan akurat.
- Arsitektur sequence seperti RNN rentan terhadap akumulasi floating point precision. Terdapat perbedaan pada kalkulasi `np.tanh()` di NumPy dibandingkan dengan komputasi operasi tensor di TensorFlow. Pada implementasi from scratch, sedikit deviasi presisi di awal timestep akan terakumulasi dan membesar di sepanjang kalimat yang pada akhirnya mempengaruhi distribusi probabilitas softmax saat memprediksi kata-kata berikutnya.
- Implementasi from scratch yang dikembangkan tidak mendukung mekanisme masking. Hal ini menyebabkan RNN from scratch tetap memproses nilai padding (<pad>) sebagai bagian dari input yang valid sehingga merusak representasi hidden state di akhir urutan teks. Berbeda dengan Keras yang memanfaatkan mask dari layer Embedding sehingga model dapat mengabaikan komputasi pada token padding.

Dari aspek waktu eksekusi, implementasi from scratch lebih cepat yaitu membutuhkan waktu 1.01 detik dibandingkan Keras yang membutuhkan 5.5 detik. Hal ini terjadi karena proses inferensi from scratch dilakukan pada subset data yang lebih kecil sehingga implementasi NumPy dapat langsung dieksekusi secara langsung tanpa harus menanggung overhead inisialisasi computational

graph dan sesi eksekusi berat yang merupakan bawaan dari arsitektur framework Keras.

3.2.7. Perbandingan Keras vs from scratch: LSTM

Tabel 3.4 Perbandingan Keras vs from scratch LSTM

Model	BLEAU-4	METEOR	Time(s)
Keras	0.3056	0.4488	9.20
Scratch	0.0715	0.1478	1.12

Terdapat perbedaan performa yang sangat signifikan antara implementasi Keras dan from scratch. Model Keras menghasilkan BLEU-4 = 0.3056 dan METEOR = 0.4488, sementara implementasi from scratch hanya mencapai BLEU-4 = 0.0715 dan METEOR = 0.1478, selisih lebih dari 4x lipat pada kedua metrik. Perbedaan ini disebabkan oleh beberapa faktor. Pertama, implementasi from scratch menggunakan greedy decoding yang berjalan timestep per timestep secara sequential, sementara Keras menggunakan implementasi LSTM yang dioptimasi dengan CUDA kernel dan dapat memanfaatkan GPU secara paralel selama training. Kedua, terdapat kemungkinan perbedaan numerik kecil dalam operasi floating point antara NumPy dan TensorFlow yang dapat terakumulasi selama inferensi panjang. Ketiga, implementasi from scratch tidak mendukung masking sehingga LSTM memproses seluruh token termasuk padding, berbeda dengan Keras yang dapat memanfaatkan mask dari Embedding layer meskipun dalam kasus ini mask juga tidak sepenuhnya berfungsi akibat masalah Concatenate yang telah dibahas. Dari sisi waktu eksekusi, implementasi from scratch justru lebih cepat yaitu 1.12s dibandingkan 9.20s karena inferensi dilakukan untuk subset data yang lebih kecil dan tanpa overhead framework Keras.

3.2.8. Pengaruh panjang maksimum caption terhadap BLEU-4: RNN

Tabel 3.5 Pengaruh panjang maksimum caption terhadap BLEU-4 RNN

Config	Max step	BLEAU
Layer = 2, Hidden= 128	10	0.1573
	20	0.1458
	34	0.1433

Berdasarkan tabel 3.5, hasil evaluasi menunjukkan bahwa skor BLEU-4 tertinggi pada arsitektur SimpleRNN dengan konfigurasi terbaik yaitu 2 Layer dengan 128 Hidden diperoleh pada max_length = 10 dengan nilai 0.1573. Nilai ini terus mengalami penurunan seiring dengan bertambahnya batasan panjang maksimum caption dimana pada max_length = 20 skor turun menjadi 0.1458 dan pada max_length = 34 kembali menyusut menjadi 0.1433. Pola penurunan skor ini dapat dijelaskan melalui dua faktor utama:

- Dari sisi dataset, kalimat ground truth pada dataset Flickr8k rata-rata berdurasi pendek yakni di kisaran 10 hingga 15 kata. Dengan memaksa model menghasilkan teks hingga 34 kata, model cenderung menghasilkan kata-kata tambahan di akhir kalimat yang tidak relevan dengan konteks gambar. Karena BLEU-4 sangat sensitif terhadap presisi n-gram (membandingkan kata tebakan dengan referensi), penambahan kata-kata di akhir kalimat ini akan secara drastis menurunkan skor presisi BLEU-4 model.
- SimpleRNN rentan terhadap masalah Vanishing Gradient. Karena fitur gambar (vektor CNN) hanya diinjeksikan satu kali di awal urutan (pada timestep $t = -1$ dalam arsitektur pre-inject), Simple RNN akan semakin kehilangan memori tentang gambar tersebut seiring bertambahnya timestep. Ketika RNN mencoba menghasilkan kata pada timestep ke-20 atau ke-34, representasi hidden state sudah sangat terdegradasi dan kehilangan konteks visual aslinya, sehingga kata-kata yang dihasilkan di bagian akhir kalimat menjadi tidak akurat. Oleh karena itu, batasan max_length = 10 menjadi sangat optimal karena RNN dipaksa berhenti sebelum ia kehilangan memori jangka pendeknya.

3.2.9. Pengaruh panjang maksimum caption terhadap BLEU-4: LSTM

Tabel 3.6 Pengaruh panjang maksimum caption terhadap BLEU-4 LSTM

Config	Max step	BLEAU
Layer = 2, Hidden= 512	10	0.2457
	20	0.2303
	34	0.2298

Hasil menunjukkan bahwa BLEU-4 tertinggi diperoleh pada max_length = 10 dengan nilai 0.2457, dan menurun seiring bertambahnya panjang maksimum caption. Pada max_length = 20 BLEU-4 turun menjadi 0.2303, dan pada max_length = 34 turun sedikit lagi menjadi 0.2298. Pola ini dapat dijelaskan sebagai berikut: caption di dataset Flickr8k rata-rata memiliki panjang 10 hingga

15 kata. Dengan `max_length = 10`, model dipaksa menghasilkan caption yang singkat dan padat sehingga lebih banyak n-gram yang overlap dengan ground truth yang juga cenderung pendek. Saat `max_length` dinaikkan ke 20 atau 34, model cenderung menghasilkan kata-kata tambahan yang tidak relevan setelah informasi utama sudah disampaikan, sehingga precision n-gram yang dihitung BLEU-4 menurun. Hal ini menunjukkan bahwa untuk dataset Flickr8k, panjang caption yang lebih pendek lebih optimal untuk skor BLEU-4, meskipun secara kualitatif caption yang lebih panjang bisa jadi lebih deskriptif.

4. Kesimpulan dan Saran

4.1. Kesimpulan

Dari berbagai pengujian dan evaluasi yang kami lakukan terhadap implementasi arsitektur *Image Captioning* ini, dapat ditarik beberapa kesimpulan utama:

- Penggunaan pre-trained CNN seperti InceptionV3 sebagai Encoder terbukti efektif dalam implementasi arsitektur Image Captioning. CNN mampu mereduksi dimensi gambar mentah menjadi representasi fitur vektor yang padat dan kaya akan semantik visual. Transfer learning dari model klasifikasi gambar ini sangat mempercepat proses komputasi karena model bahasa Decoder tidak perlu lagi mempelajari ekstraksi fitur dasar seperti garis atau tekstur dan bisa langsung memetakan fitur spasial tingkat tinggi tersebut ke dalam sekuens kata
- Implementasi Simple RNN mampu menghasilkan kalimat deskriptif dasar, namun memiliki keterbatasan fundamental terhadap panjang sekuens. Eksperimen menunjukkan bahwa penggunaan parameter yang terlalu besar justru memperburuk performa model dengan skor BLEU dan METEOR yang menurun karena model mengalami overfitting dan malah menghafal data training pada dataset Flickr8k yang jumlahnya relatif kecil. Selain itu, RNN juga sangat sensitif terhadap nilai `max_length`. Semakin panjang batasan kalimat, skor BLEU-4 juga semakin menurun dibuktikan dengan kelemahan RNN berupa Vanishing Gradient dimana memori tentang fitur gambar yang di-pre-inject di awal semakin memudar di akhir kalimat sehingga menyebabkan penambahan kata yang tidak relevan oleh model.
- Secara keseluruhan, LSTM mengatasi kelemahan memori jangka pendek yang dimiliki oleh Simple RNN. Melalui penerapan 4 gerbang kontrol (Forget, Input, Output gates), LSTM mampu mempertahankan informasi kontekstual dari fitur gambar dengan jauh lebih stabil di sepanjang sekuens kalimat. Meskipun demikian, LSTM juga tetap rentan pada batasan jumlah data dimana arsitektur LSTM yang terlalu dalam atau memiliki hidden state terlalu besar akan tetap mengalami overfitting. Penggunaan `max_length = 10` juga terbukti paling optimal bagi LSTM untuk menghindari penalti n-gram dari metrik BLEU pada target caption Flickr8k yang secara natural memang pendek.
- Terdapat perbedaan skor pada metrik BLEU dan METEOR antara model Keras dan implementasi from scratch. Keunggulan Keras tidak hanya terletak pada optimasi GPU untuk pembaruan bobot tetapi juga kemampuannya dalam menangani token padding melalui mekanisme Masking. Implementasi from scratch rentan terhadap akumulasi distorsi floating point dan tidak mampu mengabaikan iterasi pada padding yang pada akhirnya memengaruhi hidden state secara keseluruhan di tahap akhir.

4.2. Saran

Secara keseluruhan, implementasi arsitektur *Image Captioning* kami telah berhasil dilakukan. Namun, terdapat beberapa saran yang direkomendasikan untuk meningkatkan implementasi dan pengembangan model lanjutan:

- Penerapan mekanisme *Attention*
Arsitektur Pre-inject memiliki kelemahan dimana model perlahan melupakan detail gambar seiring panjangnya kalimat. Penerapan mekanisme *Attention* disarankan agar Decoder RNN dan LSTM dapat secara dinamis memberikan bobot fokus pada bagian spesifik dari piksel gambar yang merupakan representasi spasial CNN yang relevan dengan kata yang sedang diprediksi pada timestep tertentu.
- Beam Search Decoding pada tahap Inferensi/*Forward Pass*
Implementasi kami saat ini murni menggunakan Greedy Search yang hanya memilih satu kata dengan probabilitas tertinggi di setiap langkah tanpa mempertimbangkan konteks kalimat secara keseluruhan. Untuk meningkatkan skor BLEU dan METEOR, disarankan untuk mengimplementasikan Beam Search Decoding. Algoritma ini akan mengeksplorasi K percabangan kalimat alternatif secara paralel sehingga mampu menemukan urutan kata yang probabilitas gabungannya secara global paling masuk akal bukan sekadar optimal secara lokal.
- Penyempurnaan Implementasi From Scratch
Agar implementasi algoritma from scratch dapat memiliki performa lebih tinggi, terdapat dua komponen yang dapat ditambahkan. Pertama, implementasi Masking yaitu menambahkan kondisional pada proses forward dan backward propagation untuk mengabaikan perhitungan loss dan pembaruan gradien pada token <pad>. Selain itu, dapat juga diterapkan pembatasan nilai gradien misal dibatasi di rentang -5 hingga 5 di dalam optimizer untuk mencegah exploding gradient saat proses Backpropagation.
- Penerapan Regularisasi/*Dropout*
Untuk mengisi gap yang jauh antara Training Loss dan Validation Loss, disarankan untuk menambahkan layer Dropout dengan probabilitas misalnya 0.3 hingga 0.5 di antara layer Embedding dan RNN/LSTM serta sebelum layer Dense Output. Hal ini akan memaksa model untuk tidak terlalu bergantung pada neuron tertentu dan meningkatkan kemampuan generalisasi bahasa model.

5. Pembagian Tugas

Tabel 5.1 Pembagian Tugas Tiap Anggota Kelompok

Nama Anggota	NIM	Pembagian Tugas
Jonathan Kenan Budianto	13523139	Implementasi CNN, Laporan bagian CNN
Mahesa Fadhillah Andre	13523140	Implementasi LSTM, Laporan bagian LSTM
Muhammad Dzaky Atha Fadhillah	18223124	Implementasi RNN, Laporan bagian Shared dan RNN

REFERENSI

7. *Convolutional Neural Networks — Dive into Deep Learning 1.0.3 documentation.* (n.d.).

https://d2l.ai/chapter_convolutional-neural-networks/index.html

9. *Recurrent Neural Networks — Dive into Deep Learning 1.0.3 documentation.* (n.d.).

https://d2l.ai/chapter_recurrent-neural-networks/index.html

10.1. *Long Short-Term Memory (LSTM) — Dive into Deep Learning 1.0.3 documentation.* (n.d.).

https://d2l.ai/chapter_recurrent-modern/lstm.html#implementation-from-scratch

10.3. *Deep Recurrent Neural Networks — Dive into Deep Learning 1.0.3 documentation.* (n.d.).

https://d2l.ai/chapter_recurrent-modern/deep-rnn.html

10.7. *Sequence-to-Sequence Learning for Machine Translation — Dive into Deep Learning 1.0.3*

documentation. (n.d.). https://d2l.ai/chapter_recurrent-modern/seq2seq.html

Broadcasting — NUMPY v2.4 manual. (n.d.).

<https://numpy.org/doc/stable/user/basics.broadcasting.html>

Dive into Deep Learning — Dive into Deep Learning 1.0.3 documentation. (n.d.).

https://d2l.ai/chapter_computer-vision/image-captioning.html

GeeksforGeeks. (2025a, July 15). *Image Caption Generator using Deep Learning on Flickr8K*

dataset. GeeksforGeeks.

<https://www.geeksforgeeks.org/image-caption-generator-using-deep-learning-on-flickr8k-dataset/>

GeeksforGeeks. (2025b, July 26). *What is Sparse Categorical Crossentropy.* GeeksforGeeks.

<https://www.geeksforgeeks.org/machine-learning/what-is-sparse-categorical-crossentropy>

/

GeeksforGeeks. (2025c, October 4). *What is Adam Optimizer?* GeeksforGeeks.

<https://www.geeksforgeeks.org/deep-learning/adam-optimizer/>

Numpy.einsum — NumPY v2.1 Manual. (n.d.).

<https://numpy.org/doc/2.1/reference/generated/numpy.einsum.html>

Tanti, M., Gatt, A., & Camilleri, K. P. (2018). Where to put the image in an image caption generator. *Natural Language Engineering*, 24(3), 467–489.

<https://doi.org/10.1017/s1351324918000098>

Team, K. (n.d.-a). *Keras documentation: Image Captioning.*

https://keras.io/examples/vision/image_captioning/

Team, K. (n.d.-b). *Keras documentation: Save, serialize, and export models.*

https://keras.io/guides/serialization_and_saving/

Team, K. (n.d.-c). *Keras documentation: The Functional API.*

https://keras.io/guides/functional_api/

Team, K. (n.d.-d). *Keras documentation: The Sequential model.*

https://keras.io/guides/sequential_model/

Team, K. (n.d.-e). *Keras documentation: Training & evaluation with the built-in methods.*

https://keras.io/guides/training_with_built_in_methods/

Vinyals, O., Toshev, A., Bengio, S., & Erhan, D. (2014, November 17). *Show and Tell: a neural Image caption generator.* arXiv.org. <https://arxiv.org/abs/1411.4555>

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM : 1. Jonathan Kenan Budianto/13523139

2. Mahesa Fadhillah Andre/13523140

3. Muhammad Dzaky Atha Fadhilah/18223124

Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika

Kode Mata Kuliah : IF3270

Nama Mata Kuliah : Pembelajaran Mesin

Judul Tugas/Proposal : Laporan Tugas Besar 2 IF3270 - Pembelajaran Mesin Convolutional Neural Network Dan Recurrent Neural Network

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	Sedang
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Sedang
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	-
Lainnya, Jelaskan: Penggunaan Claude untuk membantu penjelasan kesalahan error sintaks dan penjelasan referensi implementasi model.	Ya	Sedang

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 16 Maret 2026



Jonathan Kenan Budianto
13523139



Mahesa Fadhillah Andre
13523140



Muhammad Dzaky A.F.
18223124